

BeHSe
Lam

32 node

OS1 kolokvijumi

kategorisani zadaci sa kolokvijuma 2006-2023 sa stranice predmeta

pripremili Luka Simić i Aleksa Marković

Jul 2024

Sadržaj

Predgovor	4
Kompajler/linker	5
1. zadatak, prvi kolokvijum, mart 2024.	5
1. zadatak, prvi kolokvijum, april 2024.	6
1. zadatak, prvi kolokvijum, mart 2023.	7
1. zadatak, prvi kolokvijum, avgust 2023.	8
1. zadatak, prvi kolokvijum, jun 2022.	9
2. zadatak, drugi kolokvijum, septembar 2016.	10
2. zadatak, drugi kolokvijum, septembar 2014.	11
3. zadatak, prvi kolokvijum, mart 2012.	12
3. zadatak, prvi kolokvijum, april 2011.	13
3. zadatak, prvi kolokvijum, maj 2011.	14
3. zadatak, drugi kolokvijum, maj 2008.	15
3. zadatak, drugi kolokvijum, maj 2006.	15
Stranična organizacija	17
3. zadatak, prvi kolokvijum, mart 2024.	17
2. zadatak, prvi kolokvijum, april 2024.	18
3. zadatak, prvi kolokvijum, mart 2023.	19
1. zadatak, prvi kolokvijum, maj 2023.	19
2. zadatak, prvi kolokvijum, maj 2023.	21
3. zadatak, prvi kolokvijum, avgust 2023.	22
2. zadatak, prvi kolokvijum, mart 2022.	23
3. zadatak, prvi kolokvijum, maj 2022.	24
2. zadatak, prvi kolokvijum, jun 2022.	24
3. zadatak, kolokvijum, jun 2020.	25
2. zadatak, kolokvijum, jul 2020.	26
3. zadatak, drugi kolokvijum, maj 2019.	28
3. zadatak, drugi kolokvijum, jun 2019.	29
3. zadatak, drugi kolokvijum, april 2018.	29
3. zadatak, drugi kolokvijum, jun 2018.	31
3. zadatak, drugi kolokvijum, april 2017.	32
3. zadatak, drugi kolokvijum, jun 2017.	33
3. zadatak, drugi kolokvijum, maj 2016.	34
3. zadatak, drugi kolokvijum, septembar 2016.	35
3. zadatak, drugi kolokvijum, maj 2015.	36
3. zadatak, drugi kolokvijum, septembar 2015.	37
3. zadatak, drugi kolokvijum, april 2014.	38
3. zadatak, drugi kolokvijum, septembar 2014.	39
2. zadatak, prvi kolokvijum, april 2013.	40

4. zadatak, drugi kolokvijum, april 2013.	40
4. zadatak, drugi kolokvijum, septembar 2013.	41
2. zadatak, prvi kolokvijum, mart 2012.	41
4. zadatak, drugi kolokvijum, maj 2012.	42
2. zadatak, prvi kolokvijum, septembar 2012.	42
4. zadatak, drugi kolokvijum, septembar 2012.	43
2. zadatak, prvi kolokvijum, april 2011.	44
2. zadatak, prvi kolokvijum, maj 2011.	44
4. zadatak, drugi kolokvijum, maj 2011.	45
2. zadatak, prvi kolokvijum, april 2010.	46
4. zadatak, drugi kolokvijum, maj 2010.	47
4. zadatak, drugi kolokvijum, maj 2009.	48
3. zadatak, prvi kolokvijum, april 2008.	48
4. zadatak, drugi kolokvijum, maj 2008.	49
5. zadatak, drugi kolokvijum, maj 2008.	50
2. zadatak, prvi kolokvijum, april 2007.	50
4. zadatak, drugi kolokvijum, maj 2007.	51
4. zadatak, prvi kolokvijum, april 2006.	52
5. zadatak, drugi kolokvijum, maj 2006.	52
Kontinualna alokacija	53
2. zadatak, prvi kolokvijum, mart 2024.	53
2. zadatak, prvi kolokvijum, avgust 2023.	53
1. zadatak, kolokvijum, avgust 2021.	54
2. zadatak, drugi kolokvijum, jun 2019.	55
2. zadatak, drugi kolokvijum, april 2018.	56
2. zadatak, drugi kolokvijum, jun 2018.	57
2. zadatak, drugi kolokvijum, april 2017.	58
2. zadatak, drugi kolokvijum, maj 2015.	59
2. zadatak, drugi kolokvijum, septembar 2015.	60
2. zadatak, drugi kolokvijum, april 2014.	62
3. zadatak, drugi kolokvijum, april 2013.	63
3. zadatak, drugi kolokvijum, septembar 2012.	63
4. zadatak, drugi kolokvijum, septembar 2011.	64
5. zadatak, drugi kolokvijum, maj 2010.	64
5. zadatak, drugi kolokvijum, maj 2009.	65
5. zadatak, drugi kolokvijum, maj 2007.	65
Segmentna organizacija	67
2. zadatak, prvi kolokvijum, mart 2023.	67
1. zadatak, prvi kolokvijum, mart 2022.	68
3. zadatak, prvi kolokvijum, mart 2022.	69
1. zadatak, prvi kolokvijum, maj 2022.	69
1. zadatak, kolokvijum, jun 2021.	70
2. zadatak, kolokvijum, oktobar 2020.	71
3. zadatak, kolokvijum, septembar 2020.	72
2. zadatak, prvi kolokvijum, mart 2014.	73
2. zadatak, prvi kolokvijum, april 2014.	73
2. zadatak, prvi kolokvijum, septembar 2011.	74
3. zadatak, prvi kolokvijum, april 2009.	74
4. zadatak, drugi kolokvijum, maj 2006.	75
Segmentno-stranična organizacija	76
2. zadatak, prvi kolokvijum, maj 2022.	76
3. zadatak, prvi kolokvijum, jun 2022.	76
2. zadatak, kolokvijum, avgust 2020.	78
2. zadatak, prvi kolokvijum, septembar 2014.	79

2. zadatak, prvi kolokvijum, mart 2013.	79
2. zadatak, prvi kolokvijum, septembar 2013.	79
2. zadatak, prvi kolokvijum, maj 2012.	80
3. zadatak, prvi kolokvijum, april 2007.	80
3. zadatak, prvi kolokvijum, april 2006.	81
Dinamičko učitavanje	82
3. zadatak, prvi kolokvijum, april 2024.	82
3. zadatak, prvi kolokvijum, maj 2023.	82
2. zadatak, drugi kolokvijum, maj 2019.	83
3. zadatak, drugi kolokvijum, septembar 2011.	84
3. zadatak, drugi kolokvijum, maj 2009.	85
3. zadatak, drugi kolokvijum, maj 2007.	86
Preklopi	88
1. zadatak, kolokvijum, jul 2021.	88
2. zadatak, drugi kolokvijum, jun 2017.	88
2. zadatak, drugi kolokvijum, maj 2016.	90
3. zadatak, drugi kolokvijum, septembar 2013.	91
3. zadatak, drugi kolokvijum, maj 2012.	93
3. zadatak, drugi kolokvijum, maj 2011.	94
3. zadatak, drugi kolokvijum, maj 2010.	95

Predgovor

Svrha ove zbirke jeste da objedini sve do sada dostupne kolokvijume iz Operativnih sistema 1 iz nekoliko razloga:

- Na kolokvijumima iz OS1 je dozvoljena literatura, i studenti često pristupaju tome tako što odštampaju kolokvijume i njihova rešenja pa se na kolokvijumu snalaze kroz taj odštampani materijal ako vide neki sličan zadatak. Ovom zbirkom postiže se organizacija takvih materijala radi lakšeg snalaženja na kolokvijumu i ušteda u papirima potrebnim za štampanje svih tih rokova.
- Zadaci su kategorisani po oblastima i sličnosti kako bi se lakše vežbali određeni tipovi zadataka na kolokvijumu.

Poglavlja zbirke idu istim redom kao kolokvijumsko gradivo na predmetu, konkretno:

1. Kompajler/linker, stranična organizacija, kontinualna alokacija, segmentna organizacija, segmentno-stranična organizacija, dinamičko učitavanje, preklopi
2. Prekidi, sistemski pozivi, interfejs niti, promena konteksta, sinhronizacija procesa, baferi, proizvođač/potrošač
3. Ulaz/izlaz, komandna linija, fajl sistem

Raspored ovih oblasti po kolokvijumima se retko menja, ali se takva situacija može desiti (usled predviđenih ili nepredviđenih okolnosti). U tom slučaju, proverite koje oblasti dolaze na kolokvijumu sa predmetnim asistentima. Bitno je napomenuti da se ovaj redosled u prošlosti menjao, pa ako se neki zadatak ranije pojavljivao na prvom kolokvijumu to ne mora da znači da je ta oblast i tekuće godine na prvom kolokvijumu. U suštini, **da li neki zadatak može doći na kolokvijumu određujete prvenstveno na osnovu toga iz koje je oblasti, a ne na osnovu toga na kom se kolokvijumu pojavio u prošlosti.**

Greške u formatiranju i kategorizaciji su sigurno prisutne. Ukoliko ih uočite, možete se javiti jednom od autora ili poslati *pull request* na repozitorijum projekta: <https://github.com/siwiki/OS-kolokvijumi>. Ispravljeni dokumenti će biti dostupni u [Releases](#). Svaka pomoć je dobrodošla.

Srećno na kolokvijumu.

Autori

Kompajler/linker

1. zadatak, prvi kolokvijum, mart 2024.

- [Postavka](#)
- [Rešenje](#)

Dat je sadržaj dva asemblerska fajla (**a.asm** i **b.asm**) za 32-bitni procesor picoRISC. Direktivom **export** asembleru se nalaže da navedene simbole (definisane dalje u tom fajlu) “izveze” kao simbole dostupne za spoljno vezivanje, dok se direktivom **import** “uvoze” navedeni simboli definisani u nekom drugom fajlu (ovom direktivom ti simboli su deklarirani u datom fajlu, ali sa nepoznatom vrednošću). Nakon prevođenja (asembliranja), linkeru se zadaje da spoji dva fajla dobijena prevođenjem fajlova **a.asm** i **b.asm**, tim redom. Linker slaže segmente u virtuelni adresni prostor redom kojim na njh nailazi počev od virtuelne adrese 0, s tim da svaki nov segment počinje od početka nove stranice veličine 4 KB (adresibilna jedinica je bajt).

a.asm	b.asm
<pre> export main, a import max seg data a dd 1, 2, 3, 4, 5 endseg seg text main: load r1,a push r1 load r1,a+4 push r1 call max pop r1 pop r1 ret endseg </pre>	<pre> export max seg text max: 2load r1,[sp+8] 2load r2,[sp+12] 1cmp r1,r2 1jle L0001 1load r0,r1 1ret L0001: 1load r0,r2 1ret endseg </pre>

1. Koje virtuelne adrese dodeljuje linker simbolima **main**, **a** i **max** tokom povezivanja (napisati ih heksadecimalno)? Obrazložiti odgovor.
2. Ukoliko instrukcija **call** na vrh steka stavlja samo PC, koje tri 32-bitne vrednosti se nalaze na vrhu steka prilikom ulaska u potprogram **max** pri izvršavanju procesa pokrenutog nad navedenim povezanim programom? Vrednosti navesti heksadecimalno u poretку kojim su stavlјene na stek – poslednju onu na vrhu steka.
3. Ukoliko se primenjuje stranična organizacija memorije, kolika je interna fragmentacija za sadržaj ova dva fajla (koliko je neiskorišćene memorije koja je alocirana za logičke segmente u ovim fajlovima)? Obrazložiti odgovor.

Rešenje

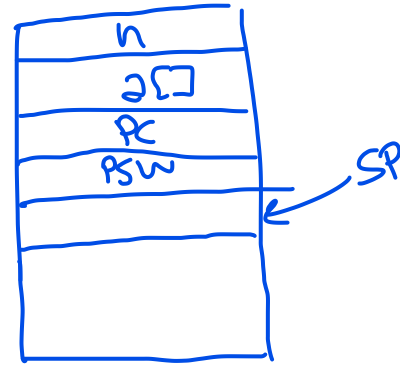
1. Sva tri logička segmenta su vrlo mala, očigledno manja od jedne stranice (videti odgovor pod c). Sva tri navedena simbola se preslikavaju u relativne pomeraје 0 svojih segmenta, tako da se simboli preslikavaju u sledeće virtuelne adrese:
a: 0, main: 0x1000 (4K), max: 0x2000 (8K)
2. Pri pozivu potprograma **max**, pozivalac (**main**) na vrh steka stavlja sadržaj prva dva 32-bitna elementa niza **a**, tj. 1 i 2, a potom vrednost PC-a tokom izvršavanja instrukcije **call**. Tako se na vrhu steka nalaze redom vrednosti: 1, 2, 0x1020 (adresa prve instrukcije iza instrukcije **call** je 4K+32, jer instrukcije zaključno sa instrukcijom **call** zauzimaju 8 32-bitnih reči).
3. Prvi segment (**data**) koristi pet 32-bitnih reči (20 bajtova). Drugi segment (**text**) koristi 11 32-bitnih reči (44 bajta). Treći segment takođe koristi 11 reči (44 bajta). Ukupno je iskorišćeno 108 bajtova od tri cele stranice (12KB). Interna fragmentacija uključuje sav ovaj neiskorišćen prostor veličine 12KB - 108B.

1. zadatak, prvi kolokvium, april 2024.

- Postavka
- Rešenje

Dat je sadržaj jednog izvornog fajla sa C kodom.

```
int a[5] = {1, 2, 3, 4, 5};
int n = 0;
int max(int a[], int n) {
    if (n <= 1) return a[0];
    int m = max(a + 1, n - 1);
    return (a[0] > m) ? a[0] : m;
}
```



1. Napisati asemblerski kod za 32-bitni procesor picoRISC, sa sintaksom direktiva pokazanim na predavanjima, kakav bi prevodilac mogao da napravi prevođenjem ovog fajla. Logički segment se na assembleru deklarise direktivom seg uz koju ide kvalifikator za tip segmenta (text, bss ili data); npr. seg text. Stek raste ka nižim adresama, SP ukazuje na prvu slobodnu lokaciju, adresibilna jedinica je bajt, a instrukcija poziva potprograma na steku čuva PC i PSW tim redom.
2. Ukoliko assembler koristi relativno adresiranje (u odnosu na PC) kod prevođenja instrukcije call kad god može, koja vrednost će biti generisana u drugoj 32-bitnoj reči mašinske instrukcije call koja radi rekurzivan poziv (napisati heksadecimalno)?
3. Da li se navedeno relativno adresiranje za instrukciju call može koristiti i za poziv funkcije max iz drugog logičkog segmenta koji se nalazi u drugom izvornom asemblerskom fajlu? Kratko obrazložiti odgovor.

Rešenje

```
1. seg data
    a dd 1, 2, 3, 4, 5
endseg

seg bss
    n dd 0
endseg

seg text
max:    load r1, [sp+12] ; r1 := a
        load r2, [sp+16] ; r2 := n
        load r3, #1    ; if (n<=1)
        cmp r2, r3
        jg  L0001
        load r0, [r1] ; return a[0]
        jmp max_END
L0001:  sub r2, r2, r3 ; r1 := n-1
        push r2
        load r4, #4
        add r5, r1, r4
        push r5
        call max ; r0 := max(a+1, n-1)
        pop r5
        pop r2
        load r1, [sp+12] ; r1 := a
        load r2, [r1] ; r2 := a[0]
        cmp r2, r0 ; (a[0] > m)?
        jle max_END
        load r0, r2
max_END: ret
endseg
```

2. 0xFFFFF3B0 ($-20 \cdot 4$ decimalno)
3. Logičke segmente po virtuelnom adresnom prostoru raspoređuje linker. Prilikom prevođenja navedenog poziva, assembler ne može koristiti relativno adresiranje, jer ne zna vrednost adrese odredišta skoka, pa mora koristiti apsolutno (memorijsko direktno) adresiranje. (Teorijski, ovo bi bilo moguće kada bi linker bio sposoban da izračunava konstantne izraze koje ostavlja assembler za nerazrešene vrednosti pomeraja u instrukcijama, pa assembler može da ostavi ovakav izraz u prevedenom fajlu koji potom rešava linker poznajući sve konstante u njemu, ali to nije praktično, jer složenost implementacije ne opravdava dobit.)

1. zadatak, prvi kolokvijum, mart 2023.

- Postavka
- Rešenje

Dat je sadržaj jednog izvornog fajla sa C kodom.

```
int a[256], n;
int max_a(int n) {
    if (n <= 1) return a[0];
    int m = max_a(n-1);
    return (a[n - 1] > m) ? a[n - 1] : m;
}
```

1. Napisati assembly kod za 32-bitni procesor picoRISC, sa sintaksom direktiva pokazanom na predavanjima, kakav bi prevodilac mogao da napravi prevođenjem ovog fajla. Logički segment se na assembleru deklarise direktivom `seg` uz koju ide kvalifikator za tip segmenta (`text`, `bss` ili `data`); npr. `seg text`. Stek raste ka nižim adresama, SP ukazuje na poslednju popunjenu lokaciju, adresibilna jedinica je bajt, a instrukcija poziva potprograma na steku čuva PC i PSW tim redom.
2. Za svaku labelu unutar assembly koda prevoda funkcije `max_a` odrediti pomeraj (razliku) njene vrednosti (adrese u koju se preslikava) u odnosu na vrednost labela koja označava prvu instrukciju ove funkcije. Vrednosti pisati decimalno i obrazložiti rezultat.
3. Ukoliko se logički segmenti nastali prevodom ovog fajla alociraju redom, jedan odmah iza drugog u virtuelnom adresnom prostoru, svaki logički segment poravnat na početak stranice veličine 4 KB, a prvi od njih počinje od virtuelne adrese 0, odrediti virtuelne adrese u koje se preslikavaju svi simboli u assembly prevodu (labela za sve definisane podatke i one koje označavaju instrukcije prevodne funkcije `max_a`). Kratko obrazložiti rezultat.

Rešenje

256 · 4 = 1024 2¹⁰

```
1.  seg bss
    a dd 256 dup 0
1024 n dd 0
    endseg
    seg text
212 max_a: 1 load r1, [sp+8]      ; r1 := n
          2 load r2, #1          ; if (n<=1)
          1 cmp r1, r2
          2 jg L0001
          2 load r0, a            ; return a[0]
          1 ret
213, 10 L0001: 1 sub r1, r1, r2   ; r1 := n-1
          1 push r1              ; r0 := max_a(n-1)
          2 call max_a
          1 pop r1
          2 load r2, #2          ; r1 := a[n-1]
          1 shl r1, r1, r2
          2 load r1, a[r1]
          1 cmp r1, r0            ; (a[n-1]>m)?
          2 jle L0002
```

```

12 76
1load r0, r1
L0002: 1ret
endseg

```

2. Instrukcije koje u sebi sadrže zapis neposrednog operanda, pomeraja ili apsolutne adrese zauzimaju dve 32-bitne reči (8 bajtova), ostale zauzimaju po jednu (4 bajta).

L0001 = max_a + 4*8 + 2*4 = max_a + 40

L0002 = L0001 + 4*8 + 6*4 = L0001 + 56 = max_a + 96

3. a = 0
 n = a+256*4 = 1024
 max_a = 4*1024 = 4096
 L0001 = max_a + 40 = 4136
 L0002 = max_a + 96 = 4192

1. zadatak, prvi kolokvijum, avgust 2023.

- Postavka
- Rešenje

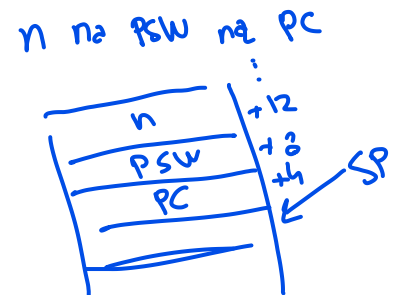
Na assembleru 32-bitnog procesora picoRISC napisati prevod dole date rekurzivne C funkcije f. Pretpostavke:

- vraćanje vrednosti tipa strukture iz funkcije implementira se tako što se u funkciju u registru R0 prenosi adresa lokacije u koju treba upisati povratnu vrednost (u R0 je pokazivač na datu strukturu u koju pozivalac prihvata vraćenu vrednost);
- stek raste ka nižim adresama, a SP ukazuje na poslednju popunjenu lokaciju;
- prilikom poziva potprograma instrukcijom call procesor na stek najpre stavlja PSW, pa potom PC (svaki je 32-bitni); adresibilna jedinica je bajt.

```

struct S { int a, b; }
struct S f(int n) {
    struct S s;
    if (n == 0) s.a = s.b = 1;
    else s = f(n-1);
    return s;
}

```



Rešenje

Osnovno rešenje bez optimizacije:

```

f:
    load r1, #1
    load r2, #8
    sub sp, sp, r2 ; struct S s
    load r2, [sp+4*4] ; r2 = n
    and r2, r2, r2 ; r2 == 0?
    jnz f_1001
    store r1, [sp+0] ; s.a = 1
    store r1, [sp+4] ; s.b = 1
    jmp f_1002
f_1001:
    ? push r0
    ? load r0, sp ; s = f(n-1)
    sub r2, r2, r1 ; r2 = n-1
    push r2
    call f
    pop r2
    pop r0

```

ovo 8 ovo je o lai

f_1002:

```

load r2, [sp+0] ; return s
store r2, [r0+0]
load r2, [sp+4]
store r2, [r0+4]
load r2, #8
add sp, sp, r2
ret

```

3. unutar ciklusa je cikla

Rešenje sa tzv. *Named Return Value* optimizacijom (NRVO, za objašnjenje videti [OOP slajdove](#) počev od 362, konkretno slajd 373):

f:

```

load r1, #1
load r2, [sp+2*4] ; r2 = n
and r2, r2, r2 ; r2 == 0?
jnz f_1001
store r1, [r0+0] ; s.a = 1
store r1, [r0+4] ; s.b = 1
ret

```

f_1001:

```

sub r1, r2, r1 ; r1 = n-1
push r1
call f
pop r1
ret

```

1. zadatak, prvi kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Posmatraju se četiri odvojena i nezavisna slučaja sadržaja tri fajla sa C kodom, a.h, a.c i b.c, data u tabeli i označena rednim brojem 1 do 4. U svakom od tih slučajeva se sprovodi sledeći postupak:

- najpre se prevodi fajl a.c; ako prevodilac prijavi grešku, postupak se prekida;
- ako u prevodenju fajla a.c nije bilo grešaka, prevodi se fajl b.c; ako prevodilac prijavi grešku, postupak se prekida;
- ako ni u prevodenju fajla b.c nije bilo grešaka, povezuju se fajlovi a.o i b.o.

Za svaki od datih slučajeva precizno objasniti grešku koju će prijaviti prevodilac, odnosno linker, ukoliko grešaka ima: navesti ko će prijaviti grešku i kakvog tipa je ta greška (i zbog kog simbola). Ukoliko grešaka nema, to naglasiti ("nema grešaka").

#	a.h	a.c	b.c
1	<i>unreputu</i> int f ();	#include "a.h" int f ();	#include "a.h" int main () { return f(); }
2	extern int x;	#include "a.h" int x = 0;	#include "a.h" int main () { return x; }
3	int x = 0;	#include "a.h" int x = 0;	extern int x; <i>2</i> int main () { return x; }
4	int x = 0;	#include "a.h" extern int x;	#include "a.h" int main () { return x; }

Rešenje

1. Greška u povezivanju jer simbol f nije definisan.
2. Nema grešaka.
3. Greška u prevodenju a.c zbog višestruke definicije identifikatora x.

4. Greška u povezivanju jer je simbol `x` višestruko definisan (u fajlu `a.o` i `b.o`).

2. zadatak, drugi kolokvijum, septembar 2016.

- Postavka
- Rešenje

Potrebno je implementirati proceduru `resolveSymbols` koja se koristi u drugom prolazu jednog linkera. Ova procedura obrađuje jedan ulazni `.obj` fajl i treba da razreši adresna polja mašinskih instrukcija koja koriste simbole koje taj fajl uvozi. Ulazni i izlazni fajl su memorijski preslikani, tehnikom virtuelne memorije, tako da se njihov sadržaj može jednostavno posmatrati kao sadržaj memorije procesa. Na početak memorijski preslikanog sadržaja ulaznog `.obj` fajla ukazuje prvi, a na početak tog prepisanog sadržaja u izlaznom (`.exe`) fajlu ukazuje drugi argument ove procedure; pre poziva ove procedure, linker je već prepisao sadržaj binarnog prevedenog koda (bez zaglavlja) svih ulaznih `.obj` fajlova u sadržaj izlaznog fajla.

Na samom početku ulaznog fajla nalazi se zaglavlje. Svi pomeraji (*offset*, odnosno relativne adrese) u njemu izraženi su u jedinicama `sizeof(char)=1`, a veličine su `unsigned long` (skraćeno `ulong`). Sadržaj početka zaglavlja je, redom, sledeći:

- jedan `ulong` koji sadrži pomeraj početka binarnog prevedenog koda u `.obj` fajlu u odnosu na početak celog sadržaja tog fajla (zapravo sadrži veličinu celog zaglavlja iza koga sledi binarni prevedeni kod);
- jedan `ulong` koji sadrži ukupan broj simbola koji se uvoze (n);
- n redom poređanih parova: ime simbola koji se uvozi (niz znakova proizvoljne dužine, završen znakom `'\0'`), iza koga sledi jedan `ulong` koji predstavlja pomeraj prvog nerazrešenog adresnog polja u mašinskoj instrukciji koje treba da sadrži vrednost razrešene adrese tog simbola; takva polja su dalje ulančana u jednostruku listu, tako da svako polje sadrži pomeraj narednog takvog polja za isti simbol, s tim da vrednost pomeraja 0 označava kraj liste (poslednje takvo polje za taj simbol); ovi pomeraji su relativni u odnosu na početak prevedenog binarnog koda unutar sadržaja `.obj` fajla.

Linker poseduje tabelu simbola čija operacija:

```
•ulong SymbolTable::resolveSymbol(char* symbol);
```

vrća pomeraj (relativnu adresu) u odnosu na početak izlaznog fajla (.exe) u koji se dati simbol prevodi, ukoliko on postoji u tabeli, a 0 ako ga nema. Grešku nedefinisanog simbola treba obraditi pozivom funkcije:

```
int errorSymbolUndefined(char* symbol);
```

Ova funkcija ispisuje korisniku poruku o nedefinisanom datom simbolu i vraća -1, što u tom slučaju treba da vrati i funkcija `resolveSymbols`. U slučaju uspeha, funkcija `resolveSymbols` treba da vrati 0.

```
int resolveSymbols (char* inputObj, char* output);
```

Rešenje

```
typedef unsigned long ulong;
const ulong OffsBinaryStartOffset = 0,
        OffsNumOfImportedSymbols = OffsBinaryStartOffset + sizeof(ulong),
        OffsImportedSymbols = OffsNumOfImportedSymbols + sizeof(ulong);
```

```
int resolveSymbols(char* inputObj, char* output) {
    ulong binaryStartOffs = *(ulong*)(inputObj + OffsBinaryStartOffset);
    char* binaryStart = inputObj + binaryStartOffs;
    ulong numOfSymbols = *(ulong*)(inputObj + OffsNumOfImportedSymbols);
    char* symbol = inputObj + OffsImportedSymbols;
    for (ulong i=0; i<numOfSymbols; i++) {
        ulong addr = SymbolTable::resolveSymbol(symbol);
        if (addr==0) return errorSymbolUndefined(symbol);
        ulong symbolLen = strlen(symbol)+1;
        ulong fieldOffs = *(ulong*)(symbol+symbolLen);
        for (; fieldOffs>0; fieldOffs**=(ulong*)(binaryStart+fieldOffs))
```

```

        *(ulong*)(output+fieldOffs) = addr;
        symbol = symbol+symbolLen+sizeof(fieldOffs);
    }
    return 0;
}

```

2. zadatak, drugi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Kod za prvi prolaz nekog linkera dat je u nastavku. (U prvom prolazu linker samo prikuplja izvezene simbole, a ne proverava i da li su svi uvezeni simboli i definisani; tu proveru radi prilikom obrade uvezanih simbola u drugom prolazu.) Popuniti izostavljene delove koda oznacene komentarima `/**1***/` do `/**5***/`.

```

void Linker::firstPass () {
    this->status = OK;
    this->binarySize = /**1***/; //BinarySize of processed output
    for (FileReader::reset(); !FileReader::isDone(); FileReader::next()) {
        ObjectFile* objFile = FileReader::currentObjectFile();
        if (objFile==0) {
            Output::error("Fatal internal error: null pointer exception.");
            exit(-1);
        }
        for (objFile.reset(); !objFile.isDone(); objFile.nextSymbol()) {
            Symbol* sym = objFile->getCurrentSymbol();
            if (sym==0) {
                /**2***/
            }
            if (sym->getKind() == Symbol::export) {
                int offset = sym->getOffset(); // offset in objFile
                offset += /**3***/;
                int status =
                    SymbolTable::addSymDef(sym->getName(), offset, objFile->getName());
                if (status==-1) {
                    /**4***/
                    this->status = ERROR;
                }
            }
        }
        int size = objFile->getBinarySize();
        /**5***/
    }
}

```

Rešenje

```

void Linker::firstPass () {
    this->status = OK;
    this->binarySize = 0;
    for (FileReader::reset(); !FileReader::isDone(); FileReader::next()) {
        ObjectFile* objFile = FileReader::currentObjectFile();
        if (objFile==0) {
            Output::error("Fatal internal error: null pointer exception.");
            exit(-1);
        }
        for (objFile.reset(); !objFile.isDone(); objFile.nextSymbol()) {
            Symbol* sym = objFile->getCurrentSymbol();

```

```

    if (sym==0) {
        Output::error("Fatal internal error: null pointer exception.");
        exit(-1);
    }
    if (sym->getKind() == Symbol::export) {
        int offset = sym->getOffset();
        offset += this.binarySize;
        int status =
            SymbolTable::addSymDef(sym->getName(), offset, objFile->getName());
        if (status==-1) {
            Output::errorMsg("Symbol %d already defined.", sym->getName());
            this.status = ERROR;
        }
    }
}
}
int size = objFile->getBinarySize();
this.binarySize += size;
}
}

```

3. zadatak, prvi kolokvijum, mart 2012.

- [Postavka](#)
- [Rešenje](#)

Neki C kompajler ne koristi uobičajeni pristup alokaciji lokalnih promenljivih i argumenata funkcija, pa ih ne smešta na kontrolni procesorski stek (onaj na kome procesor implicitno čuva povratnu adresu prilikom skoka u potprogram mašinskom instrukcijom `call`). Umesto toga, ovaj kompajler koristi sledeću tehniku za alokaciju i pristup lokalnim promenljivim i argumentima funkcija.

Za svaku lokalnu promenljivu i argument funkcije definisanu u programu, kompajler statički (u vreme prevođenja) alokira jedan (i samo jedan) deo memorije za smeštanje jedne instance te promenljive/argumenta. **Pristup toj promenljivoj ili argumentu u kodu funkcije je onda rešen statičkim vezivanjem, direktnim memorijskim adresiranjem te jedne i uvek iste lokacije koja je poznata u vreme prevođenja.** Da bi se podržali rekurzivni pozivi, svakoj takvoj promenljivoj i argumentu pridružena je jedna (i samo jedna) globalna struktura podataka koju kompajler organizuje. Ta struktura podataka implementira LIFO protokol; drugim rečima, svakoj lokalnoj promenljivoj i argumentu pridružen je jedan LIFO stek na kome se čuvaju prethodne vrednosti te promenljive/argumenta u slučaju ugnježđenih i rekurzivnih poziva. Prilikom poziva funkcije, tekuća vrednost iz statičke lokacije za svaku lokalnu promenljivu/argument se smešta na vrh njegovog sopstvenog steka, sa koga se restaurira prilikom izlaska iz pozvane funkcije. Ovo radi kod koga generiše kompajler na odgovarajućim mestima.

1. Na assembleru napisati prevod koji bi napravio ovaj kompajler za sledeću rekurzivnu C funkciju:

```

int f (int n) {
    if (n==0) return 0;
    else return f(n-1)+1;
}

```

Kod koji generiše kompajler za smeštanje vrednosti lokalne promenljive/argumenta koji je alokiran na adresi simbolički označenoj sa **X** predstaviti sledećim makroima (ti makroi biće zamenjeni odgovarajućim kodom koji održava pomenuti stek pridružen promenljivoj **X** i koji ima određeni isti oblik parametrizovan adresom **X**, a koji ovde nije od interesa):

```

push(X)
pop(X)

```

2. Objasniti šta je problem koji se mora prevazići ako se za ovaj kompajler želi napraviti višenitno jezgro poput školskog jezgra i kako se taj problem može prevazići u implementaciji jezgra, a bez izmene kompajlera.

Rešenje

1.

```

f:    load r0,[n] ; if (n==0)
      cmp r0,#0
      jne else
      ret ; r0==0, return 0
else: dec r0 ; f(n-1)
      push(n)
      store [n],r0
      call f
      pop(n)
      inc r0 ; return f(n-1)+1
      ret

```

2. Problem je to što je svakoj lokalnoj promenljivoj i argumentu pridružen jedan i samo jedan globalni i statički alocirani stek. Zbog toga taj stek može da „prati“ samo instance lokalnih promenljivih samo jedne niti, a ne više njih. Na primer, jedna nit bi mogla da pozove funkciju *f* sa datim argumentom *n* i druga učini to isto i uporedo, pokvarivši i tekuću vrednost i stek starih vrednosti za *n* prve niti. Za potrebe uporednih niti neophodno je imati zaseban skup instanci lokalnih promenljivih i argumenata pridružen svakoj niti. Prema tome, ceo skup statički alociranih lokalnih promenljivih i argumenata, zajedno sa njima pridruženim LIFO strukturama (pojedinačnim stekovima), mora da bude deo konteksta niti, što znači da se mora čuvati i restaurirati iz PCB prilikom promene konteksta niti, na sličan način kako se čuvaju i restauriraju registri procesora, odnosno analogno odvajanju zasebnog kontrolnog steka za svaku nit.

3. zadatak, prvi kolokvijum, april 2011.

- [Postavka](#)
- [Rešenje](#)

U nekom 32-bitnom RISC procesoru svi registri su 32-bitni, adrese su 32-bitne, a adresibilna jedinica je bajt. Prevodilac za jezik C za taj procesor povratnu vrednost funkcije prenosi kroz registar *r0*. Data je implementacija standardne funkcije *setjmp()* za taj procesor i taj prevodilac:

```

int setjmp (jmp_buf buf) {
    asm {
        load r0,-1*4[sp]; // r0:=buf
        store psw,0*4[r0];
        store r1,1*4[r0];
        store r2,2*4[r0];
        ...
        store r31,31*4[r0];
        pop r1;           // pop return address to save the caller's context
        store sp,32*4[r0]; // save sp from the caller's context
        store r1,33*4[r0]; // save pc from the caller's context
        push r1;          // push return address back to the stack
        load r1,1*4[r0];  // restore r1
        clr r0; // r0:=0
    }
}

```

Napisati implementaciju standardne funkcije *longjmp()*:

```
void longjmp (jmp_buf buf, int val);
```

Rešenje

```

void longjmp (jmp_buf buf, int val) {
    asm {

```

```

    load r0,-2*4[sp]; // r0:=val
    and r0,r0,r0;    // fix r0 if it is zero
    jnz continue
    load r0,#1

continue:
    load r1,-1*4[sp]; // r1:=buf

    load psw,0*4[r1];
    load r3,3*4[r1]; // restore regs r3..r31
    load r4,4*4[r1];
    ...
    load r31,31*4[r1];

    load sp,32*4[r1]; // restore sp
    load r2,33*4[r1]; // restore pc and push it on the stack
    push r2
    load r2,2*4[r1]; // restore r2
    load r1,1*4[r1]; // restore r1
    ret;             // now return
}
}

```

3. zadatak, prvi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

U nekom 32-bitnom RISC procesoru svi registri su 32-bitni, adrese su 32-bitne, a adresibilna jedinica je bajt. Prevodilac za jezik C za taj procesor povratnu vrednost funkcije prenosi kroz registar r0. Data je jedna nekorektna implementacija standardnih funkcija `setjmp()` i `longjmp()` za taj procesor i taj prevodilac:

```

int setjmp (jmp_buf buf) {
    asm {
        clr r0; // r0:=0
        push r1;
        load r1,-2*4[sp]; // r1:=buf
        store sp,0*4[r1];
        store psw,1*4[r1];
        store r2,2*4[r1];
        store r3,3*4[r1];
        ...
        store r31,31*4[r1];
        store pc,32*4[r1];
        pop r1
    }
}

void longjmp (jmp_buf buf, int val) {
    asm {
        load r0,-2*4[sp]; // r0:=val
        and r0,r0,r0;    // fix r0 if it is zero
        jnz continue
        load r0,#1
    }
continue:
    load r1,-1*4[sp]; // r1:=buf
    load sp,0*4[r1];
}

```

```

    load psw, 1*4[r1];
    load r2, 2*4[r1];
    load r3, 3*4[r1];
    ...
    load r31, 31*4[r1];
    load pc, 32*4[r1];
}
}

```

Posmatrati upotrebu ovih funkcija, na primer kao u operaciji `dispatch()` školskog jezgra i precizno objasniti šta je problem sa ovom implementacijom.

Rešenje

Problem je u tome što se povratak iz funkcije `setjmp()`, nakon skoka iz funkcije `longjmp()`, oslanja na sačuvanu vrednost povratne adrese originalnog pozivaoca funkcije `setjmp()`, kao i na sačuvanu vrednost `r1`. Međutim, nakon povratka iz „običnog“ poziva funkcije `setjmp()`, u kontekstu pozivajuće funkcije može se pozvati neka druga funkcija ili dogoditi prekid, koji će onda prepisati te vrednosti povratne adrese i sačuvanog `r1` koje su ostale iznad vrha steka.

Drugim rečima, prilikom povratka iz „običnog“ poziva funkcije `setjmp()`, povratna adresa na koju se oslanja povratak nakon skoka iz `longjmp()` ostala je iznad vrha steka i može se lako dogoditi da bude „pregažena“ nekom drugom vrednošću. Tada povratak iz `setjmp()` nakon skoka iz `longjmp()` neće biti korektan. Jedan primer je kod funkcije `dispatch()` školskog jezgra, a koji nakon poziva `setjmp()` ima pozive drugih funkcija (klase `Scheduler`).

3. zadatak, drugi kolokvijum, maj 2008.

- [Postavka](#)
- [Rešenje](#)

Objasniti na koji način prevodilac i/ili linker podržava to što je u nekim jezicima dozvoljeno da se u različitim modulima pojave definicije više entiteta sa istim imenom, ali u različitim tzv. prostorima imena (engl. *namespace*). Na primer, u jeziku C++, statički podatak-član sa istim imenom `m` (a koji po definiciji ima eksterno vezivanje, odnosno izvozi se kao simbol iz modula) može sasvim regularno da se definiše u različitim klasama `X` i `Y`.

Rešenje

Iako ovakvi entiteti imaju isto *nekvalifikovano* ime u datom programu, oni imaju različita potpuno *kvalifikovana* imena (engl. *fully qualified name*) koja se sastoje od pune staze imena njihovih okružujućih prostora imena. Na primer, statički podatak-član `m` klase `X` ima potpuno kvalifikovano ime `X::m`, dok istoimeni član klase `Y` ima potpuno kvalifikovano ime `Y::m`; slično važi za ugneždene prostore imena (npr. `P::Q::R::S::t`). Da bi se omogućilo definisanje istoimenih simbola u različitim prostorima imena, prevodilac jednostavno kao simbol u `.obj` fajlu definiše potpuno kvalifikovano ime, a ne nekvalifikovano ime, tako da linker vidi to jednoznačno puno ime. Drugim rečima, linker ne poznaje pojam prostora imena niti kvalifikovanog imena, za njega su svi simboli jednostavni nizovi znakova koji moraju biti jednoznačno definisani. Prevodilac obezbeđuje ovu jednoznačnost korišćenjem punih kvalifikovanih imena kao imena simbola koje ostavlja linkeru.

3. zadatak, drugi kolokvijum, maj 2006.

- [Postavka](#)
- [Rešenje](#)

Precizno objasniti zašto je potrebno linkeru navesti da kao svoj proizvod treba da napravi statičku biblioteku (`lib`), a ne izvršni program (`exe`)? Precizno objasniti razlike između postupka i rezultata pravljenja ove dve vrste proizvoda povezivanja.

Rešenje

Statička biblioteka sadrži zaglavlje sa spiskom simbola koje izvozi i uvozi, i samo telo biblioteke (kod), drugim rečima, proizvod je istog oblika kao i proizvod prevođenja, dok izvršni fajl sadrži telo (kod) i zaglavlje u kome se nalazi adresa prve instrukcije koja treba da se izvrši. Zbog ove razlike u proizvodima, linkeru je potrebna informacija

šta da napravi. Osim toga, u samom postupku razrešavanja simbola, prilikom pravljenja izvršnog fajla, postojanje nedefinisanog a referisanog simbola se nakon prvog prolaza prijavljuje kao greška. U slučaju pravljenja biblioteke, ovakav slučaj je dozvoljen.

Stranična organizacija

3. zadatak, prvi kolokvijum, mart 2024.

- [Postavka](#)
- [Rešenje](#)

Neki sistem sa straničnom organizacijom memorije i alokacijom stranica na zahtev (*demand paging*) omogućava logičko deljenje (logičkih) segmenata virtuelne memorije, ali tako da najviše dva procesa mogu da dele jedan segment. U svakom od tih procesa deljeni logički segment može da zauzima različite pozicije u virtuelnom adresnom prostoru (ali iste veličine). Segment jednog procesa opisan je objektom klase `SegDesc`. PMT procesa kom pripada dati segment dat je u polju `pmt`, dok je broj prve stranice tog segmenta dat u polju `startPage`. Ukoliko je segment deljen, segment drugog procesa sa kojim je dati segment deljen dat je u polju `sharedSeg` (ima vrednost *null* ako segment nije deljen).

Data polimorfna operacija `SegDesc::initPage` po potrebi učitava, odnosno inicijalizuje datu stranicu u dati okvir prethodno alociran za tu stranicu (ona ne menja PMT), u zavisnosti od vrste segmenta; ova funkcija vraća status izvršenja (0 za uspeh). PMT je predstavljen klasom `PMT`, čija funkcija `getFrame` vraća broj okvira za datu stranicu (0 ako stranica nije alocirana), a funkcija `setFrame` postavlja broj okvira za datu stranicu u PMT.

Implementirati funkciju `SegDesc::allocPage` koja se poziva prilikom obrade stranične greške i koja treba da obezbedi da data stranica za dati PMT procesa koji je izazvao straničnu grešku bude alocirana, učitana u memoriju i postavljen njen ulaz u PMT; uzeti u obzir slučaj kada je stranica deljena i već alocirana od strane procesa sa kojim je deljena. Provera da data stranica pripada datom segmentu je već obavljena. Pretpostavlja se da su polja za prava pristupa u PMT uvek inicijalizovana za sve stranice alociranih segmenata, čak i ako stranice nisu alocirane. Ova operacija treba da vraća status izvršenja (0 za uspeh).

```
typedef Frame uint16;
typedef Page uint32;
Frame getFreeFrame();

class SegDesc {
public:
    int allocPage(Page page);
    ...
protected:
    virtual int initPage(Page, Frame);
    ...
private:
    Page startPage;
    PMT* pmt;
    SegDesc* sharedSeg = nullptr;
    ...
};

class PMT {
public:
    Frame getFrame(Page) const;
    void setFrame(Page, Frame);
    ...
};
```

Rešenje

```
int SegDesc::allocPage(Page page) {
    Frame frame = 0;
    if (this->sharedSeg) {
        Page otherPage = page - this->startPage + this->sharedSeg->startPage;
        frame = this->sharedSeg->pmt->getFrame(otherPage);
    }
```

```

    }
    int status = 0;
    if (!frame) {
        frame = getFreeFrame();
        status = this->initPage(page, frame);
        if (!status) return status;
    }
    this->pmt->setFrame(page, frame);
    return status;
}

```

2. zadatak, prvi kolokvijum, april 2024.

- [Postavka](#)
- [Rešenje](#)

Neki sistem sa straničnom organizacijom virtuelne memorije i dohvatanjem stranica na zahtev (*demand paging*) ima PMT u dva nivoa, s tim da operativni sistem i PMT drugog nivoa takođe alokira na zahtev, tek kada proces prvi put ispravno pristupi nekoj stranici iz opsega te PMT drugog nivoa. Zato su inicijalno svi ulazi u PMT prvog nivoa *null*. Širina virtuelne adrese je 32 bita, stranica ima 4K adresibilnih jedinica, a PMT oba nivoa isti broj ulaza.

1. Dole su date deklaracije tipova za stranice prvog i drugog nivoa. Implementirati operaciju `getPageDesc` koja za datu virtuelnu adresu vraća descriptor stranice u PMT kojoj ta virtuelna adresa pripada. U slučaju da PMT1 drugog nivoa nije alocirana, treba je alocirati. Za alokaciju memorije za svoje strukture kernel koristi svoju internu funkciju `kmalloc` koja je na raspolaganju. U slučaju da nema prostora za PMT1, funkcija `getPageDesc` treba da vrati *null*.
2. Ukratko, ali precizno objasniti šta operativni sistem treba da uradi prilikom obrade stranične greške u opisanom sistemu.

```

const uint32 offsetw = 12;
const uint32 page1w = 10;
const uint32 PMT0_size = 1024;
const uint32 PMT1_size = 1024;

typedef uint32 PageDesc;
typedef PageDesc PMT1[PMT1_SIZE];
typedef PMT1* PMT0[PMT0_SIZE];
void* kmalloc(size_t size);

inline PageDesc* getPageDesc(PMT0 pmt, void* vaddr);

```

Rešenje

1.

```

inline PageDesc* getPageDesc (PMT0 pmt, void* vaddr) {
    uint32 page = (uint32) vaddr >> offsetw;
    uint32 page0 = page >> page1w;
    uint32 page1 = page & ~((uint32) -1 << page1w);
    PMT1* pmt1 = pmt[page0];
    if (!pmt1) pmt1 = (PMT1*) kmalloc(sizeof(PMT1));
    if (!pmt1) return nullptr;
    pmt[page0] = pmt1;
    return &pmt1[page1];
}

```
2. Operativni sistem treba najpre da proveri da li tražena virtuelna adresa koja je generisala straničnu grešku pripada nekom alociranom logičkom segmentu virtuelne memorije, na osnovu strukture deskriptora logičkih segmenata (bez pristupa PMT). Ako ne pripada, proces je prestupio u pristupu memorije i operativni sistem može da ga ugasi. Ako pripada, treba da alokira okvir za smeštanje tražene stranice, u zavisnosti od vrste logičkog segmenta učita tu stranicu sa diska, inicijalizuje sadržaj okvira ili ništa od toga (videti objašnjenja na

predavanjima), a onda, korišćenjem funkcije realizovane pod a), dohvati deskriptor stranice (i usput alokira PMT drugog nivoa ako je potrebno) i postavi ga na odgovarajuću vrednost (broj okvira i prava pristupa).

3. zadatak, prvi kolokvijum, mart 2023.

- [Postavka](#)
- [Rešenje](#)

Neki sistem sa straničnom organizacijom memorije koristi tehniku kopiranja pri upisu (*copy on write*). Deskriptor stranice u PMT je veličine 32 bita, s tim da najnižih 16 bita sadrži broj okvira, a najviša tri bita koduju prava pristupa *RWX* tim redom (bit *R* je najviši bit).

Za evidenciju svih okvira raspoložive operativne memorije sistem koristi niz `frames`; *i*-ti element ovog niza odgovara okviru broj *i*. Vrednost ovog elementa označava broj stranica koje dele isti okvir pre razdvajanja pri upisu; ako je ta vrednost 0, okvir je slobodan; ako je ta vrednost 1, okvir koristi samo jedna stranica jednog procesa; ako je ta vrednost veća od 1, taj okvir dele stranice različitih procesa.

Kada obrađuje hardverski izuzetak zbog pokušaja nedozvoljenog upisa, nakon provere da adresirana stranica pripada segmentu koji je logički dozvoljen za upis i da treba raditi kopiranje pri upisu, jezgro poziva operaciju `copyOnWrite` sa argumentom koji pokazuje na deskriptor stranice u koju je pokušao upis. Ta funkcija tada treba da iskopira sadržaj okvira u nov okvir i da stranicu koja je referencirana preusmeri u taj okvir i dozvoli upis u tu stranicu. Ostale stranice koje su koristile isti okvir, čak i ako je to samo jedna preostala, se ne menjaju. Kada se pokuša upis u poslednju preostalu stranicu koja je delila taj okvir, ova funkcija će samo dozvoliti upis u nju (nema potrebe kopirati je, jer je jedina).

Implementirati funkciju: `int copyOnWrite(uint32* pd)`. Na raspolaganju je sledeće:

- `getFreeFrame()`: vraća broj prvog slobodnog okvira; ukoliko takvog nema, vraća 0;
- `copyFrame(uint16 oldFrame, uint16 newFrame)`: kopira sadržaj iz okvira broj `oldFrame` u okvir broj `newFrame`.

Ako slobodnog okvira nema, funkcija `copyOnWrite` ne treba ništa da menja, samo da vrati status -1. U slučaju uspeha, ona treba da vrati status 0. Tipovi `uint16` i `uint32` predstavljaju neoznačene 16-bitne, odnosno 32-bitne cele brojeve, respektivno.

Rešenje

```
typedef PgDsc uint32;
typedef Frame uint16;
inline void setPgRW(PgDsc* pd) {
    *pd |= ((PgDsc)3) << 30;
}
int copyOnWrite(PgDsc* pd) {
    Frame oldFrame = (Frame)(*pd);
    if (frames[oldFrame] > 1) {
        Frame newFrame = getFreeFrame();
        if (newFrame == 0) return -1;
        frames[newFrame] = 1;
        frames[oldFrame]--;
        *pd = newFrame;
        copyFrame(oldFrame, newFrame);
    }
    setPgRW(pd);
    return 0;
}
```

1. zadatak, prvi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sa straničnom organizacijom memorije virtuelni i fizički adresni prostor su iste veličine, po 16 MB, adresibilna jedinica je bajt, a veličina stranice je 4 KB. U donjoj tabeli data je mapa fizičkog adresnog prostora, s tim da prva data oblast memorije (BIOS) počinje na fizičkoj adresi 0, a svaka sledeća data oblast počinje odmah iza prethodne, na prvoj sledećoj novoj stranici. Iznad datih oblasti nalazi se prostor na raspolaganju za procese.

Operativni sistem preslikava navedene oblasti fizičke memorije u virtuelni adresni prostor svakog procesa, i to na isto mesto, na najviše adrese virtuelnog adresnog prostora procesa, ali u obrnutom redosledu od datog (BIOS oblast je na najvišim virtuelnim adresama, MMIO odmah ispod njega itd.).

1. U prazna polja date tabele za svaku oblast upisati heksadecimalne vrednosti njene veličine, početne fizičke adrese i početne virtuelne adrese u prostoru svakog procesa.

Simbolički naziv	Vrsta i sadržaj	Veličina	Veličina (hex)	Početna fiz. adr. (hex)	Početna v. adr. (hex)
BIOS	ROM, BIOS	8 KB		0	
MMIO	Memorijski preslikan ulaz-izlaz	512 KB			
SRAM	Statički perzistentni RAM, konfiguracija	4 KB			
KTXT	DRAM, kod kernela	64 KB			
KDATA	DRAM, podaci kernela	436 KB			

2. Implementirati funkciju `mapKernelVMem` koju jezgro poziva prilikom inicijalizacije memorijskog konteksta novog procesa i koja treba da inicijalizuje deo PMT procesa koji se odnosi na opisane oblasti. Na raspolaganju je funkcija `initPMTEntry` koja inicijalizuje jedan ulaz u datom PMT za datu stranicu, tako da se ta stranica preslikava u dati okvir. Prava pristupa definišu se poslednjim parametrom na način kao i u POSIX-u, pri čemu su definisane simboličke konstante `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` sa istim značenjima kao i tamo, ali i `PROT_USR` koja dozvoljava definisan pristup stranici i u korisničkom režimu. Tip `size_t` je celobrojni tip koji može da prihvati vrednosti virtuelne ili fizičke adrese.

```
void mapKernelVMem (PMT* pmt);
initPMTEntry(PMT*, size_t page, size_t frame, int prot);
```

Rešenje

1.

```
const size_t BIOS_PADDR = 0x0;
const size_t BIOS_SIZE = 0x2000;
const size_t BIOS_VADDR = 0xFFE000;
const size_t MMIO_PADDR = 0x2000;
const size_t MMIO_SIZE = 0x80000;
const size_t MMIO_VADDR = 0xF7E000;
const size_t SRAM_PADDR = 0x82000;
const size_t SRAM_SIZE = 0x1000;
const size_t SRAM_VADDR = 0xF7D000;
const size_t KTXT_PADDR = 0x83000;
const size_t KTXT_SIZE = 0x10000;
const size_t KTXT_VADDR = 0xF6D000;
const size_t KDATA_PADDR = 0x93000;
const size_t KDATA_SIZE = 0x6D000;
const size_t KDATA_VADDR = 0xF00000;
```
2.

```
static const size_t PG_SIZE = 0x1000;
static const size_t BIOS_SZ_PG = BIOS_SIZE/PG_SIZE+(BIOS_SIZE%PG_SIZE!=0);
static const size_t BIOS_START_PG = BIOS_VADDR/PG_SIZE;
static const size_t BIOS_START_FR = BIOS_PADDR/PG_SIZE;

static const size_t MMIO_SZ_PG = MMIO_SIZE/PG_SIZE+(MMIO_SIZE%PG_SIZE!=0);
static const size_t MMIO_START_PG = MMIO_VADDR/PG_SIZE;
static const size_t MMIO_START_FR = MMIO_PADDR/PG_SIZE;
```

```

static const size_t SRAM_SZ_PG = SRAM_SIZE/PG_SIZE+(SRAM_SIZE%PG_SIZE!=0);
static const size_t SRAM_START_PG = SRAM_VADDR/PG_SIZE;
static const size_t SRAM_START_FR = SRAM_PADDR/PG_SIZE;

static const size_t KTXT_SZ_PG = KTXT_SIZE/PG_SIZE+(KTXT_SIZE%PG_SIZE!=0);
static const size_t KTXT_START_PG = KTXT_VADDR/PG_SIZE;
static const size_t KTXT_START_FR = KTXT_PADDR/PG_SIZE;

static const size_t KDATA_SZ_PG =KDATA_SIZE/PG_SIZE+(KTXT_SIZE%PG_SIZE!=0);
static const size_t KDATA_START_PG = KDATA_VADDR/PG_SIZE;
static const size_t KDATA_START_FR = KDATA_PADDR/PG_SIZE;

void mapKernelVMem(PMT* pmt) {
    for (size_t i=0; i<BIOS_SZ_PG; i++)
        initPMTEntry(pmt, BIOS_START_PG+i, BIOS_START_FR+i, PROT_EXEC|PROT_READ);
    for (size_t i=0; i<BIOS_SZ_PG; i++)
        initPMTEntry(pmt, MMIO_START_PG+i, MMIO_START_FR+i, PROT_READ|PROT_WRITE);
    for (size_t i=0; i<BIOS_SZ_PG; i++)
        initPMTEntry(pmt, SRAM_START_PG+i, SRAM_START_FR+i, PROT_READ|PROT_WRITE);
    for (size_t i=0; i<BIOS_SZ_PG; i++)
        initPMTEntry(pmt, KTXT_START_PG+i, SRAM_START_FR+i, PROT_EXEC);
    for (size_t i=0; i<BIOS_SZ_PG; i++)
        initPMT(pmt, KDATA_START_PG+i, KDATA_START_FR+i, PROT_READ|PROT_WRITE);
}

```

2. zadatak, prvi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

Implementirati funkciju `handleProtectionFault` koju jezgro operativnog sistema poziva pri obradi hardverskog izuzetka generisanog zbog bilo kakvog prestupa u pravima na zahtevanu operaciju sa memorijom. Prvi parametar ove funkcije ukazuje na deskriptor tekućeg procesa, drugi sadrži broj stranice koja je adresirana, a treći parametar koduje zahtevanu operaciju u bitima koji su maskirani odgovarajućim simboličkim konstantama kao u POSIX-u. Ova funkcija treba da vrati kod greške dat sledećim simboličkim konstantama u sledećim slučajevima:

- `ERR_ILLEGAL_ADDRESS` u slučaju da adresirana stranica ne pripada nijednom logičkom segmentu virtuelnog adresnog prostora datog procesa;
- `ERR_ILLEGAL_OP` u slučaju da je proces izvršio operaciju nedozvoljenu za tu stranicu;
- `ERR_ILLEGAL_USR_OP` u slučaju da je proces izvršio (inače dozvoljenu) operaciju za stranicu koja nije dostupna u korisničkom režimu rada procesora;
- u slučaju da se radi o okidaču za kopiranje pri upisu (*copy on write*) treba da pozove funkciju `copyOnWrite` koja će uraditi ovo kopiranje i da vrati ono što i ova funkcija vrati.

Na raspolaganju su sledeće funkcije:

- `SegDsc* getSegDesc(Process*, size_t page)`: za datu stranicu datog procesa vraća deskriptor logičkog segmenta, odnosno `null` ako stranica ne pripada nijednom;
- `int getSegProt(SegDsc*)`: vraća bite prava pristupa u datom deskriptoru logičkog segmenta, kodovane u bitima kojima se pristupa kao u POSIX-u, simboličkim konstantama `PROT_READ`, `PROT_WRITE`, `PROT_EXEC` sa istim značenjima kao i tamo, ali i `PROT_USR` koja označava dozvoljen definisan pristup stranici i u korisničkom režimu;
- `PMT* getPMT(Process*)`: vraća PMT za dati proces; uvek uspeva;
- `PageDsc* getPageDesc(PMT*, size_t page)`: vraća deskriptor date stranice u datom PMT; uvek uspeva za validne argumente;
- `int getPageProt(PageDsc*)`: vraća bite prava pristupa u datom deskriptoru stranice u PMT, kodovane u bitima na isti način kao i za funkciju `getSegProt`;
- `int copyOnWrite(PageDsc*)`: obavlja kopiranje date stranice; vraća kod greške u slučaju neuspeha.

```
int handleProtectionFault(Process* proc, size_t page, int op);
```

Rešenje

```
int handleProtectionFault(Process* proc, size_t page, int op) {
    SegDsc* sd = getSegDesc(proc, page);
    if (!sd) return ERR_ILLEGAL_ADDRESS;

    int lprot = getSegProt(sd);
    if ((lprot & op) == 0) return ERR_ILLEGAL_OP;
    if ((lprot & PROT_USR) == 0) return ERR_ILLEGAL_USR_OP;

    PMT* pmt = getPMT(proc);
    PgDsc* pd = getPageDesc(pmt, page);
    int prot = getPageProt(pd);

    if ((lprot & PROT_WRITE) && !(prot & PROT_WRITE)) return copyOnWrite(pd);

    return ERR_UNKNOWN; // Should never occur
}
```

3. zadatak, prvi kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

Neki sistem sa straničnom orgnizacijom virtuelne memorije obrađuje događaje vezane za alokaciju i zamenu stranica polimorfno, pozivom odgovarajućih funkcija dinamičkim vezivanjem preko pokazivača koji se nalaze u poljima odgovarajućih struktura. Konkretnije, u sledećim strukturama postoje sledeća polja koja su pokazivači na funkcije za obradu sledećeg:

- U strukturi PCB polje `handleMemSegFault` je pokazivačkog tipa `void*(PCB* pcb, uint32 page)` i pokazuje na funkciju koju treba pozvati ako proces sa datim PCB adresira datu stranicu broj `page` koja ne pripada nijednom alociranom logičkom segmentu virtuelne memorije; ova funkcija će dalje ugasiti proces ili uraditi nešto drugo ako je redefinisana.
- U strukturi `SegDsc`, koja predstavlja deskriptor jednog logičkog segmenta, polja `coldLoad` i `hotLoad` su istog pokazivačkog tipa `void*(SegDsc* sd, PCB* pcb, PageDsc* pd, uint32 page, void* frame)` i pokazuju na funkcije koje treba pozvati kada datu stranicu broj `page` datog procesa `pcb` kojoj odgovara deskriptor logičkog segmenta `sd` i deskriptor stranice `pd` treba učitati odnosno inicijalizovati u dati okvir `frame` kad se ta stranica koristi i učitava prvi (`coldLoad`) odnosno svaki naredni put (`hotLoad`).

U jednom bitu deskriptora stranice `PageDsc` u PMT sistem vodi evidenciju o tome da li je data stranica ranije već bila korišćena i alocirana ili nije. Za održavanje ovog bita na raspolaganju su sledeće funkcije:

- `bool isFirstAccess(PageDsc*)`: vraća `true` ako je u datom deskriptoru stranice stranica označena tako da joj se ranije nije pristupalo (pa zato prvo obraćanje zahteva učitavanje sa `coldLoad`);
- `void setAccessed(PageDsc*)`: u datom deskriptoru stranice označava stranicu kao ranije već korišćenu (pa zato svako naredno učitavanje zahteva poziv `hotLoad`).

Na raspolaganju su i sledeće funkcije:

- `void* allocFrame()`: alocira jedan slobodan okvir memorije i vraća njegovu adresu. Ova funkcija brine o zameni stranica ukoliko slobodnog okvira nema (izabira stranicu- žrtvu za izbacivanje, po potrebi je snima), tako da uvek vraća validnu nenultu vrednost;
- `SegDsc* getSegDsc(PCB*, uint32 page)`: vraća pokazivač na deskriptor logičkog segmenta za dati proces i datu stranicu; ukoliko data stranica ne pripada nijednom alociranom logičkom segmentu, vraća `null`;
- `PageDsc* getPageDesc(PCB* pcb, uint32 page)`: vraća deskriptor date stranice u PMT datog procesa. Ova funkcija uvek vraća validnu nenultu vrednost, čak i ako je PMT organizovana u više nivoa, jer po potrebi alocira PMT višeg nivoa za traženu stranicu.

Implementirati sledeću funkciju:

```
void handlePageFault(PCB* pcb, uint32 page);
```

koju kernel poziva kada obrađuje hardverski izuzetak zbog stranične greške koju je izazvao dati proces adresiranjem date stranice. (Ovde se radi samo o straničnoj greški, ne o nedozvoljenoj operaciji.)

Rešenje

```
void handlePageFault(PCB* pcb, uint32 page) {
    SegDsc* sd = getSegDesc(pcb, page);
    if (!sd) {
        pcb->handleMemSegFault(pcb, page);
        return;
    }
    void* frame = allocFrame();
    PageDsc* pd = getPageDesc(pcb, page);
    if (isFirstAccess(pd)) {
        setAccessed(pd);
        sd->coldLoad(sd, pcb, pd, page, frame);
    } else
        sd->hotLoad(sd, pcb, pd, page, frame);
}
```

2. zadatak, prvi kolokvijum, mart 2022.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu adresibilna jedinica je bajt, virtuelni i fizički adresni prostori su iste velične 4 GB, a stranica je veličine 4 KB. Sistem koristi straničenje u dva nivoa. PMT oba nivoa imaju isti broj ulaza i svaki ulaz zauzima po 32 bita. U jednom ulazu u PMT drugog nivoa najniža tri bita zadaju prava pristupa do stranice (RWX), a biti do njih sadrže broj okvira (ili 0 ako stranica nije alocirana). PMT prvog nivoa sadrži celu početnu adresu PMT-a drugog nivoa (PMT ne mora biti poravnata na stranicu).

Da bi obradio sistemske pozive u kojima se neki parametar zadaje kao pokazivač (virtuelna adresa u adresnom prostoru pozivajućeg procesa), operativni sistem mora da konvertuje datu virtuelnu u fizičku adresu, jer se kod kernela izvršava u režimu bez preslikavanja adresa. Implementirati funkciju koja obavlja ovu konverziju (vraća *null* ako virtuelna adresa ne pripada alociranom delu virtuelnog adresnog prostora); prvi parametar ove funkcije je pokazivač na PMT prvog nivoa pozivajućeg procesa. Pretpostaviti da je `uint32` deklarisan kao neoznačeni 32-bitni celobrojni tip. Napisati sve potrebne deklaracije za tipove tabele prvog (PMT0) i drugog (PMT1) nivoa.

```
void* v2pAddr(PMT0 pmt, void* vaddr);
```

Rešenje

```
const uint32 offsetw = 12;
const uint32 page1w = 10;
const uint32 PMT0_size = 1024;
const uint32 PMT1_size = 1024;

typedef uint32 PMT1[PMT1_SIZE];
typedef uint32 PMT0[PMT0_SIZE];

void* v2pAddr(PMT0 pmt, void* vaddr) {
    uint32 page = (uint32)vaddr >> offsetw;
    uint32 offset = (uint32)vaddr & ~((uint32)-1 << offsetw);
    uint32 page0 = page >> page1w;
    uint32 page1 = page & ~((uint32)-1 << page1w);
    uint32* pmt1 = (uint32*)pmt[page0];
```



```

    if (!pmt1) {
        return nullptr;
    }
    uint32 dsc = pmt1[page1];
    uint32 frame = dsc >> 3;
    if (!frame) {
        return nullptr;
    }
    uint32 paddr = (frame << offsetw) + offset;
    return (void*)paddr;
}

```

3. zadatak, prvi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi straničnu organizaciju memorije sa stranicom veličine 128 KB. Adresibilna jedinica je 16-bitna reč, virtuelni adresni prostor je veličine 8 GB, a fizički adresni prostor je veličine 2 GB. PMT je u jednom nivou. U deskriptoru stranice u PMT najviša dva bita koduju prava pristupa, dok je u najnižim bitima broj okvira; ako stranica nije alocirana, ceo deskriptor ima vrednost 0.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svih polja.
2. Implementirati funkciju `sharePages` koja se koristi u implementaciji sistemskog poziva kojim se zahteva logičko deljenje niza od `cnt` susednih stranica počev od stranice `pg1` procesa čija je PMT data parametrom `pmt1` sa nizom od `cnt` susednih stranica počev od stranice `pg2` procesa čija je PMT data parametrom `pmt2`, pri čemu su pomenute stranice drugog procesa već alocirane u okvire, a pomenute stranice prvog procesa nisu, već treba da se dele sa drugim. Ukoliko drugi proces nije alocirao sve pomenute svoje stranice ili je prvi proces već alocirao neku od svojih pomenutih stranica, ova funkcija treba da vrati grešku (negativnu vrednost). Tip `long` je širine 64 bita, tip `int` 32 bita, a tip `short` 16 bita.

```

typedef unsigned short ushort;
typedef ushort PMT[PMT_SIZE];

int sharePages(PMT pmt1, ushort pg1, PMT pmt2, ushort pg2, ushort cnt);

```

Rešenje

1. VA: Page(16):Offset(16); PA: Frame(14):Offset(16)
2.

```

ushort MAXPAGE = -1;
int sharePages(PMT pmt1, ushort pg1, PMT pmt2, ushort pg2, ushort cnt) {
    if (MAXPAGE-cnt <= pg1 || MAXPAGE-cnt <= pg2) {
        return -1; // Overflow
    }
    for (ushort i = 0; i < cnt; i++) {
        if (pmt1[pg1 + i] != 0 || pmt2[pg2 + i] == 0) {
            return -1;
        }
    }
    for (ushort i = 0; i < cnt; i++) {
        pmt1[pg1 + i] = pmt2[pg2 + i];
    }
    return 0;
}

```

2. zadatak, prvi kolokvijum, jun 2022.

- [Postavka](#)

- [Rešenje](#)

U nekom sistemu jedan objekat klase `FreeFrames`, čiji je interfejs dat dole, vodi evidenciju o slobodnim okvirima memorije predviđene za smeštanje stranica procesa. Objekat ove klase inicijalizuje se zadavanjem kontinualnog memorijskog prostora od `size` susednih stranica počev od adrese `mem`. Veličina stranice je `PAGE_SIZE` (u jedinicama `sizeof(char)`). Operacija `alloc` vraća jedan slobodan okvir i izbacuje ga iz evidencije slobodnih okvira. Operacija `free` dodaje slobodni okvir na koga ukazuje argument u evidenciju slobodnih okvira.

Implementirati klasu `FreeFrames` u potpunosti tako da operacije `alloc` i `free` budu najefikasnije moguće ($\mathcal{O}(1)$).

```
class FreeFrames {
public:
    FreeFrames(void* mem, size_t size);
    void* alloc();
    void free(void* frame);
};
```

Rešenje

```
class FreeFrames {
public:
    FreeFrames(void* mem, size_t size);
    void* alloc();
    void free(void* frame);
private:
    char* head;
};

void FreeFrames::FreeFrames (void* mem, size_t size) {
    char* frame = head = (char*)mem;
    for (int i = 0; i < size - 1; i++) {
        frame = (*(char**)frame) = frame + PAGE_SIZE;
    }
    *(char**)frame = 0;
}

inline void* FreeFrames::alloc () {
    void* ret = head;
    if (head) {
        head = *(char**)head;
    }
    return ret;
}

inline void FreeFrames::free (void* frame) {
    *(char**)frame = head;
    head = (char*)frame;
}
```

3. zadatak, kolokvijum, jun 2020.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 8TB (terabajta) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 4KB. PMT (*page map table*) je organizovana u dva nivoa, s tim da je broj ulaza u PMT prvog nivoa dva puta manji od broja ulaza u PMT drugog nivoa.

Posmatra se jedan proces kreiran nad sledećim programom sa greškom:

```

#define M 4
#define N 0x1000
int src[M][N], dst[M][N];

int main (int argc, const char* argv[]) {
    for (int i=0; i<=M; i++)
        for (int j=0; j<=N; j++)
            dst[i][j] = src[i][j];
}

```

pod sledećim pretpostavkama:

- tip `int` je veličine 32 bita; promenljive `i` i `j` je prevodilac formirao kao automatske promenljive na steku;
 - logički segment za kod ovog programa veličine je jedne stranice i nalazi se na dnu virtuelnog adresnog prostora procesa (na najnižim adresama), a segment za stek je veličine 32 stranice i nalazi se odmah iznad segmenta za kod;
 - nizovi `src` i `dst` smešteni su jedan odmah iza drugog, tim redom, u isti logički segment memorije alociran za statičke podatke koji se nalazi iznad segmenta za stek; ovaj segment je onoliki koliko je najmanje stranica potrebno za smeštanje ovih nizova i dozvoljen je za upis;
 - operativni sistem ne učitava nijednu stranicu pri kreiranju procesa, već sve stranice učitava tek na zahtev (*demand paging*).
1. Prikazati logičku strukturu virtuelne adrese i označiti veličinu svakog polja.
 2. Za koje vredosti promenljivih `i` i `j` i pristup do kog od dva niza `src` i `dst` će procesor prvi put generisati izuzetak koji će operativni sistem smatrati kao prestup (grešku) procesa? Obrazložiti.
 3. Koliko straničnih grešaka (*page fault*) će operativni sistem obraditi uspešno za ovaj proces dok ga ne ugasi zbog prestupa, ako je operativni sistem za ovaj proces odvojio dovoljno okvira operativne memorije da smesti sve stranice koje taj proces regularno adresira? Obrazložiti.

Rešenje

1. VA(43): Page1(15):Page2(16):Offset(12).
2. Kod pogrešnog pristupa elementu iza kraja svakog reda osim poslednjeg (tj. za sve $i < M-1$ i $j = N$), proces pristupa prvom elementu sledećeg reda istog niza, tako da ti pristupi ne predstavljaju prestup. Kod prekoračenja poslednjeg reda niza `src` ($i = M-1$ i $j = N$), proces zapravo pristupa prvom elementu prvog reda niza `dst`, tako da ni to nije prestup. Međutim, kod upisa tog elementa u niz `dst`, proces će izazvati straničnu grešku (*page fault*) koju će operativni sistem tumačiti kao prestup, jer adresira stranicu koja ne pripada nekom alociranom segmentu. Ovo se dešava za $i=3$ i $j=N$.
3. Ovaj proces tokom izvršavanja regularno adresira sledeće stranice:
 - jednu stranicu segmenta za kod, za dohvaćanje instrukcija programa
 - samo jednu stranicu segmenta za stek, jer je to dovoljno za samo jedan poziv potprograma `main` sa argumentima ove funkcije i njene dve automatske celobrojne promenljive na steku
 - po 16 celih stranica za svaki niz `src` i `dst`, jer svaki niz sadrži $4 \cdot 0x1000 = 4 \cdot 2^{12}$ elemenata po 4 bajta, odnosno 2^{16} bajtova, što je po 16 stranica po 2^{12} bajtova.

Sve ukupno, proces regularno adresira 34 stranice. Kako svaka od njih ostaje u memoriji nakon prvog adresiranja i eventaulnog učitavanja, i pošto je procesu dodeljeno dovoljno okvira, ovaj proces će generisati isto toliki broj (34) straničnih grešaka koje će biti uspešno obrađene.

2. zadatak, kolokvijum, jul 2020.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi straničnu organizaciju memorije. Logičke segmente koje je proces alocirao sistem opisuje strukturama tipa `SegDesc` u dvostruko ulančanoj listi za svaki proces. Pritom se za svaki proces pri kreiranju uvek alociraju najmanje dva logička segmenta koji pokrivaju najniže i najviše adrese virtuelnog adresnog prostora procesa (preslikavaju se u prostor koji koristi kernel). Logički segment je uvek poravnat i zaokružen na stranice. U strukturi `SegDesc` polja `next` i `prev` služe za ulančavanje u listu, polje `pg` sadrži broj prve stranice logičkog

segmenta, a polje `sz` sadrži veličinu segmenta izraženu u broju stranica. Tip `size_t` je neoznačen celobrojni tip dovoljno velik da predstavi veličinu virtuelnog adresnog prostora. Konstanta `PAGE_SZ` predstavlja veličinu stranice u bajtovima (adresibilnim jedinicama), a konstanta `PAGE_OFFS_SZ` veličinu polja u virtuelnoj adresi za broj bajta unutar stranice; obe ove konstante su tipa `size_t`. Data je i pomoćna funkcija `insert_seg_desc` koja alokira jednu strukturu `SegDesc` i ulančava je u listu kao što je pokazano.

```
struct SegDesc {
    SegDesc *prev, *next;
    size_t pg, sz;
    ...
};

int insert_seg_desc (SegDesc* sd, size_t pg, size_t sz) {
    SegDesc* nsd = alloc_seg_desc();
    if (!nsd) return -1; // Error: cannot allocate SegDesc
    nsd->pg = pg; nsd->sz = sz;
    nsd->next = sd->next; nsd->prev = sd;
    sd->next->prev = nsd; sd->next = nsd;
    return 0;
}
```

Implementirati internu funkciju kernela `vmalloc` koja treba da alokira logički segment kako je proces tražio. Traženi logički segment je proizvoljne veličine `size` u bajtovima (iako ovaj parametar ne mora biti zaokružen, segment se uvek alokira kao ceo broj stranica). Ukoliko je parametar `addr` različit od `null`, pokušava se alokacija počev od stranice kojoj pripada ova adresa; ukoliko tu nije moguće alokirati segment tražene veličine, funkcija odbija alokaciju i treba da vrati grešku (vrednost `null`). Ukoliko je parametar `addr` jednak `null`, sistem treba da pokuša alokaciju segmenta bilo gde gde je to moguće; ukoliko nije moguće, treba da vrati `null`. U slučaju uspeha, u oba slučaja ova funkcija treba da vrati adresu početka segmenta (poravnatu na stranicu). Prvi parametar je glava liste struktura `SegDesc` procesa koji je tražio ovu uslugu (sigurno različito od `null`).

```
void* vmalloc (SegDesc* head, void* addr, size_t size);
```

Rešenje

```
void* vmalloc (SegDesc* head, void* addr, size_t size) {
    size_t sz = (size+PAGE_SIZE-1)>>PAGE_OFFS_SZ;
    SegDesc* sg = head;
    if (!addr) {
        for (; sg && sg->next; sg=sg->next) {
            size_t pg = sg->pg+sg->sz;
            if (sg->next->pg - pg >= sz) {
                if (insert_seg_desc(sg,pg,sz)<0) return 0;
                return (void*)(pg<<PAGE_OFFS_SZ);
            }
        }
        return 0;
    }
    else {
        size_t pg = addr>>PAGE_OFFS_SZ;
        for (; sg && sg->next; sg=sg->next) {
            if (sg->pg+sg->sz<=pg && pg+sz<=sg->next->pg) {
                if (insert_seg_desc(sg,pg,sz)<0) return 0;
                return (void*)(pg<<PAGE_OFFS_SZ);
            }
        }
        return 0;
    }
}
```

}

3. zadatak, drugi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog računara je 4GB i organizovan je stranično, adresibilna jedinica je bajt, a PMT je organizovana u dva nivoa. Broj ulaza u PMT oba nivoa je isti, kao i širina svakog ulaza koja je po 32 bita. PMT svakog nivoa zauzima tačno jednu celu stranicu.

Neki operativni sistem na ovom računaru zauzima najnižih 4MB raspoložive fizičke memorije za potrebe kernela. Kako ne bi menjao memorijski kontekst prilikom obrade sistemskih poziva, sistem preslikava (mapira) najviše stranice svakog kreiranog procesa u ovaj memorijski prostor kernela, s tim što postavlja indikatore u deskriptoru tih stranica u PMT tako da one nisu dostupne u nepriviligovanom (korisničkom) režimu rada procesora.

Kernel koristi tehniku *copy-on-write*. U implementaciji sistemskog poziva *fork*, sve PMT procesa roditelja i deteta, uključujući i PMT prvog nivoa, se ne kopiraju, nego se dele, sve dok je to moguće, odnosno dok kernel ne mora da ih razdvoji (kopira). Osim toga, pošto je deo memorije koji pripada kernelu deljen u virtuelnim adresnim prostorima svih procesa na istom mestu, one PMT koje se odnose na taj deo memorije svi procesi dele sve vreme.

1. Kolika je veličina stranice i koliko ulaza ima PMT prvog nivoa? Prikazati postupak kojim se došlo do odgovora.
2. Neki proces izvršio je sistemski poziv *exec* koji inicijalizuje memorijsku mapu tog procesa tako da, osim dela koji se preslikava u memoriju kernela, alocira jedan segment za kod veličine 1MB na početku svog virtuelnog adresnog prostora, jedan segment za podatke veličine 3MB odmah nakon segmenta koda, i segment za stek veličine 4MB odmah ispod memorijskog dela koji pripada kernelu. Koliko operativne memorije (osim one koja je prethodno već bila zauzeta zbog postojanja drugih procesa) je alocirano prilikom izvršavanja ovog sistemskog poziva za potrebe smeštanja PMT ovog procesa? Obrazložiti.
3. Ovaj proces sada izvršava sistemski poziv *fork*. Koliko dodatne memorije (osim one koja je već bila zauzeta zbog postojanja ovog roditeljskog i drugih procesa) je dodatno alocirano za potrebe smeštanja PMT novog procesa deteta prilikom izvršavanja ovog sistemskog poziva? Obrazložiti.
4. Posmatrani proces (roditelj) je potom odmah izvršio instrukciju poziva potprograma, što je uzrokovalo upisom u samo jednu stranicu na vrhu steka. Koliko dodatne memorije (osim one koja je već bila zauzeta pre toga) je dodatno alocirano za potrebe smeštanja PMT oba procesa, i roditelja i deteta zbog izvršavanja ove instrukcije? Obrazložiti.

Rešenje

1. Neka n označava broj bita za adresiranje ulaza u PMT svakog nivoa (broj ulaza u PMT svakog nivoa je 2^n), a neka je veličina stranice 2^p bajtova. Na osnovu uslova zadatka važi: $n + n + p = 32$, jer je takva struktura 32-bitne virtuelne adrese; $2^n \cdot 2^2 = 2^p$, odnosno $n + 2 = p$, jer je veličina jedne PMT jednaka veličini stranice. Rešavanjem se dobija $n = 10, p = 12$, pa jedna PMT ima $2^{10} = 1024 = 1K$ ulaza, a stranica je veličine 4KB, koliko zauzima i jedna PMT prvog ili drugog nivoa.
2. Prema tome, jedna PMT drugog nivoa „pokriva“ $1K \cdot 4KB = 4MB$ virtuelnog adresnog prostora. Zbog toga, za memorijski prostor kernela, procesi dele tačno jednu PMT drugog nivoa na koju ukazuje poslednji ulaz u PMT prvog nivoa i ta PMT drugog nivoa se ne alocira za svaki proces posebno.

PMT prvog nivoa ovog procesa, koja je svakako morala biti alocirana, zauzima 4KB. Za potrebe segmenata koda i podataka ovog procesa potrebna je i dovoljna samo jedna PMT drugog nivoa ($1MB + 3MB = 4MB$), a za segment steka još jedna. Tako je za PMT prvog i drugog nivoa ukupno alocirano tri stranice, odnosno 12KB memorije.
3. Pošto su i PMT prvog i PMT drugog nivoa procesa roditelja i procesa deteta inicijalno potpuno iste, one se mogu u potpunosti deliti odmah nakon sistemskog poziva *fork* (ovi procesi imaju inicijalno isti i PMTP), i to sve dok neki od procesa ne izvrši upis u neku svoju stranicu. Prema tome, nije potrebno alocirati nikakav dodatni memorijski prostor za PMT procesa deteta.
4. Kada jedan proces izvrši upis u jednu svoju stranicu, potrebno je „razdvojiti“, odnosno kopirati jednu PMT drugog nivoa procesa roditelja i procesa deteta. Osim toga, i odgovarajući ulazi u PMT prvog nivoa koji ukazuju na tu PMT drugog nivoa tako postaju različiti, jer moraju da ukazuju na različite PMT drugog nivoa,

pa je potrebno „razdvojiti“, odnosno kopirati i PMT prvog nivoa. Zato je alocirano dve dodatne stranice, za smeštanje ove dve dodatne PMT, odnosno 8KB memorije.

3. zadatak, drugi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 4GB i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 8KB. PMT (*page map table*) je organizovana u dva nivoa, s tim da je broj ulaza u PMT prvog nivoa dva puta manji od broja ulaza u PMT drugog nivoa. Posmatra se jedan proces kreiran nad sledećim programom:

```
#define N 0x1000
int src[N], dst[N];

int main () {
    for (int i=0; i<N; i++) dst[i] = src[i];
}
```

pod sledećim pretpostavkama:

- tip `int` je veličine 32 bita; promenljivu `i` je prevodilac formirao kao registarsku promenljivu (njena vrednost se ne čuva u operativnoj memoriji);
 - segment za kod ovog programa veličine je jedne stranice, a segment za stek je veličine 32 stranice;
 - nizovi `src` i `dst` smešteni su jedan odmah iza drugog u segment memorije alociran za statičke podatke; ovaj segment je onoliko koliko je najmanje stranica potrebno za smeštanje ovih nizova;
 - operativni sistem ne učitava nijednu stranicu pri kreiranju procesa, već sve stranice učitava tek na zahtev (*demand paging*).
1. Prikazati logičku strukturu virtuelne adrese i označiti veličinu svakog polja.
 2. Koliko straničnih grešaka (*page fault*) će izazvati izvršavanje ovog procesa ako je operativni sistem za ovaj proces odvojio dovoljno okvira operativne memorije da smesti sve stranice koje taj proces adresira? Obrazložiti.
 3. Ako je operativni sistem za ovaj proces odvojio samo 4 okvira operativne memorije, a za zamenu (izbacivanje) bira onu stranicu istog tog procesa koja je najdavnije učitana u memoriju, koja stranica će biti prva izbačena iz memorije i kojom stranicom će biti zamenjena (stranice imenovati kao n -te stranice segmenta za kod, stek ili podatke)? Obrazložiti.

Rešenje

1. VA(32): Page1(9):Page2(10):Offset(13).
2. Ovaj proces adresira sledeće stranice:
 - jednu stranicu segmenta za kod, za dohvaćanje instrukcija programa
 - samo jednu stranicu iz segmenta za stek, jer je to dovoljno za samo jedan poziv potprograma `main` bez automatskih objekata na steku (argumenata i lokalnih varijabli)
 - po 2 stranice za svaki niz `src` i `dst`, jer svaki niz sadrži $0x1000 = 2^{12}$ elemenata po 4 bajta, odnosno 2^{14} bajtova, što je 2 stranice po 2^{13} bajtova.

Sve ukupno, proces adresira 6 stranica. Kako svaka od njih ostaje u memoriji nakon prvog adresiranja i učitavanja, i pošto je procesu dodeljeno dovoljno okvira, ovaj proces će generisati isto toliki broj (6) straničnih grešaka.
3. U memoriju će biti najpre učitana stranica segmenta za kod, stranica segmenta za stek, i po jedna stranica segmenta za podatke, tj. prva i treća stranica tog segmenta, u kojima se nalaze prve polovine nizova `src` i `dst`, tim redom. Kada proces bude adresirao prvi element druge polovine niza `src`, za tu stranicu (drugu u segmentu za podatke) neće biti mesta, pa će biti izbačena najdavnije učitana stranica, a to je stranica segmenta za kod.

3. zadatak, drugi kolokvijum, april 2018.

- [Postavka](#)
- [Rešenje](#)

Neki sistem zauzima najnižih n okvira raspoložive fizičke memorije za potrebe kernela. Kako ne bi menjao memorijski kontekst prilikom obrade sistemskih poziva, sistem preslikava (mapira) najviših n stranica svakog kreiranog procesa u ovaj memorijski prostor kernela, s tim da postavlja indikatore u deskriptoru tih stranica u PMT (*page map table*) tako da one nisu dostupne u nepriviligovanom (korisničkom) režimu rada procesora. Ostatak virtuelnog adresnog prostora dostupan je samom procesu za njegove potrebe i u korisničkom režimu. Virtuelni adresni prostor je 16EB (eksabajt, $1\text{E}=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 16KB. Fizički adresni prostor je veličine 1TB (terabajt).

PMT je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava u najnižim bitima, dok 5 najviših bita ima sledeće značenje (tim redom, od najvišeg ka najnižem): bit koji govori da li je data stranica dostupna u nepriviligovanom (korisničkom) režimu rada procesora, bit koji govori da li je preslikavanje stranice moguće ili ne, i još 3 bita koja koduju prava pristupa (*rwX*), dok se ostali biti ne koriste. Jedan ulaz u PMT prvog i drugog nivoa zauzima minimalan, ali ceo broj bajtova. Implementirati sledeću funkciju:

```
void initPMT (unsigned* pmt, unsigned long pageFrom, unsigned long nPages,
              unsigned long frameFrom, unsigned short rwx);
```

Ovu funkciju poziva kod kernela kada inicijalizuje PMT novokreiranog procesa, na čiju (već alociranu) PMT prvog nivoa ukazuje argument `pmt`, kako bi obavio opisano preslikavanje dela virtuelnog prostora u kernel prostor. Ona treba da preslika `nPages` susednih stranica počev od stranice `pageFrom` u isto toliko susednih okvira počev od okvira `frameFrom`. Argument `rwX` sadrži bite prava pristupa koje treba postaviti u deskriptore svih tih stranica.

Na raspolaganju je interna funkcija kernela `alloc_pmt2` koja alocira jednu PMT drugog nivoa u memoriji, popunjava sve njene ulaze nulama i vraća broj okvira u kom počinje ta alocirana PMT. Pretpostaviti da će ova funkcija uvek uspeti da alocira PMT u memoriji (ignorisati greške). Veličine tipova su sledeće: `int` – 32 bita, `long` – 64 bita, `short` – 16 bita.

Obratiti pažnju na to da se ceo kernel kod izvršava u memorijskom kontekstu nekog procesa, što znači da su sve adrese (vrednosti pokazivača) virtuelne. Za konverziju neke fizičke adrese (vrednosti pokazivača) u kernel prostoru u virtuelnu adresu (u najvišem delu virtuelnog prostora svakog procesa), na raspolaganju je sledeća funkcija:

```
void* kaddrPtoV (void* physicalAddr);
```

Rešenje

VA(64): Page1(25):Page2(25):Offset(14)

PA(40): Frame(26):Offset(14)

```
const unsigned short pg1w = 25, pg2w = 25, offsw = 14;
```

```
inline unsigned getPMT1EntryForPage (unsigned long pg) {
    return pg >> pg2w;
}
```

```
inline unsigned getPMT2EntryForPage (unsigned long pg) {
    return pg & ~(-1L << pg2w);
}
```

```
void setPMTEntryForPage (unsigned* pmt1, unsigned long pg, unsigned val) {
    unsigned pmt1entry = getPMT1EntryForPage(pg);
    if (pmt1[pmt1entry] == 0)
        pmt1[pmt1entry] = alloc_pmt2();
    unsigned* pmt2 = (unsigned*)((unsigned long)pmt1[pmt1entry] << offsw);
    pmt2 = (unsigned*)kaddrPtoV(pmt2);
    unsigned pmt2entry = getPMT2EntryForPage(pg);
    pmt2[pmt2entry] = val;
}
```

```

}

void initPMT (unsigned* pmt, unsigned long pageFrom, unsigned long nPages,
             unsigned long frameFrom, unsigned short rwx) {
    for (unsigned long i=0; i<nPages; i++) {
        unsigned descr = (frameFrom+i) | (((01<<3)|rwx)<<27);
        setPMTEntryForPage(pmt, pageFrom+i, descr);
    }
}

```

3. zadatak, drugi kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 4GB i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 16KB. Fizički adresni prostor je veličine 16MB. PMT (*page map table*) je organizovana u dva nivoa, s tim da je broj ulaza u PMT prvog i drugog nivoa isti.

Za svaki proces operativni sistem kreira memorijski kontekst kao skup deskriptora logičkih memorijskih segmenata. Deskriptor segmenta predstavljen je strukturom `SegDesc` u kojoj, između ostalog, postoje sledeća polja:

- `unsigned long startingPage`: početna stranica segmenta u virtuelnom prostoru;
- `unsigned long size`: veličina segmenta izražena u broju stranica;
- `SegDesc *next, *prev`: sledeći i prethodni deskriptor segmenta u dvostruko ulančanoj listi segmenata istog procesa; na prvi segment u listi ukazuje polje `segDesc` u PCB procesa;
- `short rwx`: u tri najniža bita definisana su prava pristupa stranicama datog segmenta, odnosno dozvoljene operacije za taj logički segment.

Kada stranica nije u memoriji ili ne pripada alociranom segmentu, u odgovarajućem ulazu u PMT nalazi se vrednost 0 koja hardveru indikuje da preslikavanje nije moguće; procesor tada generiše izuzetak tipa stranične greške (*page fault*). U slučaju povrede prava pristupa stranici, odnosno nedozvoljene operacije, procesor generiše izuzetak koji naziva *operation denied*. Sistem primenjuje *copy on write* tehniku.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Implementirati funkciju `resolveVAddrExc`:

```

enum AddrExcKind {pageFault, opDenied};

int resolveVAddrExc (PCB* pcb, unsigned long vaddr, AddrExcKind kind,
                   short rwx, SegDesc** ret);

```

Ovu funkciju poziva kod kernela kada obrađuje neki izuzetak prilikom adresiranja virtuelne adrese za proces na čiji PCB ukazuje prvi argument, prilikom pristupa adresi datoj drugim argumentom. Tip izuzetka daje argument `kind`, a operaciju koja je pokušana argument `rwx`. Ukoliko data virtuelna adresa nije regularna (ne pripada definisanom logičkom segmentu), ili tražena operacija nije dozvoljena za tu adresu, ova funkcija treba da vrati rezultat `MEM_ACCESS_FAULT`, na osnovu koga će kernel ugasiti proces. U suprotnom, radi se o slučaju kada je adresiranje dozvoljeno, ali stranica nije u memoriji i treba je učitati, i kada funkcija treba da vrati `LOAD_PAGE`, ili o slučaju kada treba iskopirati stranicu prilikom upisa, i kada funkcija treba da vrati `COPY_ON_WRITE`. Kad god je virtuelna adresa unutar definisanog logičkog segmenta, u izlazni argument na koga ukazuje argument `ret` treba upisati pokazivač na deskriptor segmenta kom pripada data (regularna) virtuelna adresa.

Rešenje

1. VA(32): Page1(9):Page2(9):Offset(14).
PA(24): Frame(10):Offset(14).
- 2.

```

const unsigned offsw = 14;
enum AddrExcKind {pageFault, opDenied};

```



```

int resolveVAddrExc (PCB* pcb, unsigned long vaddr, AddrExcKind kind,
                    short rwx, SegDesc** ret) {
    if (pcb==0) return ERR_FATAL; // Fatal exception!
    unsigned long page = vaddr>>offsw;
    for (SegDesc* sd = pcb->segDesc; sd!=0; sd = sd->next) {
        if (page>=sd->startingPage && page<sd->startingPage+sd->size) {
            *ret = sd;
            switch (kind) {
                case pageFault: return LOAD_PAGE;
                case opDenied:
                    if ((rwx&2) && (sd->rwx&2))
                        return COPY_ON_WRITE;
                    else
                        return MEM_ACCESS_FAULT;
            }
        }
    }
    *ret = 0;
    return MEM_ACCESS_FAULT;
}

```

3. zadatak, drugi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajt, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 16KB. Fizički adresni prostor je veličine 1TB (terabajt). PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava i još 2 najniža bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije alocirana, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka), dok se ostali biti ne koriste. Jedan ulaz u PMT prvog i drugog nivoa zauzima minimalan, ali ceo broj bajtova. Sistem ne vrši učitavanje stranica na zahtev niti zamenu stranica, već su sve stranice procesa (odnosno sve one i samo one koje pripadaju logičkim segmentima virtuelne memorije koje je proces alocirao) učitane prilikom njegovog kreiranja i stalno su u memoriji do njegovog gašenja.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Implementirati sledeću funkciju:

```
int sharePage (unsigned* pmtFrom, unsigned long pageFrom, unsigned* pmtTo, unsigned long pageTo);
```

Ovu funkciju poziva kod kernela kada obrađuje sistemski poziv kojim jedan proces, na čiju PMT prvog nivoa ukazuje `pmtTo`, zatraži deljenje svoje stranice `pageTo` sa stranicom `pageFrom` drugog procesa, na čiju PMT prvog nivoa ukazuje `pmtFrom`, odnosno želi da svoju stranicu `pageTo` preusmeri na isti okvir koji već koristi stranica `pageFrom`. U slučaju da data stranica nije alocirana u `pmtFrom`, treba vratiti -1 kao signal greške. Pretpostavlja se da proces koji traži ovu uslugu (`pmtTo`) nije već imao alociranu datu stranicu `pageTo` (ona se alocira tek pri ovom pozivu).

Na raspolaganju je interna funkcija kernela `alloc_pmt` koja alocira jednu PMT u memoriji, popunjava sve njene ulaze nulama i vraća broj okvira u kom počinje ta alocirana PMT. Veličine tipova su sledeće: `int` – 32 bita, `long` – 64 bita, `short` – 16 bita.

Rešenje

1. VA(64): Page1(25):Page2(25):Offset(14).

PA(40): Frame(26):Offset(14).

2.

```
const unsigned short pg1w = 25, pg2w = 25, offsw = 14;

inline unsigned getPMT1Entry (unsigned long vaddr) {
    return vaddr >> (pg2w + offsw);
}

inline unsigned getPMT2Entry (unsigned long vaddr) {
    return (vaddr >> offsw) & ~(-1L << pg2w);
}

int getPMTEntry (unsigned* pmt1, unsigned long vaddr, unsigned* value) {
    unsigned pmt1entry = getPMT1Entry(vaddr);
    if (pmt1[pmt1entry] == 0) {
        *value = 0;
        return -1;
    }
    unsigned* pmt2 = (unsigned*)(((unsigned long)pmt1[pmt1entry]) << offsw);
    unsigned pmt2entry = getPMT2Entry(vaddr);
    *value = pmt2[pmt2entry];
    return (*value & 3 == 0) ? -1 : 0;
}

void setPMTEntry (unsigned* pmt1, unsigned long vaddr, unsigned value) {
    unsigned pmt1entry = getPMT1Entry(vaddr);
    if (pmt1[pmt1entry] == 0)
        pmt1[pmt1entry] = alloc_pmt();
    unsigned* pmt2 = (unsigned*)(((unsigned long)pmt1[pmt1entry]) << offsw);
    unsigned pmt2entry = getPMT2Entry(vaddr);
    pmt2[pmt2entry] = value;
}

int sharePage (unsigned* pmtFrom, unsigned long pageFrom,
unsigned* pmtTo, unsigned long pageTo) {
    unsigned long vaddrFrom = pageFrom << offsw;
    unsigned long vaddrTo = pageTo << offsw;
    unsigned value;
    if (getPMTEntry(pmtFrom, vaddrFrom, &value) < 0)
        return -1;
    setPMTEntry(pmtTo, vaddrTo, &value);
    return 0;
}
```

3. zadatak, drugi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 4GB i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 4KB. Fizički adresni prostor je veličine 256MB. PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine).

Kada sistem kreira nov proces, ne učitava inicijalno nijednu njegovu stranicu, niti alokira ijednu PMT drugog nivoa, već samo alokira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alokiraju i PMT drugog nivoa. Prilikom inicijalizacije procesa, sistem

samo kreira memorijski kontekst kao skup deskriptora logičkih memorijskih segmenata definisanih u izvršnom fajlu. Deskriptor segmenta je predstavljen strukturom `SegDesc` u kojoj, između ostalog, postoje sledeća polja:

- `unsigned long startingPage`: početna stranica segmenta u virtuelnom prostoru;
- `unsigned long size`: veličina segmenta izražena u broju stranica;
- `SegDesc *next, *prev`: sledeći i prethodni deskriptor segmenta u dvostruko ulančanoj listi segmenata istog procesa; na prvi segment u listi ukazuje polje `segDesc` u PCB procesa.

Kada stranica nije u memoriji ili ne pripada alociranom segmentu, u odgovarajućem ulazu u PMT nalazi se vrednost 0 koja hardveru indikuje da preslikavanje nije moguće; procesor tada generiše izuzetak tipa stranične greške (*page fault*).

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Implementirati sledeću funkciju:

```
int resolvePageFault (PCB* pcb, unsigned long addr, SegDesc** ret);
```

Ovu funkciju poziva kod kernela kada obrađuje izuzetak tipa stranične greške za proces na čiji PCB ukazuje prvi argument, prilikom adresiranja adrese date drugim argumentom. Ova funkcija treba da proveri da li je ovo adresiranje dozvoljeno ili nije; ukoliko nije, treba da vrati rezultat `MEM_ACCESS_FAULT`, na osnovu koga će kernel ugasiti proces; u suprotnom, radi se o slučaju kada je adresiranje dozvoljeno, ali stranica nije u memoriji i treba je učitati, pa treba vratiti rezultat `LOAD_PAGE`, a u izlazni argument na koga ukazuje argument `ret` upisati pokazivač na deskriptor segmenta kom pripada data adresa.

Rešenje

1. VA(32): Page1(10):Page2(10):Offset(12).
PA(28): Frame(16):Offset(12).

2.

```
const unsigned offsw = 12;

int resolvePageFault (PCB* pcb, unsigned long addr, SegDesc** ret) {
    if (pcb==0) return ERR_FATAL; // Exception: fatal exception!
    unsigned long page = addr>>offsw;
    SegDesc* sd = pcb->segDesc;
    while (sd!=0) {
        if (page>=sd->startingPage && page<sd->startingPage+sd->size) {
            *ret = sd;
            return LOAD_PAGE;
        }
        sd = sd->next;
    }
    return MEM_ACCESS_FAULT;
}
```

3. zadatak, drugi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu koristi se straničenje sa PMT u jednom nivou i tehnika *copy-on-write*. Tabela deskriptora alociranih memorijskih segmenata jednog procesa ima sledeći sadržaj:

Segment#	Starting page	Size in pages	RWX
1	00h	2h	001
2	A0h	4h	100
3	C1h	3h	110

PMT ovog procesa u posmatranom stanju ima sledeći sadržaj (svi ulazi koji nisu navedeni imaju sadržaj 000 u polju RWX, što hardveru znači da preslikavanje nije dozvoljeno iz bilo kog razloga i da treba generisati *page fault*):

Page#	Frame#	RWX
00h	210h	
01h	211h	
A0h	212h	
A1h	213h	
A2h	214h	
A3h	215h	
C1h	216h	
C2h	217h	
C3h	218h	

1. U prethodnu tabelu upisati vrednosti u kolonu *RWX*, ako su sve stranice učitane.
2. U opisanom stanju ovaj proces kreira proces-dete pozivom `fork()`. Prikazati PMT procesa-roditelja nakon uspešnog završetka ovog poziva.
3. Nakon toga, proces-roditelj izvršava instrukciju koja upisuje vrednost u lokaciju na stranici C2h. Prikazati PMT oba procesa nakon što je operativni sistem obradio izuzetak koji je nastao tokom izvršavanja ove instrukcije. Pretpostaviti da je prvi slobodan okvir 219h.

Rešenje

Page#	Frame#	RWX	Page#	Frame#	RWX	Page#	Frame#	RWX	Page#	Frame#	RWX
00h	210h	001	00h	210h	001	00h	210h	001	00h	210h	001
01h	211h	001	01h	211h	001	01h	211h	001	01h	211h	001
A0h	212h	100	A0h	212h	100	A0h	212h	100	A0h	212h	100
A1h	213h	100	A1h	213h	100	A1h	213h	100	A1h	213h	100
A2h	214h	100	A2h	214h	100	A2h	214h	100	A2h	214h	100
A3h	215h	100	A3h	215h	100	A3h	215h	100	A3h	215h	100
C1h	216h	110	C1h	216h	100	C1h	216h	100	C1h	216h	100
C2h	217h	110	C2h	217h	100	C2h	219h*	110	C2h	217h*	110
C3h	218h	110	C3h	218h	100	C3h	218h	100	C3h	218h	100

(a) Rešenje pod a

(b) PMT procesa-roditelja

(c) PMT procesa-roditelja

(d) PMT procesa-dete

*) Moguće je i da je OS alocirao nov okvir za proces-dete, pa je korektan odgovor i ako 217h i 219h zamene mesta.

3. zadatak, drugi kolokvijum, septembar 2016.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajt, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 64KB. Fizički adresni prostor je veličine 4TB (terabajt). PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava hardveru da preslikavanje nije dozvoljeno ili moguće. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava i još 2 najniža bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka), dok se ostali biti ne koriste. Jedan ulaz u PMT prvog i drugog nivoa zauzima minimalan, ali ceo broj bajtova. Kada sistem kreira nov proces, ne učitava inicijalno ni jednu njegovu stranicu, niti alocira ijednu PMT drugog nivoa, već samo alocira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog nivoa.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Implementirati sledeću funkciju:

```
void releasePMTEntry (unsigned* pmt1, unsigned long page);
```

Ovu funkciju poziva kod kernela kada iz memorije izbacuje datu stranicu `page` procesa čija je PMT prvog nivoa zadata prvim argumentom. Ova funkcija treba da ažurira odgovarajuće ulaze u PMT (po potrebi, oba nivoa), s tim što treba i da dealocira PMT drugog nivoa, ako ona više nije potrebna. Na raspolaganju je interna funkcija kernela `dealloc_pmt(unsigned* pmt)` koja dealocira jednu PMT u memoriji. Ignorirati sve eventualne greške.

Rešenje

1. VA(64): Page1(24):Page2(24):Offset(16).

PA(42): Frame(26):Offset(16).

- 2.

```
const unsigned short pg1w = 24, pg2w = 24, offsw = 16;
const unsigned pmt1size = 1<<pg1w, pmt2size = 1<<pg2w;

void releasePMTEntry (unsigned* pmt1, unsigned long page) {
    unsigned pmt1entry = page>>(pg2w);
    unsigned* pmt2 = (unsigned*)((unsigned long)pmt1[pmt1entry]<<offsw);
    unsigned pmt2entry = (page) & ~(-1L<<pg2w);
    pmt2[pmt2entry] = 0;
    for (pmt2entry=0; pmt2entry<pmt2size; pmt2entry++)
        if (pmt2[pmt2entry]!=0) return;
    // PMT2 empty, release it:
    dealloc_pmt(pmt2);
    pmt1[pmt1entry] = 0;
}
```

3. zadatak, drugi kolokvijum, maj 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu koristi se straničenje sa PMT u jednom nivou i tehnika *copy-on-write*. Tabela deskriptora alociranih memorijskih segmenata jednog procesa ima sledeći sadržaj:

Segment#	Starting page	Size in pages	RWX
1	00h	4h	001
2	12h	3h	100
3	1Ah	2h	110

PMT ovog procesa u posmatranom stanju ima sledeći sadržaj (svi ulazi koji nisu navedeni imaju sadržaj 000 u polju RWX, što hardveru znači da preslikavanje nije dozvoljeno iz bilo kog razloga i da treba generisati *page fault*):

Page#	Frame#	RWX
00h	10h	
01h	11h	
02h	12h	
03h	13h	
12h	14h	
13h	15h	
14h	16h	
1Ah	17h	
1Bh	18h	

1. U prethodnu tabelu upisati vrednosti u kolonu *RWX*, ako su sve stranice učitane.
2. U opisanom stanju ovaj proces kreira proces-dete pozivom `fork()`. Prikazati PMT oba procesa (i roditelja i deteta) nakon uspešnog završetka ovog poziva.
3. Nakon toga, proces-dete izvršava instrukciju koja upisuje vrednost u lokaciju na stranici 1Ah. Prikazati PMT procesa-deteta nakon što je operativni sistem obradio izuzetak koji je nastao tokom izvršavanja ove instrukcije. Pretpostaviti da je prvi slobodan okvir 19h.

Rešenje

Page#	Frame#	RWX	Page#	Frame#	RWX	Page#	Frame#	RWX	Page#	Frame#	RWX
00h	10h	001	00h	10h	001	00h	10h	001	00h	10h	001
01h	11h	001	01h	11h	001	01h	11h	001	01h	11h	001
02h	12h	001	02h	12h	001	02h	12h	001	02h	12h	001
03h	13h	001	03h	13h	001	03h	13h	001	03h	13h	001
12h	14h	100	12h	14h	100	12h	14h	100	12h	14h	100
13h	15h	100	13h	15h	100	13h	15h	100	13h	15h	100
14h	16h	100	14h	16h	100	14h	16h	100	14h	16h	100
1Ah	17h	110	1Ah	17h	100	1Ah	17h	100	1Ah	19h	110
1Bh	18h	110	1Bh	18h	100	1Bh	18h	100	1Bh	18h	100

(e) Rešenje pod a

(f) PMT procesa-roditelja

(g) PMT procesa-deteta

(h) PMT procesa-deteta

3. zadatak, drugi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 1EB (eksabajt, $1E = 2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 4KB. Fizički adresni prostor je veličine 4TB (terabajt). PMT (*page map table*) je organizovana u tri nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT sva tri nivoa isti (PMT sva tri nivoa su iste veličine). PMT sva tri nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u jednom ulazu u PMT prvog i drugog nivoa, u najnižim bitima, čuva samo broj okvira u kom počinje PMT drugog/trećeg nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT trećeg nivoa, u najnižim bitima čuva se broj okvira u koji se stranica preslikava i još 2 bita sdesna koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka); jedan ulaz u PMT trećeg nivoa zauzima minimalan, ali ceo broj bajtova.

Kada sistem kreira nov proces, ne učitava inicijalno nijednu njegovu stranicu, niti alokira ijednu PMT drugog i trećeg nivoa, već samo alokira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog i trećeg nivoa.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. U PCB procesa polje `pmt1` tipa `unsigned` sadrži broj okvira od koga počinje PMT prvog nivoa. Na raspolaganju je sledeća funkcija koja okvir sa datim brojem proglašava slobodnim (upisuje ga u evidenciju slobodnih okvira):

```
void deallocateFrame (unsigned frameNumber);
```

Implementirati sledeću funkciju koja se koristi prilikom gašenja procesa ili njegovog izbacivanja iz memorije (*swap out*) i koja oslobađa sve okvire koje proces koristi:

```
void deallocateAllFrames (PCB* pcb);
```

Pretpostavlja se da proces ne deli stranice sa drugim procesima. Mašinska reč, kao i tip `int` su veličine 32 bita, dok su pokazivači, kao i tip `long` veličine 64 bita.

Rešenje

1. VA(60): Page1(16):Page2(16):Page3(16):Offset(12).

PA(42): Frame(30):Offset(12).

2.

```
const unsigned short pg1w = 16, pg2w = 16, pg3w = 16, offsw = 12;
const unsigned pmt1Size = 1U<<pg1w,
                    pmt2Size = 1U<<pg2w, pmt3Size = 1U<<pg3w;
typedef unsigned long ulong;

void deallocateAllFrames (PCB* pcb) {
    if (pcb==0) return; // Exception

    // Traverse PMT1:
    unsigned* pmt1 = (unsigned*)((ulong)(pcb->pmt1)<<offsw);
    for (unsigned i1 = 0; i1<pmt1Size; i1++) {
        if (pmt1[i1]==0) continue;

        // Traverse PMT2:
        unsigned* pmt2 = (unsigned*)((ulong)(pmt1[i1])<<offsw);
        for (unsigned i2 = 0; i2<pmt2Size; i2++) {
            if (pmt2[i2]==0) continue;

            // Traverse PMT3:
            unsigned* pmt3 = (unsigned*)((ulong)(pmt2[i2])<<offsw);
            for (unsigned i3 = 0; i3<pmt3Size; i3++) {
                if (pmt3[i3]==0) continue;
                // Deallocate frame:
                unsigned frame = pmt3[i3]>>2;
                deallocateFrame(frame);
            }
        }
    }
}
```

3. zadatak, drugi kolokvijum, april 2014.

- Postavka
- Rešenje

Virtuelni adresni prostor nekog sistema je 1EB (eksabajt, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 4KB. Fizički adresni prostor je veličine 4TB (terabajt). PMT (*page map table*) je organizovana u tri nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT sva tri nivoa isti (PMT sva tri nivoa su iste veličine). PMT sva tri nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u jednom ulazu u PMT prvog/drugog nivoa čuva samo broj okvira u kom počinje PMT drugog/trećeg nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT trećeg nivoa čuva se broj okvira u koji se stranica preslikava i još 2 bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka); jedan ulaz u PMT trećeg nivoa zauzima minimalan, ali ceo broj bajtova. Kada sistem kreira nov proces, ne učitava inicijalno nijednu njegovu stranicu, niti alokira ijednu PMT drugog i trećeg nivoa, već samo alokira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog i trećeg nivoa. Odgovoriti na sledeća pitanja uz detaljna obrazložena postupka.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Koliko memorije minimalno zauzima PMT alocirana za proces prilikom njegovog kreiranja?
3. Koliko ukupno memorije zauzimaju sve PMT alocirane za proces koji je u dosadašnjem toku izvršavanja adresirao najnižih 1GB svog virtuelnog adresnog prostora?

Rešenje

1. VA(60): Page1(16):Page2(16):Page3(16):Offset(12).
PA(42): Frame(30):Offset(12).
2. Širina PMT3 je $30+2=32$ bita. Ista je i širina PMT1 i PMT2. PMT1 ima 2^{16} ulaza širine 32 bita (4B), što je ukupno: $2^{18}B=256KB$.
3. Ovaj proces koristio je 2^{30} svojih najnižih adresa, što je $2^{30-12} = 2^{18}$ stranica. Jedna PMT trećeg nivoa pokriva 2^{16} stranica, pa je ovaj proces alocirao PMT prvog nivoa, jednu PMT drugog nivoa i četiri PMT trećeg nivoa. Zato ukupna veličina PMT iznosi $6 \cdot 256KB=1,5MB$.

3. zadatak, drugi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajt, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 16KB. Fizički adresni prostor je veličine 1TB (terabajt). PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava i još 2 najniža bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka), dok se ostali biti ne koriste. Jedan ulaz u PMT prvog i drugog nivoa zauzima minimalan, ali ceo broj bajtova. Kada sistem kreira nov proces, ne učitava inicijalno ni jednu njegovu stranicu, niti alocira ijednu PMT drugog nivoa, već samo alocira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog nivoa.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Implementirati sledeću funkciju:

```
void setPMTEntry (unsigned* pmt1, unsigned long vaddr, unsigned frame, short r, short w, short x);
```

Ovu funkciju poziva kod kernela kada treba da postavi ulaze u PMT (po potrebi, oba nivoa) kada je alocirao okvir za neku stranicu virtuelne memorije prilikom obrade stranične greške.

Značenje argumenata je sledeće:

- **pmt1**: pokazivač na PMT prvog nivoa (već alocirana prilikom kreiranja procesa);
- **vaddr**: puna virtuelna adresa koja se obrađuje (pripada stranici koja je učitana);
- **frame**: broj okvira koji je već alociran za datu stranicu;
- **r, w, x**: prava pristupa koja treba postaviti (0 ili 1; čitanje, upis, izvršavanje, respektivno).

Veličine tipova su sledeće: **int** – 32 bita, **long** – 64 bita, **short** – 16 bita.

Na raspolaganju je interna funkcija kernela `alloc_pmt()` koja alocira jednu PMT u memoriji, popunjava sve njene ulaze nulama i vraća broj okvira u kom počinje ta alocirana PMT. Ignorirati sve eventualne greške.

Rešenje

1. VA(64): Page1(25):Page2(25):Offset(14).
PA(40): Frame(26):Offset(14).
- 2.

```
const unsigned short pg1w = 25, pg2w = 25, offsw = 14;
```

```
void setPMTEntry (unsigned* pmt1, unsigned long vaddr, unsigned fr,
    short r, short w, short x) {
```

```

unsigned pmt1entry = vaddr>>(pg2w+offsw);
if (pmt1[pmt1entry]==0)
    pmt1[pmt1entry] = alloc_pmt();
unsigned* pmt2 = (unsigned*)((unsigned long)pmt1[pmt1entry] << offsw);
unsigned pmt2entry = (vaddr>>offsw) & ~(-1L<<pg2w);
pmt2[pmt2entry] = (fr<<2) | ((r||w)<<1) | (x||w);
}

```

2. zadatak, prvi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sa straničnom organizacijom memorije virtuelni adresni prostor je veličine 4 GB, adresibilna jedinica je 32-bitna reč, a fizička adresa je veličine 32 bita. Ceo virtuelni adresni prostor ima 64 K stranica.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i navesti širinu svakog polja.
2. Koliko 32-bitnih reči zauzima PMT? Obrazložiti.

Rešenje

1. VA: Page(16):Offset(14); PA: Frame(18):Offset(14).
2. 64K 32-bitnih reči.

Pošto virtuelni prostor ima $64K = 2^{16}$ stranica, toliko ulaza ima i PMT. Svaki ulaz je veličine najmanje jedne adresibilne jedinice (32-bitne reči), što je i dovoljno za smeštanje broja okvira i eventualnih dodatnih bita, pa PMT zauzima 64 K reči.

4. zadatak, drugi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajta, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 1KB. Fizički adresni prostor je veličine 1TB (terabajt). PMT (*page map table*) je organizovana u tri nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT sva tri nivoa isti (PMT sva tri nivoa su iste veličine). PMT sva tri nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u jednom ulazu u PMT prvog/drugog nivoa čuva samo broj okvira u kom počinje PMT drugog/trećeg nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT trećeg nivoa čuva se broj okvira u koji se stranica preslikava i još 2 bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka); jedan ulaz u PMT trećeg nivoa zauzima minimalan, ali ceo broj bajtova.

Kada sistem kreira nov proces, ne učitava inicijalno ni jednu njegovu stranicu, niti alocira ijednu PMT drugog i trećeg nivoa, već samo alocira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog i trećeg nivoa.

Odgovoriti na sledeća pitanja uz detaljna obrazloženja postupka.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Koliko memorije minimalno zauzima PMT alocirana za proces prilikom njegovog kreiranja?
3. Koliko ukupno memorije zauzimaju sve PMT alocirane za proces koji je u dosadašnjem toku izvršavanja adresirao najnižih 1GB svog virtuelnog adresnog prostora?

Rešenje

1. VA(64): Page1(18):Page2(18):Page3(18):Offset(10).
PA(40): Frame(30):Offset(10).

- Širina PMT3 je $30+2=32$ bita. Ista je i širina PMT1 i PMT2. PMT1 ima 2^{18} ulaza širine 32 bita (4B), što je ukupno: $2^{20}B=1MB$.
- Ovaj proces koristio je 2^{30} svojih najnižih adresa, što je $2^{30-10} = 2^{20}$ stranica. Jedna PMT trećeg nivoa pokriva 2^{18} stranica, pa je ovaj proces alocirao PMT prvog nivoa, jednu PMT drugog nivoa i četiri PMT trećeg nivoa. Zato ukupna veličina PMT iznosi $6 \cdot 1MB=6MB$.

4. zadatak, drugi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajt, $1E=2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 16KB. Fizički adresni prostor je veličine 1TB (terabajt). PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava i još 2 bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka), dok se ostali biti ne koriste. Jedan ulaz u PMT prvog i drugog nivoa zauzima minimalan, ali ceo broj bajtova.

Kada sistem kreira nov proces, ne učitava inicijalno ni jednu njegovu stranicu, niti alocira ijednu PMT drugog nivoa, već samo alocira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog nivoa.

Odgovoriti na sledeća pitanja uz detaljna obrazloženja postupka.

- Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
- Koliko memorije minimalno zauzima PMT alocirana za proces prilikom njegovog kreiranja?
- Koliko ukupno memorije zauzimaju sve PMT alocirane za proces koji je u dosadašnjem toku izvršavanja koristio najnižih 256GB svog virtuelnog adresnog prostora?

Rešenje

- VA(64): Page1(25):Page2(25):Offset(14).
PA(40): Frame(26):Offset(14).
- Širina PMT2 je 32 bita, od koga 26 sadrže broj okvira, 2 bite zaštite, a ostali su neiskorišćeni. Ista je i širina PMT1. PMT1 ima 2^{25} ulaza širine 32 bita (4B), što je ukupno: $2^{27}B=128MB$.
- Ovaj proces koristio je 2^{38} svojih najnižih adresa, što je $2^{38-14} = 2^{24}$ stranica. Jedna PMT drugog nivoa pokriva 2^{25} stranica, pa je ovaj proces alocirao PMT prvog nivoa i jednu PMT drugog nivoa. Zato ukupna veličina PMT iznosi: $2 \cdot 128MB=256MB$.

2. zadatak, prvi kolokvijum, mart 2012.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava straničnu organizaciju virtuelne memorije, pri čemu je virtuelni adresni prostor veličine 16GB, adresibilna jedinica je 32-bitna reč, a fizički adresni prostor je veličine 1GB. Stranica je veličine 1KB. U deskriptoru stranice u jednom ulazu u tabeli preslikavanja stranica (PMT) najviši bit ukazuje na to da li je stranica u memoriji ili ne, a najniži biti predstavljaju broj okvira u fizičkoj memoriji u koji se stranica preslikava. Deskriptor stranice ne sadrži druge informacije.

- Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svakog polja. Ako je početak PMT nekog procesa na fizičkoj adresi FF00h, na kojoj adresi je deskriptor stranice ABCh tog procesa?
- Na jeziku C napisati kod funkcije

```
void setPageDescr(unsigned* pmt, unsigned page, unsigned frame);
```

koja u tabelu na čiji početak ukazuje dati pokazivač `pmt`, za stranicu sa datim brojem `page`, upisuje deskriptor tako da se ta stranica preslikava u okvir sa datim brojem `frame`.

Rešenje

1. VA: Page(24):Offset(8);
PA: Frame(20):Offset(8).
Nalazi se na adresi 109BCh.
2.

```
void setPageDescr(unsigned* pmt, unsigned page, unsigned frame){
    pmt[page] = frame | ~((unsigned int)~0 / 2);
}
```

4. zadatak, drugi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor nekog sistema je 16EB (eksabajta, $1E = 2^{60}$) i organizovan je stranično, adresibilna jedinica je bajt, a stranica je veličine 4KB. Fizički adresni prostor je veličine 4TB (terabajta). PMT (*page map table*) je organizovana u dva nivoa, s tim da su i broj ulaza, kao i širina ulaza u PMT prvog i drugog nivoa isti (PMT oba nivoa su iste veličine). PMT oba nivoa smeštaju se u memoriju uvek poravnate na fizički okvir, odnosno uvek počinju na početku okvira. Zbog toga se u ulazu prvog nivoa čuva samo broj okvira u kom počinje PMT drugog nivoa, dok se preostali biti do celog broja bajtova u ulazu ne koriste; vrednost 0 u svim bitima označava da preslikavanje nije dozvoljeno. U jednom ulazu PMT drugog nivoa čuva se broj okvira u koji se stranica preslikava i još 2 bita koja koduju prava pristupa (00 – nedozvoljen pristup, stranica nije u memoriji, 01 – dozvoljeno samo izvršavanje instrukcije, 10 – dozvoljeno samo čitanje podataka, 11 – dozvoljeno i čitanje i upis podataka); jedan ulaz u PMT drugog nivoa zauzima minimalan, ali ceo broj bajtova.

Kada sistem kreira nov proces, ne učitava inicijalno ni jednu njegovu stranicu, niti alokira ijednu PMT drugog nivoa, već samo alokira PMT prvog nivoa, čije sve ulaze inicijalizuje nulama. Stranice se potom dohvataju na zahtev, tokom izvršavanja procesa, kada se po potrebi alociraju i PMT drugog nivoa. Odgovoriti na sledeća pitanja uz detaljna obrazloženja postupka.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti veličinu svakog polja.
2. Koliko memorije minimalno zauzima PMT alocirana za proces prilikom njegovog kreiranja?
3. Koliko ukupno memorije zauzimaju sve PMT alocirane za proces koji je u dosadašnjem toku izvršavanja koristio najnižih 512GB svog virtuelnog adresnog prostora?

Rešenje

1. VA(64): Page1(26):Page2(26):Offset(12).
PA(42): Frame(30):Offset(12).
2. Širina PMT2 je $30+2=32$ bita. Ista je i širina PMT1. PMT1 ima 2^{26} ulaza širine 32 bita (4B), što je ukupno: $2^{28}B = 256MB$.
3. Ovaj proces koristio je 2^{39} svojih najnižih adresa, što je $2^{39-12} = 2^{27}$ stranica. Jedna PMT drugog nivoa pokriva 2^{26} stranica, pa je ovaj proces alocirao PMT prvog nivoa i dve PMT drugog nivoa. Zato ukupna veličina PMT iznosi: $3 \cdot 256MB = 768MB$.

2. zadatak, prvi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Virtuelna memorija nekog računara organizovana je stranično. Veličina virtuelnog adresnog prostora je 16 MB, adresibilna jedinica je bajt, a veličina stranice je 64 KB. Veličina fizičkog adresnog prostora je 16 MB. Operativni

sistem učitava stranice na zahtev, tako što se stranica učitava u prvi slobodni okvir fizičke memorije kada joj se pristupi. Kada se kreira proces, ni jedna njegova stranica se ne učitava odmah, već tek kad joj se prvi put pristupi. U početnom trenutku, slobodni okviri fizičke memorije su okviri počev od 20h zaključno sa 2Fh. Neki proces generiše sledeću sekvencu virtuelnih adresa tokom svog izvršavanja (sve vrednosti su heksadecimalne):

30F00, 30F02, 30F04, 822F0, 822F2, 322F0, 322F2, 322F4, 522F0, 522F2, 602F0, 602F2

Prikazati izgled prvih 16 ulaza tabele preslikavanja stranica (PMT) za ovaj proces posle izvršavanja date sekvence. Za svaki ulaz u PMT prikazati indikator prisutnosti stranice u fizičkoj memoriji (0 ili 1) i broj okvira u fizičkoj memoriji u koji se stranica preslikava, ukoliko je stranica učitana; ukoliko nije, prikazati samo ovaj indikator.

Rešenje

VA: Page(8):Offset(16)

PA: Frame(8):Offset(16)

Sekvenca stranica koje se traže: 3, 3, 3, 8, 8, 3, 3, 3, 5, 5, 6, 6

PMT na kraju sekvence:

Entry	Flag	Frame
0	0	
1	0	
2	0	
3	1	20h
4	0	
5	1	22h
6	1	23h
7	0	
8	1	21h
9	0	
A	0	
B	0	
C	0	
D	0	
E	0	
F	0	

4. zadatak, drugi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Neki sistem sa straničnom organizacijom virtuelne memorije koristi tehniku *copy-on-write* i sistemski poziv *fork* za kreiranje procesa.

Odgovoriti na sledeća pitanja uz precizna obrazloženja.

Da li ovakav sistem mora da kreira i novu PMT za novokreirani proces odmah pri kreiranju procesa u sistemskom pozivu *fork*? Ako mora, zašto mora? Ako ne mora, u kom trenutku najkasnije mora da kreira novu PMT za proces? Šta se dešava sa vrednošću PMTP u PCB procesa?

Rešenje

Sistem ne mora da kreira novu PMT za novokreirani proces prilikom izvršavanja sistemskog poziva *fork*, jer oba procesa inicijalno dele sve stranice, pa su njihove PMT inicijalno potpuno iste. Isto važi i za PMTP koji ukazuje na istu PMT, pa su i oni isti. Kada bilo koji od ovih procesa generiše izuzetak zbog zabranjenog upisa u deljenu stranicu, sistem mora da razdvoji stranice kopiranjem u različite fizičke okvire (*copy-on-write*). Kako sada procesi imaju razdvojenu stranicu preslikanu u različite fizičke okvire, njihove PMT postaju različite (različit je sadržaj

deskriptora za razdvojenu stranicu), pa je to najkasniji trenutak kada sistem mora da formira sopstvenu PMT za novokreirani proces. Kako sada procesi imaju različite PMT, i njihovi PMTP postaju tada različiti.

2. zadatak, prvi kolokvijum, april 2011.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava straničnu organizaciju virtuelne memorije. Virtuelna adresa je 32-bitna, stranica je veličine 64KB, a adresibilna jedinica je bajt. Fizički adresni prostor je veličine 4GB. U tabeli preslikavanja stranica (PMT) svaki ulaz zauzima samo onoliko prostora koliko je potrebno da se smesti broj okvira u koji se data stranica preslikava, pošto se informacije o smeštaju stranica na disku čuvaju u drugoj strukturi. Pri tome, vrednost 0 u ulazu u PMT označava da data stranica nije učitana u fizičku memoriju (nijedna stranica korisničkog procesa ne preslikava se u okvir 0 gde se inače nalazi interapt vektor tabela). Trenutno stanje PMT dva posmatrana procesa A i B je sledeće (brojevi ulaza i vrednosti su dati heksadecimalno):

Proces A:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	FE12	FEFF	0	0	0	0	14	FE	0

Proces B:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0	12	2314	01AD	0	22	01AE	0	0

Prikazati sadržaj ovih tabela nakon što je operativni sistem završio sve sledeće akcije tim redom:

- Obradio straničnu grešku (*page fault*) koju je generisao proces A kada je adresirao adresu 30203h u svom adresnom prostoru, tako što je izbacio stranicu broj 1 procesa B i na njeno mesto učitao traženu stranicu procesa A.
- Obradio straničnu grešku (*page fault*) koju je generisao proces B kada je adresirao adresu 1F00Fh u svom adresnom prostoru, tako što je izbacio stranicu broj FFh procesa A i na njeno mesto učitao traženu stranicu procesa B.
- Obradio sistemski poziv procesa A kojim je taj proces zahtevao deljenje svoje stranice broj 2 sa stranicom broj 1 procesa B.

Rešenje

Proces A:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	FE12	FEFF	14	12	0	0	0	FE	0

Proces B:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0	14	2314	01AD	0	22	01AE	0	0

2. zadatak, prvi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

Virtuelna memorija nekog računara organizovana je stranično. Veličina virtuelnog adresnog prostora je 2 MB, adresibilna jedinica je 16-bitna reč, a veličina stranice je 128 KB. Veličina fizičkog adresnog prostora je 32 MB. Operativni sistem učitava stranice na zahtev, tako što se stranica učitava u prvi slobodni okvir fizičke memorije kada joj se pristupi. Kada se kreira proces, ni jedna njegova stranica se ne učitava odmah, već tek kad joj se prvi put pristupi. U početnom trenutku, slobodni okviri fizičke memorije su okviri počev od 10h zaključno sa 1Fh. Neki proces generiše sledeću sekvencu virtuelnih adresa tokom svog izvršavanja (sve vrednosti su heksadecimalne):

30F00, 30F02, 30F04, 922F0, 922F2, 322F0, 322F2, 322F4, 522F0, 522F2, 402F0, 402F2

Prikazati izgled cele tabele preslikavanja stranica (PMT) za ovaj proces posle izvršavanja ove sekvence. Za svaki ulaz u PMT prikazati indikator prisutnosti stranice u fizičkoj memoriji (0 ili 1) i broj okvira u fizičkoj memoriji u koji se stranica preslikava, ukoliko je stranica učitana; ukoliko nije, prikazati samo ovaj indikator.

Rešenje

VA: Page(4):Offset(16)

PA: Block(8):Offset(16)

Sekvenca stranica koje se traže: 3, 3, 3, 9, 9, 3, 3, 3, 5, 5, 4, 4

PMT na kraju sekvence:

Entry	Flag	Frame
0	0	
1	0	
2	0	
3	1	10h
4	1	13h
5	1	12h
6	0	
7	0	
8	0	
9	1	11h
A	0	
B	0	
C	0	
D	0	
E	0	
F	0	

4. zadatak, drugi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sa straničnom organizacijom virtuelne memorije koristi se *copy-on-write* tehnika deljenja stranica između procesa. U deskriptoru stranice u tabeli preslikavanja stranica (PMT) procesa nalaze se samo sledeće informacije: broj okvira u koji se stranica preslikava (0 označava da stranica nije u memoriji, okvir broj 0 se ne dodeljuje procesima) i biti prava pristupa (*R*-dozvoljeno čitanje u fazi izvršavanja instrukcije/dohvatanja operanda, *W*-dozvoljen upis, *E*-dozvoljeno čitanje u fazi dohvatanja instrukcije). Pored PMT, operativni sistem za svaki proces vodi posebnu strukturu podataka koju naziva *VMStruct* i u kojoj se čuvaju podaci koje koristi operativni sistem, a koji nisu potrebni hardveru za preslikavanje adresa. Svaki deskriptor u ovoj strukturi opisuje čitav skup susednih stranica koje predstavljaju logičku celinu, jer su svi ovi podaci za njih isti, i koji se u ovom kontekstu naziva *region*. U ovom deskriptoru čuvaju se sledeći podaci: prva stranica u skupu stranica na koje se deskriptor odnosi (*Start Page#*), broj susednih stranica na koje se deskriptor odnosi (*Region Length*), biti prava pristupa na logičkom nivou (*R*-dozvoljeno čitanje u fazi izvršavanja instrukcije/dohvatanja operanda, *W*-dozvoljen upis, *E*-dozvoljeno čitanje u fazi dohvatanja instrukcije), da li su stranice u ovom regionu deljene tako da ih treba kopirati pri prvom upisu (*Copy-On-Write*: 0-nisu deljene, 1-jesu deljene i treba ih kopirati pri upisu, samo ako je upis dozvoljen na logičkom nivou), kao i mesto na disku gde se zamenjuju stranice iz tog regiona (nije relevantno za ovaj zadatak). Posmatra se proces *Parent* čiji su delovi struktura *PMT* i *VMStruct* dati u nastavku.

Page#	Frame#	RWE	StartPage#	Region Length	RWE-Copy-On-Write	Opis
A04h	23h	?	A00h	50h	001-0	Code Region
BF0h	14h	?	B00h	FFh	110-0	Data Region
C0Ah	7Ah	?	C00h	70h	100-0	Input Buffer Region

(i) PMT

(j) VMStruct

Ovaj proces izvršava sistemski poziv `fork()` i uspešno kreira proces potomak *Child*. Prikazati iste delove struktura *PMT* i *VMStruct* za oba procesa *Parent* i *Child* neposredno nakon ovog poziva. Sistem primenjuje *demand paging* strategiju (dohvata stranicu tek kada se prvi put zatraži, ne alokira je odmah pri kreiranju procesa).

Rešenje

Proces Parent:

Page#	Frame#	RWE	StartPage#	Region Length	RWE-Copy-On-Write	Opis
A04h	23h	001	A00h	50h	001-0	Code Region
BF0h	14h	100	B00h	FFh	110-0	Data Region
C0Ah	7Ah	100	C00h	70h	100-0	Input Buffer Region

(k) PMT

(l) VMStruct

Proces Child: Sve isto.

Napomena: Primititi da se u opisanom sistemu bit Copy-On-Write (u daljem tekstu CoW) odnosi na čitav set stranica. Zbog toga ovaj bit nije dovoljan da bi se odredilo da li pri upisu u neku stranicu iz seta, tu stranicu treba kopirati. Stranice koje pri upisu stvarno treba kopirati su one koje pripadaju regionu za koji je postavljen bit CoW i kojima je u trenutku upisa bitom W u PMT zabranjen upis. Kada jednom dođe do upisa i samim tim i do kopiranja stranice, za novu kopiju se setuje bit W u PMT i pri kasnijim upisima u tu stranicu ne treba vršiti kopiranje te stranice bez obzira što stranica pripada regionu za koji je setovan CoW bit. Stoga bi se sistem mogao implementirati tako da se u CoW bit jednom upiše vrednost pri pokretanju procesa i više nikada ne menja. U takvoj varijanti, CoW bit bi čak bio suvišan jer bi uvek imao istu vrednost kao i W bit u VMStruct strukturi.

2. zadatak, prvi kolokvijum, april 2010.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava straničnu organizaciju virtuelne memorije. Virtuelna adresa je 64-bitna, stranica je veličine 64KB, a adresibilna jedinica je bajt. Fizički adresni prostor je veličine 4GB. U tabeli preslikavanja stranica (PMT) svaki ulaz zauzima samo onoliko prostora koliko je potrebno da se smesti broj okvira u koji se data stranica preslikava, pošto se informacije o smeštaju stranica na disku čuvaju u drugoj strukturi. Pri tome, vrednost 0 u ulazu u PMT označava da data stranica nije učitana u fizičku memoriju (nijedna stranica korisničkog procesa ne preslikava se u okvir 0 gde se inače nalazi interapt vektor tabela). Trenutno stanje PMT dva posmatrana procesa A i B je sledeće (brojevi ulaza i vrednosti su dati heksadecimalno):

Proces A:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	FE12	FEFF	0	0	0	0	14	FE	0

Proces B:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0	12	2314	01AD	0	22	01AE	0	0

Prikazati sadržaj ovih tabela nakon što operativni sistem završi sve sledeće akcije tim redom:

- Obradio straničnu grešku (*page fault*) koju je generisao proces A kada je adresirao adresu FE030203 u svom adresnom prostoru, tako što je izbacio stranicu broj 2 procesa B i na njeno mesto učitao traženu stranicu procesa A.
- Obradio straničnu grešku (*page fault*) koju je generisao proces B kada je adresirao adresu 02FEFF u svom adresnom prostoru, tako što je izbacio stranicu broj FF procesa A i na njeno mesto učitao traženu stranicu procesa B.
- Obradio sistemski poziv procesa A kojim je taj proces zahtevao deljenje svoje stranice broj 2 sa stranicom broj 1 procesa B.

Rešenje

Proces A:

Ulaz	0	1	2	3	...	FE	FF	100	...	FE03	...
Vrednost	FE12	FEFF	12	0	0	0	0	FE	0	2314	0

Proces B:

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0	12	14	01AD	0	22	01AE	0	0

4. zadatak, drugi kolokvijum, maj 2010.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor sistema je 4GB, adresibilna jedinica je bajt, a virtuelni adresni prostor je organizovan stranično sa stranicom veličine 64KB. Fizički adresni prostor je veličine 4GB. Deskriptor stranice koji se čuva u tabeli preslikavanja (PMT) i koga hardver za preslikavanje adresa učitava i koristi za proveru prisutnosti stranice i preslikavanje sadrži samo broj okvira u fizičkoj memoriji u koji se data stranica preslikava, s tim da vrednost 0 označava da data stranica nije u memoriji, pošto se stranica nikada ne preslikava u okvir 0 u fizičkoj memoriji koji je rezervisan za interapt vektor tabelu; ostale informacije o stranici operativni sistem čuva u zasebnim strukturama van PMT. Tabele preslikavanja stranica su organizovane u dva nivoa, s tim da tabela prvog nivoa ima 1K ulaza. Posmatra se proces koji koristi prvih 400 stranica i poslednjih 64 stranica svog virtuelnog adresnog prostora, dok su mu ostale zabranjene za pristup jer ih ne koristi. Odgovoriti na sledeća pitanja i precizno obrazložiti odgovore:

1. Prikazati logičku strukturu virtuelne adrese i označiti širinu svakog polja. Označiti i podelu polja za broj stranice na polja za indeksiranje PMT prvog i drugog nivoa.
2. Kolika je veličina jednog ulaza u PMT prvog nivoa i šta taj ulaz sadrži?
3. Koliko bi maksimalno ukupno memorije zauzimale PMT nekog procesa koji bi koristio ceo svoj virtuelni adresni prostor?
4. Koliko ukupno memorije zauzimaju PMT za opisani proces?

Rešenje

1. Virtuelni adresni prostor: $4\text{GB} = 2^2 \cdot 2^{30}\text{B} = 2^{32}\text{B}$, pa je virtuelna adresa širine 32 bita.

Fizički adresni prostor: $4\text{GB} = 2^{32}\text{B}$, pa je fizička adresa širine 32 bita.

Veličina stranice i okvira: $64\text{KB} = 2^6 \cdot 2^{10}\text{B} = 2^{16}\text{B}$, pa je širina polja za pomeraj unutar stranice i okvira 16 bita.

Odatle sledi da je širina polja unutar virtuelne adrese za broj stranice $32-16 = 16$ bita, širina polja za broj okvira unutar fizičke adrese $32-16 = 16$ bita, a širina deskriptora (ulaza u PMT drugog nivoa) isto toliko – 16 bita, odnosno 2 bajta.

Stranica prvog nivoa ima $1\text{K} = 2^{10}$ ulaza, pa je širina polja za indeksiranje PMT prvog nivoa 10 bita, a za indeksiranje PMT drugog nivoa $16-10 = 6$ bita. Prema tome, struktura virtuelne adrese je: `Page_L1(10):Page_L2(6):Offset(16)`.

2. Ulaz u PMT prvog nivoa sadrži adresu početka PMT drugog nivoa u fizičkoj memoriji, s tim da vrednost 0 može da označava nekorisćeni opseg stranica (invalidan ulaz), pošto se ni PMT drugog nivoa ne može smestiti počev od adrese 0. Prema tome, širina ulaza u PMT prvog nivoa je najmanje jednaka širini fizičke adrese, što je 32 bita. Drugim rečima, jedan ulaz u PMT prvog nivoa zauzima 4 bajta.
3. PMT prvog nivoa zauzima 1K ulaza po 4 bajta, dakle 4KB.

Jedan ulaz u PMT drugog nivoa sadrži broj okvira, koji je širine 16 bita, pa zauzima 2 bajta. PMT drugog nivoa ima $2^6 = 64$ ulaza, pa zauzima 128B. Prema tome, PMT ukupno zauzimaju maksimalno: $4 \cdot 2^{10}\text{B}$ (veličina PMT prvog nivoa) + 2^{10} (broj PMT drugog nivoa) $\cdot 2^7\text{B}$ (veličina PMT drugog nivoa) = $2^{12}\text{B} + 2^{17}\text{B}$, što iznosi 132KB.

4. Dati proces ima validna samo prvih sedam i poslednji ulaz u PMT prvog nivoa, dakle za njega postoje samo osam PMT drugog nivoa u memoriji. Ukupna veličina PMT za ovaj proces je zato: $4 \cdot 2^{10}B$ (veličina PMT prvog nivoa) + 8 (broj PMT drugog nivoa) $\cdot 2^7B$ (veličina PMT drugog nivoa) = $4 \cdot 2^{10}B + 2^{10}B = 5KB$.

4. zadatak, drugi kolokvijum, maj 2009.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor sistema je 4GB, adresibilna jedinica je bajt, a virtuelni adresni prostor je organizovan stranično sa stranicom veličine 16KB. Fizički adresni prostor je veličine 1GB. Deskriptor stranice koji se čuva u tabeli preslikavanja (PMT) i koga hardver za preslikavanje adresa učitava i koristi za proveru prisutnosti stranice i preslikavanje sadrži samo broj okvira u fizičkoj memoriji u koji se data stranica preslikava, s tim da vrednost 0 označava da data stranica nije u memoriji, pošto se stranica nikada ne preslikava u okvir 0 u fizičkoj memoriji koji je rezervisan za interapt vektor tabelu; ostale informacije o stranici operativni sistem čuva u zasebnim strukturama van PMT. Tabele preslikavanja stranica su organizovane u dva nivoa, s tim da tabela prvog nivoa ima 2K ulaza. Posmatra se proces koji koristi prvih 800 stranica i poslednjih 56 stranica svog virtuelnog adresnog prostora, dok su mu ostale zabranjene za pristup jer ih ne koristi. Odgovoriti na sledeća pitanja i precizno obrazložiti odgovore:

1. Prikazati logičku strukturu virtuelne adrese i označiti širinu svakog polja. Označiti i podelu polja za broj stranice na polja za indeksiranje PMT prvog i drugog nivoa.
2. Kolika je veličina jednog ulaza u PMT prvog nivoa i šta taj ulaz sadrži?
3. Koliko bi maksimalno ukupno memorije zauzimale PMT nekog procesa koji bi koristio ceo svoj virtuelni adresni prostor?
4. Koliko ukupno memorije zauzimaju PMT za opisani proces?

Rešenje

1. Virtuelni adresni prostor: $4GB = 2^2 \cdot 2^{30}B = 2^{32}B$, pa je virtuelna adresa širine 32 bita. Fizički adresni prostor: $1GB = 2^{30}B$, pa je fizička adresa širine 30 bita. Veličina stranice i okvira: $16KB = 2^4 \cdot 2^{10}B = 2^{14}B$, pa je širina polja za pomeraj unutar stranice i okvira 14 bita. Odatle sledi da je širina polja unutar virtuelne adrese za broj stranice $32-14 = 18$ bita, širina polja za broj okvira unutar fizičke adrese $30-14 = 16$ bita, a širina deskriptora (ulaza u PMT drugog nivoa) isto toliko – 16 bita, odnosno 2 bajta. Stranica prvog nivoa ima $2K = 2^{11}$ ulaza, pa je širina polja za indeksiranje PMT prvog nivoa 11 bita, a za indeksiranje PMT drugog nivoa $18-11 = 7$ bita. Prema tome, struktura virtuelne adrese je: Page_L1(11):Page_L2(7):Offset(14).
2. Ulaz u PMT prvog nivoa sadrži adresu početka PMT drugog nivoa u fizičkoj memoriji, s tim da vrednost 0 može da označava nekorisćeni opseg stranica (invalidan ulaz), pošto se ni PMT drugog nivoa ne može smestiti počev od adrese 0. Prema tome, širina ulaza u PMT prvog nivoa je najmanje jednaka širini fizičke adrese, što je 30 bita. Drugim rečima, jedan ulaz u PMT prvog nivoa zauzima 4 bajta.
3. PMT prvog nivoa zauzima 2K ulaza po 4 bajta, dakle 8KB. Jedan ulaz u PMT drugog nivoa sadrži broj okvira, koji je širine 16 bita, pa zauzima 2 bajta. PMT drugog nivoa ima $2^7 = 128$ ulaza, pa zauzima 256B. Prema tome, PMT ukupno zauzimaju maksimalno: $4 \cdot 2^{11}B$ (veličina PMT prvog nivoa) + 2^{11} (broj PMT drugog nivoa) $\cdot 2^8B$ (veličina PMT drugog nivoa) = $2^{13}B + 2^{19}B$, što iznosi 520KB.
4. Dati proces ima validna samo prvih sedam i poslednji ulaz u PMT prvog nivoa, dakle za njega postoje samo osam PMT drugog nivoa u memoriji. Ukupna veličina PMT za ovaj proces je zato: $4 \cdot 2^{11}B$ (veličina PMT prvog nivoa) + 8 (broj PMT drugog nivoa) $\cdot 2^8B$ (veličina PMT drugog nivoa) = $8 \cdot 2^{10}B + 2 \cdot 2^{10}B = 10KB$.

3. zadatak, prvi kolokvijum, april 2008.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava straničnu organizaciju virtuelne memorije. Virtuelni adresni prostor je veličine 4GB, stranica je veličine 256KB, a adresibilna jedinica je bajt. Fizički adresni prostor je veličine 16GB. U tabeli preslikavanja stranica (PMT) svaki ulaz zauzima samo onoliko prostora koliko je potrebno da se smesti broj okvira u koji se data stranica kojoj odgovara taj ulaz preslikava, pošto se informacije o smeštaju stranica na disku čuvaju u drugoj strukturi. Pri tome, vrednost -1 (sve jedinice) u ulazu u PMT označava da data stranica nije dozvoljena za pristup, jer proces nije deklariseo da koristi taj deo adresnog prostora, a vrednost 0 označava da data stranica jeste dozvoljena za pristup, ali nije u fizičkoj memoriji. (Naravno, prvi i poslednji okvir u fizičkom adresnom prostoru se zbog toga nikada ne

koriste za smeštanje stranica procesa.) Trenutno stanje PMT je sledeće (brojevi ulaza i vrednosti su dati heksa decimalno):

Ulaz	0	1	2	3	...	3FE	3FF	400	...
Vrednost	0	25F	0	FF0	-1	2AD0	0	14	-1

U donjoj tabeli date su virtuelne adrese koje se posmatraju nezavisno jedna od druge (ne u sekvenci, već se posmatra svaka za dato stanje PMT nezavisno od drugih, kao da druge nisu generisane).

Popuniti donju tabelu za svaku od datih virtuelnih adresa (zapis je heksadecimalan) na sledeći način: ukoliko je pristup toj virtuelnoj adresi nedozvoljen, odnosno ako se generiše izuzetak zbog nelegalnog pristupa memoriji (engl. *memory access violation*), u polje upisati MAV; ako data adresa generiše izuzetak tipa stranične greške (engl. *page fault*), upisati PF; inače, upisati vrednost (pune) fizičke adrese u koju se ova preslikava, u heksadecimalnom zapisu.

Virtuelna adresa	FFC2673	D385A	FF7FB32	FFFA8C4	10012EB8
Rezultat adresiranja					

Rešenje

Virtuelna adresa	FFC2673	D385A	FF7FB32	FFFA8C4	10012EB8
Rezultat adresiranja	PF	3FC1385A	MAV	PF	512EB8

4. zadatak, drugi kolokvijum, maj 2008.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor sistema je 4GB, adresibilna jedinica je bajt, a virtuelni adresni prostor je organizovan stranično sa stranicom veličine 4KB. Fizički adresni prostor je veličine 256MB. Deskriptor stranice koji se čuva u tabeli preslikavanja (PMT) i koga hardver za preslikavanje adresa učitava i koristi za proveru prisutnosti stranice i preslikavanje sadrži samo broj okvira u fizičkoj memoriji u koji se data stranica preslikava, s tim da vrednost 0 označava da data stranica nije u memoriji, pošto se stranica nikada ne preslikava u okvir 0 u fizičkoj memoriji koji je rezervisan za interapt vektor tabelu; ostale informacije o stranici operativni sistem čuva u zasebnim strukturama van PMT. Tabele preslikavanja stranica su organizovane u dva nivoa, s tim da tabela prvog nivoa ima 1K ulaza. Posmatra se proces koji koristi prve 1032 stranice i poslednjih 10 stranica svog virtuelnog adresnog prostora, dok su mu ostale zabranjene za pristup jer ih ne koristi. Odgovoriti na sledeća pitanja i precizno obrazložiti odgovore:

1. Prikazati logičku strukturu virtuelne adrese i označiti širinu svakog polja. Označiti i podelu polja za broj stranice na polja za indeksiranje PMT prvog i drugog nivoa.
2. Kolika je veličina jednog ulaza u PMT prvog nivoa i šta taj ulaz sadrži?
3. Koliko bi maksimalno ukupno memorije zauzimale PMT nekog procesa koji bi koristio ceo svoj virtuelni adresni prostor?
4. Koliko ukupno memorije zauzimaju PMT za opisani proces?

Rešenje

1. Virtuelni adresni prostor: $4GB = 22 \cdot 2^{30}B = 2^{32}B$, pa je virtuelna adresa širine 32 bita. Fizički adresni prostor: $256MB = 28 \cdot 2^{20}B = 2^{28}B$, pa je fizička adresa širine 28 bita. Veličina stranice i okvira: $4KB = 22 \cdot 2^{10}B = 2^{12}B$, pa je širina polja za pomeraj unutar stranice i okvira 12 bita.

Odatle sledi da je širina polja unutar virtuelne adrese za broj stranice $32-12 = 20$ bita, širina polja za broj okvira unutar fizičke adrese $28-12 = 16$ bita, a širina deskriptora (ulaza u PMT drugog nivoa) isto toliko – 16 bita, odnosno 2 bajta.

Stranica prvog nivoa ima $1K = 2^{10}$ ulaza, pa je širina polja za indeksiranje PMT prvog nivoa 10 bita, a za indeksiranje PMT drugog nivoa $20-10 = 10$ bita.

Prema tome, struktura virtuelne adrese je: Page_L1(10):Page_L2(10):Offset(12).

2. Ulaz u PMT prvog nivoa sadrži adresu početka PMT drugog nivoa u fizičkoj memoriji, s tim da vrednost 0 može da označava nekorisćeni opseg stranica (invalidan ulaz), pošto se ni PMT drugog nivoa ne može smestiti

počev od adrese 0. Prema tome, širina ulaza u PMT prvog nivoa je najmanje jednaka širini fizičke adrese, što je 28 bita. Drugim rečima, jedan ulaz u PMT prvog nivoa zauzima 4 bajta.

3. PMT prvog nivoa zauzima 1K ulaza po 4 bajta, dakle 4KB. Jedan ulaz u PMT drugog nivoa sadrži broj okvira, koji je širine 16 bita, pa zauzima 2 bajta. PMT drugog nivoa ima $2^{10} = 1K$ ulaza, pa zauzima 2KB. Prema tome, PMT ukupno zauzimaju maksimalno: $4 \cdot 2^{10}B$ (veličina PMT prvog nivoa) $+ 2^{10}$ (broj PMT drugog nivoa) $\cdot 2^{11}B$ (veličina PMT drugog nivoa) $= 4 \cdot 2^{10}B + 2^{21}B$, što je približno (odnosno nešto veće od) $2^{21}B = 2MB$.
4. Dati proces ima validna samo prva dva i poslednji ulaz u PMT prvog nivoa, dakle za njega postoje samo tri PMT drugog nivoa u memoriji. Ukupna veličina PMT za ovaj proces je zato: $4 \cdot 2^{10}B$ (veličina PMT prvog nivoa) $+ 3$ (broj PMT drugog nivoa) $\cdot 2^{11}B$ (veličina PMT drugog nivoa) $= 4 \cdot 2^{10}B + 6 \cdot 2^{10}B = 10KB$.

5. zadatak, drugi kolokvijum, maj 2008.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu primenjuje se tehnika *copy-on-write* i učitavanje stranica virtuelne memorije na zahtev (*demand paging*). Za potrebe preslikavanja stranica postoje dve odvojene strukture: tabela preslikavanja stranica (PMT) koju hardver za preslikavanje adresa jedino koristi i koja sadrži *minimum* potrebnih informacija za ovo hardversko preslikavanje, kako bi bila što manja, i struktura koju isključivo koristi operativni sistem i koja čuva ostale informacije o virtuelnom adresnom prostoru procesa. Drugim rečima, u PMT su uključene one i samo one informacije koje su potrebne hardveru za preslikavanje adresa. Za svaku od sledećih informacija navesti da li se nalazi u ovoj PMT ili ne:

- Indikator da li se stranica nalazi u fizičkoj memoriji.
- Indikator da li je stranica uopšte dozvoljena za pristup (registrovana kao korišćeni deo virtuelnog adresnog prostora).
- Indikator da li je hardveru dozvoljen upis u stranicu.
- Indikator da je logički dozvoljen upis u stranicu, ali je ona deljena sa *copy-on-write* semantikom.
- Indikator da li je stranica „prljava“, odnosno menjana od svog učitavanja.
- Broj okvira u koji se stranica preslikava.
- Broj bloka na particiji za zamenu stranica.

Rešenje

- Indikator da li se stranica nalazi u fizičkoj memoriji. Da
- Indikator da li je stranica uopšte dozvoljena za pristup (registrovana kao korišćeni deo virtuelnog adresnog prostora). Ne
- Indikator da li je hardveru dozvoljena upis u stranicu. Da
- Indikator da li je dozvoljen upis u stranicu, ali je ona deljena sa *copy-on-write* semantikom. Ne
- Indikator da li je stranica „prljava“, odnosno menjana od svog učitavanja. Da
- Broj okvira u koji se stranica preslikava. Da
- Broj bloka na particiji za zamenu stranica. Ne

2. zadatak, prvi kolokvijum, april 2007.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava straničnu organizaciju virtuelne memorije, pri čemu je virtuelni adresni prostor veličine 16GB, adresibilna jedinica je 32-bitna reč, a fizički adresni prostor je veličine 1GB. Stranica je veličine 1KB. Jedan ulaz u tabeli preslikavanja stranica (PMT) zauzima jednu reč, s tim da najviši bit ukazuje na to da li je stranica u memoriji ili ne, a najniži biti predstavljaju broj okvira u fizičkoj memoriji u koji se stranica preslikava.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svakog polja.
2. Na jeziku C napisati kod funkcije

```
void setPageDescr(unsigned int* pmt, unsigned int page, unsigned int frame);
```

koja u tabelu na čiji početak ukazuje dati pokazivač `pmt`, za stranicu sa datim brojem `page`, upisuje deskriptor tako da se ta stranica preslikava u okvir sa datim brojem `frame`.

Rešenje

1. VA: Page(24) Offset(8)
PA: Block(20) Offset(8)
2.

```
void setPageDescr(unsigned int* pmt, unsigned int page, unsigned int frame){
    pmt[page] = frame | ~((unsigned int)~0 / 2);
}
```

4. zadatak, drugi kolokvijum, maj 2007.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor sistema je 16EB (EB je oznaka za eksabajt, engl. exabyte, kilo- mega-giga-tera-peta-eksa), adresibilna jedinica je bajt, a virtuelni adresni prostor je organizovan stranično sa stranicom veličine 16MB. Fizički adresni prostor je veličine 1TB (terabajt). Deskriptor stranice koji se čuva u tabeli preslikavanja (PMT) i koga hardver za preslikavanje adresa učitava i koristi za proveru prisutnosti stranice i preslikavanje sadrži samo broj okvira u fizičkoj memoriji u koji se data stranica preslikava, s tim da vrednost 0 označava da data stranica nije u memoriji, pošto se stranica nikada ne preslikava u okvir 0 u fizičkoj memoriji koji je rezervisan za interapt vektor tabelu; ostale informacije o stranici operativni sistem čuva u zasebnim strukturama van PMT. Tabele preslikavanja stranica su organizovane u dva nivoa, s tim da tabela prvog nivoa ima 1M ulaza. Posmatra se proces koji koristi prvih 1K stranica i poslednjih 200 stranica svog virtuelnog adresnog prostora, dok su mu ostale zabranjene za pristup jer ih ne koristi. Odgovoriti na sledeća pitanja i precizno obrazložiti odgovore:

1. Prikazati logičku strukturu virtuelne adrese i označiti širinu svakog polja. Označiti i podelu polja za broj stranice na polja za indeksiranje PMT prvog i drugog nivoa.
2. Kolika je veličina jednog ulaza u PMT prvog nivoa i šta taj ulaz sadrži?
3. Koliko bi maksimalno ukupno memorije zauzimale PMT nekog procesa koji bi koristio ceo svoj virtuelni adresni prostor?
4. Koliko ukupno memorije zauzimaju PMT za opisani proces?

Rešenje

1. Virtuelni adresni prostor: $16EB = 2^4 \cdot 2^{60}B = 2^{64}B$, pa je virtuelna adresa širine 64 bita. Fizički adresni prostor: $1TB = 2^{40}B$, pa je fizička adresa širine 40 bita. Veličina stranice i okvira: $16MB = 2^4 \cdot 2^{20}B = 2^{24}B$, pa je širina polja za pomeraj unutar stranice i okvira 24 bita.

Odatle sledi da je širina polja unutar virtuelne adrese za broj stranice $64-24 = 40$ bita, širina polja za broj okvira unutar fizičke adrese $40-24 = 16$ bita, a širina deskriptora (ulaza u PMT drugog nivoa) isto toliko – 16 bita, odnosno 2 bajta.

Stranica prvog nivoa ima $1M = 2^{20}$ ulaza, pa je širina polja za indeksiranje PMT prvog nivoa 20 bita, a za indeksiranje PMT drugog nivoa $40 - 20 = 20$ bita. Prema tome, struktura virtuelne adrese je: Page_L1(20):Page_L2(20):Offset(24).

2. Ulaz u PMT prvog nivoa sadrži adresu početka PMT drugog nivoa u fizičkoj memoriji, s tim da vrednost 0 može da označava nekorisćeni opseg stranica (invalidan ulaz), pošto se ni PMT drugog nivoa ne može smestiti počev od adrese 0. Prema tome, širina ulaza u PMT prvog nivoa je jednaka širini fizičke adrese, što je 40 bita. Drugim rečima, jedan ulaz u PMT prvog nivoa zauzima 5 bajtova.
3. PMT prvog nivoa zauzima 1M ulaza po 5 bajtova, dakle 5MB. Jedan ulaz u PMT drugog nivoa sadrži broj okvira, koji je širine 16 bita, pa zauzima 2 bajta. PMT drugog nivoa ima $2^{20} = 1M$ ulaza, pa zauzima 2MB. Prema tome, PMT ukupno zauzima maksimalno: $5 \cdot 2^{20}B$ (veličina PMT prvog nivoa) $+ 2^{20}$ (broj PMT drugog nivoa) $\cdot 2^{21}B$ (veličina PMT drugog nivoa) $= 5 \cdot 2^{20}B + 2^{41}B$, što je približno (odnosno nešto veće od) $2^{41}B = 2TB$ (terabajta).

4. Dati proces ima validan samo prvi i poslednji ulaz u PMT prvog nivoa, dakle za njega postoje samo dve PMT drugog nivoa u memoriji. Ukupna veličina PMT za ovaj proces je zato: $5 \cdot 2^{20}B$ (veličina PMT prvog nivoa) + 2 (broj PMT drugog nivoa) $\cdot 2^{21}B$ (veličina PMT drugog nivoa) = $5 \cdot 2^{20}B + 4 \cdot 2^{20}B = 9MB$, što je značajno manje od fizičkog adresnog prostora.

4. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu tabele preslikavanja stranica virtuelne memorije (PMT) složene su u niz od N elemenata, tako da procesu sa identifikatorom i pripada tabela u i -tom elementu ovog niza. Da li vrednost koju treba upisati u registar procesora koji ukazuje na tabelu preslikavanja (MTP) treba da bude element strukture PCB? Precizno obrazložiti odgovor.

Rešenje

Ne treba, pošto je identifikator procesa ujedno i ulaz u opisani niz. Nije potrebno dodatno čuvati tu informaciju.

5. zadatak, drugi kolokvijum, maj 2006.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sa straničnom organizacijom virtuelne memorije u datom trenutku raspored stranica tri procesa po okvirima u fizičkoj memoriji izgleda redom ovako (oznake su složene redom po rastućem broju okvira, slova A, B i C označavaju procese kojima pripadaju stranice u tim okvirima, brojevi označavaju brojeve stranica tih procesa, K označava okvir koji pripada jezgru OS-a, a DLL označava okvir čiji je sadržaj DLL koga koriste sva tri procesa A, B i C):

K, A1, B3, C1, DLL, C0, B2, K, K

U virtuelnom adresnom prostoru procesa A dati DLL se nalazi na stranici broj 2, a u prostorima procesa B i C na stranici broj 5. Napisati kako izgledaju tabele preslikavanja stranica za sva tri procesa A, B i C (u tabelu upisati brojeve okvira i T ili F kao oznaku da li je stranica u memoriji ili ne, za prvih 6 stranica).

Strana	0	1	2	3	4	5
Proces A						
Proces B						
Proces C						

Rešenje

Strana	0	1	2	3	4	5
Proces A	F	T 1	T 4	F	F	F
Proces B	F	F	T 6	T 2	F	T 4
Proces C	T 5	T 3	F	F	F	T 4

Kontinualna alokacija

2. zadatak, prvi kolokvijum, mart 2024.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije, ali u jedinicama koje su blokovi memorije određene konstantne veličine (alokacija se uvek radi u delovima veličine zaokružene na cele blokove i poravnato na blokove). Za evidenciju slobodnih i zauzetih blokova memorije sistem koristi posebnu strukturu zapisanu u nizu `memMap` veličine `NumOfBlocks` (ukupan broj blokova dostupnih za alokaciju). Svaki element ovog niza odgovara jednom bloku memorije koja se evidentira ovim nizom: elementi ovog niza redom, jedan na jedan odgovaraju blokovima memorije koji se alociraju. Za određeni slobodan fragment memorije koji počinje u bloku sa rednim brojem `b` i koji je veličine `k` susednih blokova, element `memMap[b]` ima vrednost `k`, dok narednih `k-1` elemenata ovog niza ima vrednost 0. Slično, za zauzet segment memorije koji počinje u bloku sa rednim brojem `b` i ima veličinu `k` susednih blokova, element `memMap[b]` ima vrednost `-k` (negativna vrednost), dok narednih `k-1` elemenata ima vrednost 0. Inicijalno je element sa indeksom 0 ovog niza jednak `NumOfBlocks`, dok su svi ostali elementi niza jednaki 0 (cela memorija je slobodna).

Implementirati funkciju `alloc` koja treba da alocira deo memorije zadate veličine `size` susednih blokova po *first fit* algoritmu i vrati redni broj prvog bloka koji je alociran, a u slučaju da slobodnog prostora nema, vraća -1.

```
const int NumOfBlocks = ...;
int memMap[NumOfBlocks] = {NumOfBlocks};
int alloc (int size);
```

Rešenje

```
int alloc(int size) {
    if (size <= 0 || size > NumOfBlocks) return -1; // Exception
    for (int i = 0; i < NumOfBlocks; i++)
        if (memMap[i] >= size) {
            if (memMap[i] > size)
                memMap[i+size] = memMap[i]-size;
            memMap[i] = -size;
            return i;
        }
    return -1; // No free mem
}
```

Moguće je učiniti ovu implementaciju efikasnijom, tako da preskače sve blokove koji pripadaju istom zauzetom ili slobodnom segmentu čim detektuje prvi – ostavlja se čitaocu.

2. zadatak, prvi kolokvijum, avgust 2023.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije za procese, s tim da tu memoriju alocira uvek u celim blokovima veličine `BLK_SIZE`, kako bi ublažio eksternu fragmentaciju. Adrese su 32-bitne (tip `uint32`). U strukturi `PCB` polje `baseBlk` označava broj memorijskog bloka koji je prvi alociran za proces (proces zauzima uvek memoriju počev od pomeraja 0 tog bloka), a polje `numOfBlocks` označava broj blokova memorije koje proces zauzima; proces uvek zauzima cele alocirane blokove i poravnat je na početak bloka. Evidenciju o slobodnim blokovima sistem vodi u bit-vektoru `freeMemBlks`; svaki bit ovog vektora predstavlja jedan blok memorije (redom od nižih prema višim elementima niza i redom od najnižeg do najvišeg bita svakog 32-bitnog elementa niza, blokovi su numerisani počev od 0), a vrednost 1 označava da je blok slobodan.

1. Implementirati funkciju `getMemCtxt` koja treba da izračuna baznu adresu (i upiše je u parametar `base`) i najvišu dozvoljenu virtuelnu adresu (i upiše je u parametar `limit`) za dati proces. Ovu funkciju sistem poziva prilikom promene konteksta da bi izračunate vrednosti upisao u odgovarajuće registre procesora kada datom procesu dodeljuje procesor.

- Implementirati funkciju `isMemBlkFree` koja za dati redni broj memorijskog bloka vraća informaciju o tome da li je dati blok slobodan ili ne, kao i funkciju `allocMemBlk` koja memorijski blok sa zadatim brojem označava zauzetim.
- Implementirati funkciju `expand` koju sistem poziva kada tekući proces izazove izuzetak zbog prekoračenja najviše dozvoljene virtualne adrese. Ova funkcija treba da pokuša da dodeli još jedan dodatan memorijski blok datom procesu, ako takav postoji iza prostora koji proces već zauzima, proširujući tako memorijski prostor procesa; u tom slučaju ova funkcija treba da vrati 0. U suprotnom, ova funkcija treba da vrati -1. Najviši blok fizičke memorije je uvek zauzet od strane jezgra operativnog sistema i nikad se ne dodeljuje procesima.

```
extern uint32 freeMemBlks[];
extern size_t BLK_SIZE;
void getMemCtxt(PCB* pcb, uint32& base, uint32& limit);
bool isMemBlkFree(size_t num);
void allocMemBlk(size_t num);
int expand(PCB* pcb);
```

Rešenje

```
inline void getMemCtxt(PCB* pcb, uint32& base, uint32& limit) {
    base = pcb->baseBlk * BLK_SIZE;
    limit = pcb->numOfBlks * BLK_SIZE - 1;
}

inline bool isMemBlkFree(size_t num) {
    return freeMemBlks[num / 32] & (1 << (num % 32));
}

inline void allocMemBlk(size_t num) {
    freeMemBlks[num / 32] &= ~(uint32)(1 << (num % 32));
}

int expand(PCB* pcb) {
    size_t newBlk = pcb->baseBlk + pcb->numOfBlocks;
    if (!isMemBlkFree(newBlk)) return -1;
    allocMemBlk(newBlk);
    pcb->numOfBlocks++;
    return 0;
}
```

1. zadatak, kolokvijum, avgust 2021.

- [Postavka](#)
- [Rešenje](#)

Implementirati bibliotečnu funkciju `malloc` za dinamičku alokaciju memorijskog bloka veličine `size` (sve deklaracije date su dole). Ako nema dovoljno slobodne memorije, ova funkcija vraća `nullptr`.

Evidencija slobodnih blokova u virtualnom adresnom prostoru procesa koju vodi ovaj alokator organizovana je kao neuređena jednostruko ulančana lista sa glavom u `freeMemHead`. Svaki element liste je jedan slobodan deo memorije koji na početku ima zaglavlje tipa `BlockHeader` u kome je pokazivač na sledeći takav zapis, kao i veličina slobodnog bloka iza ovog zaglavlja (neto, bez veličine zaglavlja). Funkcija `malloc` najpre traži slobodan blok dovoljne veličine u ovoj svojoj evidenciji slobodnih blokova algoritmom *first fit*. Ako takav blok ne nađe, ona alokira nov logički segment veličine `BLOCK_SIZE` ili tražene veličine `size` (šta god je veće) u virtualnom adresnom prostoru sistemskim pozivom `mmap` i dodaje ga u svoju evidenciju. Ukoliko bi nakon alokacije ostao slobodan fragment veličine manje od `MIN_BLOCK_SIZE`, alokira se ceo slobodan blok (drugim rečima, ne ostavlja se slobodan fragment manji od `MIN_BLOCK_SIZE`).

Da bi funkcija `free` za oslobađanje memorijskog bloka znala njegovu veličinu, uz alokirani blok ostaje zaglavlje, s tim da funkcija `malloc` vraća pokazivač na slobodan blok ispred koga je zaglavlje, a ne na zaglavlje.

```

void* malloc (size_t size);
struct BlockHeader {
    BlockHeader* next;
    size_t size;
};
static BlockHeader* freeMemHead;
const size_t BLOCK_SIZE = ..., MIN_BLOCK_SIZE = ...;
void* mmap (void* addr, size_t size, int protection);
const int PROT_RD = ..., PROT_WR = ..., PROT_RW = PROT_RD | PROT_WR;

```

Rešenje

```

void* malloc (size_t size) {

    // Try to find an existing free block in the list (first fit):
    BlockHeader *blk = freeMemHead, *prev = nullptr;
    for (; blk!=nullptr; prev=blk, blk=blk->next)
        if (blk->size>=size) break;

    // If not found, allocate a new memory segment and add it to the list:
    if (blk == nullptr) {
        size_t sz = max(size,BLOCK_SIZE);
        blk = (BlockHeader*)mmap(nullptr,sz+sizeof(BlockHeader),PROT_RW);
        if (blk == nullptr) return nullptr; // No free memory
        if (prev) prev->next = blk;
        else freeMemHead = blk;
        blk->next = nullptr;
        blk->size = sz;
    }

    // Allocate the requested block:
    size_t remainingSize = blk->size - size;
    if (remainingSize >= sizeof(BlockHeader) + MIN_BLOCK_SIZE) {
        // A fragment remains
        blk->size = size;
        size_t offset = sizeof(BlockHeader) + size;
        BlockHeader* newBlk = (BlockHeader*)((char*)blk + offset);
        if (prev) prev->next = newBlk;
        else freeMemHead = newBlk;
        newBlk->next = blk->next;
        newBlk->size = remainingSize - sizeof(BlockHeader);
    } else {
        // No remaining fragment, allocate the entire block
        if (prev) prev->next = blk->next;
        else freeMemHead = blk->next;
    }
    blk->next = nullptr;
    return (char*)blk + sizeof(BlockHeader);
}

```

2. zadatak, drugi kolokvijum, jun 2019.

- [Postavka](#)
- [Rešenje](#)

Neki operativni sistem primenjuje kontinualnu alokaciju memorije i nudi sistemski poziv kojim proces može da zatraži „skupljanje“ prostora koji zauzima, tj. oslobađanje vršnog dela svog prostora koji je do sada zauzimao.

Implementirati operaciju

```
int shrink (PCB* pcb, size_t by);
```

koja se koristi u implementaciji ovog sistemskog poziva i koja treba da smanji alociran memorijski prostor procesa na čiji PCB ukazuje prvi argument za veličinu datu drugim argumentom. Pri tom, ovu priliku sistem po potrebi koristi i da izvrši lokalnu defragmentaciju na sledeći način: ako je neposredno ispred memorije koju proces zauzima već postojao slobodan fragment, a kako je njegovim „skupljanjem“ iza njega sigurno nastao slobodan fragment, proces se relocira na početak slobodnog fragmenta ispred njega, kako bi se ova dva slobodna fragmenta spojila u jedan. U slučaju uspeha, ova funkcija treba da vrati 0, a u slučaju neke greške -1.

U strukturi PCB postoje, između ostalog, i sledeća polja:

- `char* base`: početna (bazna) fizička adresa procesa u memoriji;
- `size_t size`: veličina memorijskog prostora koji proces trenutno zauzima.

Na raspolaganju su i sledeće funkcije:

- `mem_free(void* at, size_t sz)`: oslobađa prostor na adresi datoj prvim argumentom i veličine date drugim argumentom, uz spajanje sa slobodnim fragmenatima ispred i iza novonastalog po potrebi;
- `relocate(PCB* pcb, void* to)`: relocira dati proces na novo zadato mesto, uz svo neophodno ažuriranje strukture za evidenciju slobodne memorije.

Slobodni fragmenti dvostruko su ulančani u listu na čiji prvi element ukazuje `fmem_head`. Fragmenti su ulančani u listu po rastućem redosledu svojih početnih adresa. Svaki slobodni fragment predstavljen je strukturom `FreeMem` koja je smeštena na sam početak tog slobodnog fragmenta:

```
struct FreeMem {
    FreeMem* next; // Next in the list
    FreeMem* prev; // Previous in the list
    size_t size;   // Size of the free fragment
};
```

Rešenje

```
int shrink (PCB* pcb, size_t by) {
    if (pcb==0 || by>pcb->size) return -1; // Exception
    if (by==0) return 0; // Nothing to do
    pcb->size -= by;
    mem_free(pcb->base+pcb->size,by);
    FreeMem* above = (FreeMem*)(pcb->base+pcb->size);
    FreeMem* under = above->prev;
    if (under && ((char*)under+under->size == pcb->base)
        relocate(pcb,under);
    return 0;
}
```

2. zadatak, drugi kolokvijum, april 2018.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu koristi se kontinualna alokacija memorije. U PCB postoje sledeća polja:

- `char* mem_base_addr`: početna adresa u memoriji na koju je smešten proces;
- `size_t mem_size`: veličina dela memorije koju zauzima proces u jedinicama `sizeof(char)`.

Fragmenti slobodne memorije ulančani su u jednostruko ulančanu listu, uređenu po rastućem poretku adresa slobodnih fragmenata, pri čemu se svaki element te liste, tipa `FreeSegment`, smešta na sam početak slobodnog fragmenta. Glava ove liste je u globalnoj promenljivoj `mem_free_head`. Struktura `FreeSegment` izgleda ovako:

```
struct FreeSegment {
    FreeSegment* next; // Next free segment in the list
```



```
size_t size; // Total size of the free segment in sizeof(char)
};
```

Napisati funkciju `proc_relocate()` koja vrši relokaciju datog procesa sa ciljem obavljanja lokalizovane, ograničene kompakcije na sledeći način. Ako i samo ako i ispred i iza datog procesa u memoriji postoji slobodan fragment, onda se taj proces pomera na početak slobodnog fragmenta ispred njega, kako bi se slobodan prostor ta dva fragmenta spojio u jedan, iza tog procesa. Ukoliko je ova kompakcija urađena, funkcija vraća 1, inače vraća 0.

```
int proc_relocate (PCB* pcb);
```

Rešenje

```
int proc_relocate (PCB* pcb) {
    FreeMem *cur=fmem_head, *prev=0;
    for (; cur && (char*)cur+cur->size!=pcb->mem_base_addr;
        prev=cur, cur=cur->next);

    if (cur==0 || cur->next==0 ||
        (char*)cur->next!=pcb->mem_base_addr+pcb->size)
        // No room or no need for relocation
        return 0;

    // Relocate the process to cur
    // and join the two fragments (cur and cur->next):

    size_t new_size = cur->size + cur->next->size;
    FreeMem* new_next = cur->next->next;

    memcpy(cur,pcb->mem_base_addr,pcb->mem_size);
    pcb->mem_base_addr = cur;

    if (prev) {
        prev->next = (FreeMem*)(pcb->mem_base_addr+pcb->mem_size);
        prev->next->next = new_next;
        prev->next->size = new_size;
    } else {
        fmem_head = (FreeMem*)(pcb->mem_base_addr+pcb->mem_size);
        fmem_head->next = new_next;
        fmem_head->size = new_size;
    }

    return 1;
}
```

2. zadatak, drugi kolokvijum, jun 2018.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije sa *best-fit* algoritmom. Zapisi o slobodnim delovima memorije organizovani su u ulančanu listu uređenu neopadajuće po veličini slobodnih delova memorije. U svakom zapisu je informacija o adresi početka slobodnog dela memorije i njegovoj veličini. Dat je sadržaj ove liste u nekom trenutku (sve vrednosti su heksadecimalne).

Zapis broj	Adresa početka	Veličina
1	3510	20
2	3680	30
3	3470	50
4	35A0	90

1. Prikazati tu listu nakon alokacije dela memorije veličine 40h.
2. Prikazati tu listu nakon što se, posle akcije pod a), izvrši oslobađanje dela memorije veličine 70h, na adresi 3530h.
3. Prikazati tu listu nakon što se, posle akcije pod 2), izvrši kompakcija slobodnog prostora pomeranjem alociranih delova na sam početak prvog slobodnog dela. Iza poslednjeg slobodnog dela nalazi se memorijski prostor koji se ne koristi u ovoj alokaciji.

Rešenje

1.

Zapis broj	Adresa početka	Veličina
1	34B0	10
2	3510	20
3	3680	30
4	35A0	90

2.

Zapis broj	Adresa početka	Veličina
1	34B0	10
2	3680	30
3	3510	120

3.

Zapis broj	Adresa početka	Veličina
1	3550	160

2. zadatak, drugi kolokvijum, april 2017.

- [Postavka](#)
- [Rešenje](#)

Neki operativni sistem primenjuje kontinualnu alokaciju memorije i nudi sistemski poziv kojim proces može da zatraži proširenje prostora koji zauzima. Implementirati operaciju

```
int extend (PCB* pcb, size_t by);
```

koja se koristi u implementaciji ovog sistemskog poziva i koja treba da proširi memorijski prostor procesa na čiji PCB ukazuje prvi argument za veličinu datu drugim argumentom. Ova funkcija treba da pokuša proširenje prostora procesa najpre bez njegove relokacije, ako je moguće (ukoliko je prostor iza njega slobodan i dovoljne je veličine), a potom i uz relokaciju, ako je to neophodno i moguće. U slučaju uspeha, ova funkcija treba da vrati 0, a u slučaju neuspeha -1.

U strukturi PCB postoje, između ostalog, i sledeća polja:

- `void* base`: početna (bazna) fizička adresa procesa u memoriji;
- `size_t size`: veličina memorijskog prostora koji proces trenutno zauzima.

Na raspolaganju su i sledeće funkcije:

- `mem_allocate_at(size_t sz, void* at)`: alocira prostor veličine date prvim argumentom na (fizičkoj) adresi datoj drugim argumentom; u slučaju uspeha vraća 0, a u slučaju neuspeha (na datoj adresi nije slobodan prostor potrebne veličine) vraća -1;
- `mem_allocate(size_t sz, void** location)`: alocira prostor veličine date prvim argumentom gde god je to moguće; u slučaju uspeha vraća 0, a adresu alociranog prostora smešta u pokazivač na koji ukazuje drugi argument; u slučaju neuspeha (nema slobodnog prostora potrebne veličine) vraća -1;
- `mem_free(void* at, size_t sz)`: oslobađa prostor na adresi datoj prvim argumentom i veličine date drugim argumentom;
- `mem_copy(void* from, void* to, size_t sz)`: kopira memorijski sadržaj veličine date trećim argumentom sa prve na drugu adresu.

Rešenje

```

int extend (PCB* pcb, size_t by) {
    if (pcb==0) return -1; // Exception
    if (mem_allocate_at(by,pcb->base+pcb->size)) {
        // Extension without relocation
        pcb->size+=by;
        return 0;
    }
    void* newLocation;
    if (mem_allocate(pcb->size+by,&newLocation)) {
        // Extension with relocation
        mem_copy(pcb->base,newLocation,pcb->size);
        mem_free(pcb->base,pcb->size);
        pcb->base = newLocation;
        pcb->size+=by;
        return 0;
    }
    return -1; // No free memory, cannot extend
}

```

2. zadatak, drugi kolokvijum, maj 2015.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu koristi se kontinualna alokacija memorije. U PCB postoje sledeća polja:

- `char* mem_base_addr`: početna adresa u memoriji na koju je smešten proces;
- `size_t mem_size`: veličina dela memorije koju zauzima proces;
- `PCB* mem_next`: pokazivač na sledeći u listi procesa; PCB procesa su ulančani u ovu listu po redosledu njihovog smeštanja u memoriji (po rastućem redosledu `mem_base_addr`); glava ove liste je globalna promenljiva `proc_mem_head`.

Na početak dela operativne memorije koja se koristi za smeštanje procesa ukazuje pokazivač `user_proc_mem_start`, a na njegov kraj (na poslednji znak) pokazivač `user_proc_mem_end`; oba su tipa `char*`.

Fragmenti slobodne memorije ulančani su u jednostruko ulančanu listu, pri čemu se svaki element te liste, tipa `FreeSegment`, smešta na sam početak slobodnog fragmenta. Glava ove liste je u globalnoj promenljivoj `mem_free_head`. Struktura `FreeSegment` izgleda ovako:

```

struct FreeSegment {
    FreeSegment* next; // Next free segment in the list
    size_t size; // Total size of the free segment
};

```

Napisati proceduru `mem_compact()` koja vrši kompakciju slobodnog prostora relokacijom procesa.

Rešenje

```

void proc_relocate (PCB* pcb, char* to) {
    if (pcb->mem_base_addr==to) return;
    memcpy(to,pcb->mem_base_addr,pcb->mem_size);
    pcb->mem_base_addr = to;
}

void mem_compact () {
    if (mem_free_head==0) return; // No free memory, no need for compaction
    char* to = user_proc_mem_start;
    for (PCB* pcb=proc_mem_head; pcb!=0; pcb=pcb->mem_next) {

```

```

    proc_relocate(pcb,to);
    to+=pcb->mem_size;
}
size_t free_mem = user_proc_mem_end-to+1;
if (free_mem>=sizeof(FreeSegment)) {
    mem_free_head = (FreeSegment*)to;
    mem_free_head->size = free_mem;
    mem_free_head->next = 0;
}
else mem_free_head = 0; // No more free memory (should not ever happen)
}

```

2. zadatak, drugi kolokvijum, septembar 2015.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije. Slobodni fragmenti dvostruko su ulančani u listu na čiji prvi element ukazuje `fmem_head`. Fragmenti su ulančani u listu po rastućem redosledu svojih početnih adresa. Svaki slobodni fragment predstavljen je strukturom `FreeMem` koja je smeštena na sam početak tog slobodnog fragmenta:

```

struct FreeMem {
    FreeMem* next; // Next in the list
    FreeMem* prev; // Previous in the list
    size_t size;   // Size of the free fragment
};

```

Implementirati funkciju

```
int mem_free(char* address, size_t size);
```

Ova funkcija treba da dealocira zauzeti kontinualni segment memorije na datoj adresi i date veličine, uz eventualno (po potrebi) spajanje novooslobođenog fragmenta sa susednim slobodnim fragmentima ispred i iza njega.

Rešenje

Jednostavnije, ali manje efikasno rešenje:

```

// Helper: Try to join cur with the cur->next free segment:
int tryToJoin (FreeMem* cur) {
    if (!cur) return 0;
    if (cur->next && (char*)cur+cur->size == (char*)(cur->next)) {
        // Remove the cur->next segment:
        cur->size += cur->next->size;
        cur->next = cur->next->next;
        if (cur->next) cur->next->prev = cur;
        return 1;
    } else
        return 0;
}

int mem_free(char* addr, size_t size) {
    // Find the place where to insert the new free segment (just after cur):
    FreeMem* cur=0;
    if (!fmem_head || addr<(char*)fmem_head)
        cur = 0; // insert as the first
    else
        for (cur=fmem_head; cur->next!=0 && addr>(char*)(cur->next);
            cur=cur->next);
}

```

```

// Insert the new segment after cur:
FreeMem* newSeg = (FreeMem*)addr;
newSeg->size = size;
newSeg->prev = cur;
if (cur) newSeg->next = cur->next;
else newSeg->next = fmem_head;
if (newSeg->next) newSeg->next->prev = newSeg;
if (cur) cur->next = newSeg;
else fmem_head = newSeg;

// Try to merge with the previous and next segments:
tryToJoin(newSeg);
tryToJoin(cur);
}

```

Složenije, ali nešto efikasnije rešenje:

```

int mem_free(char* addr, size_t size) {
    // Find the place where to insert the new free segment (just after cur):
    FreeMem* cur=0;
    if (!fmem_head || addr<(char*)fmem_head)
        cur = 0; // insert as the first
    else
        for (cur=fmem_head; cur->next!=0 && addr>(char*)(cur->next);
            cur=cur->next);

    // Try to append it to the previous free segment cur:
    if (cur && (char*)cur+cur->size==addr) {
        cur->size+=size;
        // Try to join cur with the next free segment:
        if (cur->next && (char*)cur+cur->size == (char*)(cur->next)) {
            // Remove the cur->next segment:
            cur->size += cur->next->size;
            cur->next = cur->next->next;
            if (cur->next) cur->next->prev = cur;
        }
        return;
    }
    else

    // Try to append it to the next free segment:
    FreeMem* nxtSeg = cur?cur->next:fmem_head;
    if (nxtSeg && addr+size==(char*)nxtSeg) {
        FreeMem* newSeg = (FreeMem*)addr;
        newSeg->size = nxtSeg->size+size;
        newSeg->prev=nxtSeg->prev;
        newSeg->next=nxtSeg->next;
        if (nxtSeg->next) nxtSeg->next->prev = newSeg;
        if (nxtSeg->prev) nxtSeg->prev->next = newSeg;
        else fmem_head = newSeg;
        return;
    }
    else

    // No need to join; insert the new segment after cur:
    FreeMem* newSeg = (FreeMem*)addr;
    newSeg->size = size;

```

```

newSeg->prev = cur;
if (cur) newSeg->next = cur->next;
else newSeg->next = fmem_head;
if (newSeg->next) newSeg->next->prev = newSeg;
if (cur) cur->next = newSeg;
else fmem_head = newSeg;
}

```

2. zadatak, drugi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije. Slobodni fragmenti dvostruko su ulančani u listu na čiji prvi element ukazuje `fmem_head`. Svaki slobodni fragment predstavljen je strukturom `FreeMem` koja je smeštena na sam početak tog slobodnog fragmenta:

```

struct FreeMem {
    FreeMem* next; // Next in the list
    FreeMem* prev; // Previous in the list
    size_t size;   // Size of the free fragment
};

```

Implementirati funkciju

```
int mem_alloc(void* address, size_t by);
```

Ova funkcija pokušava da pronađe slobodan fragment na adresi datoj prvim argumentom i da iz njega alocira deo veličine date drugim argumentom. Ukoliko na datoj adresi ne počinje slobodan fragment ili on nije dovoljne veličine, ova funkcija vraća -1. Ukoliko na datoj adresi postoji slobodan fragment dovoljne veličine, ova funkcija ažurira listu slobodnih framenata na sledeći način:

- ukoliko bi iza alociranog dela preostao fragment koji je manji ili jednak veličini strukture `FreeMem`, ceo taj slobodni fragment se alocira (ne ostavlja se ostatak koji je premali za evidenciju) i izbacuje iz liste slobodnih;
- u suprotnom, preostali deo se ostavlja kao slobodan fragment.

U oba slučaja, funkcija vraća veličinu stvarno alociranog dela (veće ili jednako traženoj veličini).

Rešenje

```

int mem_alloc(void* addr, size_t size) {
    for (FreeMem* cur=fmem_head; cur!=0; cur=cur->next) {
        if (cur!=addr || cur->size<size) continue;
        // Found
        if (cur->size-size<=sizeof(FreeMem)) {
            // No remaining fragment
            if (cur->prev) cur->prev->next = cur->next;
            else fmem_head = cur->next;
            if (cur->next) cur->next->prev = cur->prev;
            return cur->size;
        } else {
            FreeMem* newfrgm = (FreeMem*)((char*)cur+size);
            if (cur->prev) cur->prev->next = newfrgm;
            else fmem_head = newfrgm;
            if (cur->next) cur->next->prev = newfrgm;
            newfrgm->prev = cur->prev;
            newfrgm->next = cur->next;
            newfrgm->size = cur->size-size;
            return size;
        }
    }
}

```

```

}
return -1;
}

```

3. zadatak, drugi kolokvijum, april 2013.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije za procese. U strukturi PCB procesa postoje sledeća polja:

- `Word* mem_base`: pokazivač na lokaciju u memoriji na kojoj je proces smešten; tip `Word` predstavlja jednu adresibilnu jedinicu memorije;
- `size_t mem_size`: veličina memorijskog segmenta koju zauzima proces; `size_t` je celobrojni tip za izražavanje veličina memorijskih segmenata, u jedinicama `Word`.

Sistemske pozivom `mem_extend` proces može da traži povećanje svog memorijskog prostora za traženu veličinu (proširenje iza kraja). Sistem će ispuniti ovaj zahtev ukoliko iza prostora koji proces već zauzima postoji slobodan segment dovoljne veličine. U suprotnom, odbije taj zahtev (ne radi relokaciju procesa).

Slobodni segmenti ulančani su u listu, a funkcija

```
int mem_alloc(Word* address, size_t by);
```

pokušava da pronađe slobodan segment na adresi datoj prvim argumentom i da iz njega alokira deo veličine date drugim argumentom. U slučaju uspeha, funkcija vraća 0. Ukoliko na datoj adresi ne počinje slobodan segment ili on nije dovoljne veličine, ova funkcija vraća negativnu vrednost (kod greške).

Korišćenjem već implementirane funkcije `mem_alloc`, implementirati funkciju

```
int mem_extend (PCB* p, size_t by);
```

koja se koristi u implementaciji opisanog sistemskog poziva `mem_extend`. Ova funkcija treba da vrati 0 u slučaju uspeha, a negativnu vrednost u slučaju greške.

Rešenje

```

int mem_extend (PCB* p, size_t by) {
    // Error: invalid argument
    if (p == 0 || by < 0) return -1;
    if (by == 0) return 0;
    Word* tail = p->mem_base + p->mem_size;
    // Error: allocation failed
    if (mem_alloc(tail, by) < 0) return -2;
    // Extend
    p->mem_size += by;
    return 0;
}

```

3. zadatak, drugi kolokvijum, septembar 2012.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije sa *best-fit* algoritmom. Zapisi o slobodnim delovima memorije organizovani su u ulančanu listu uređenu neopadajuće po veličini slobodnih delova memorije. U svakom zapisu je informacija o adresi početka slobodnog dela memorije i njegovoj veličini. Dat je sadržaj ove liste u nekom trenutku (sve vrednosti su heksadecimalne). Prikazati tu listu nakon oslobađanja dela memorije veličine 70h, na adresi 2520h.

Zapis broj	Adresa početka	Veličina
1	2500	20
2	2670	30
3	2460	40
4	2590	90

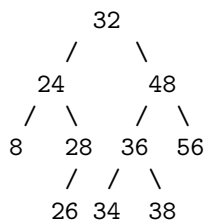
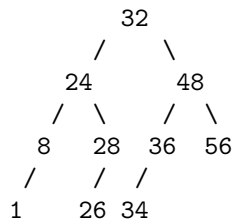
Rešenje

Zapis broj	Adresa početka	Veličina
1	2670	30
2	2460	40
3	2500	120

4. zadatak, drugi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Neki sistem primenjuje kontinualnu alokaciju memorije sa *best-fit* algoritmom. Radi efikasnijeg pronalaženja najpovoljnijeg dela memorije, sistem održava binarno stablo slobodnih fragmenata memorije, pri čemu svaki čvor ovog binarnog stabla predstavlja jedan slobodni fragment, u levom podstablu su manji, a u desnom veći fragmenti. Na slici je prikazano stanje ovog stabla u datom trenutku (u čvor je upisana veličina fragmenta u jedinicama alokacije). Prikazati stanje ovog stabla nakon alokacije memorije veličine 37 jedinica.

**Rešenje****5. zadatak, drugi kolokvijum, maj 2010.**

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu primenjuje se kontinualna alokacija operativne memorije. Deo definicije strukture PCB je sledeći:

```

struct PCB {
    ...
    void* memLocation; // Current place in memory
    size_t memSize;    // Size of memory space
};

```

Implementirati operaciju:

```

void relocate(PCB* process, void* newPlace);

```

kojom sistem premešta memorijski prostor procesa sa tekućeg na novozadato (već alocirano) mesto u memoriji.

U datom sistemu, operacija

```
void free (void* addr, size_t size);
```

deallocira (proglašava slobodnim) memorijski prostor veličine `size` počev od adrese `addr`.

Pomoć: Standardna bibliotečna funkcija `memcpy` ima sledeću deklaraciju:

```
void* memcpy (void* destination, const void* source, size_t size);
```

Rešenje

```
void relocate(PCB* p, void* newPlace) {
    if (p==0) return; // Exception!
    if (newPlace==p->memLocation || p->memSize==0) return; // Nothing to do
    memcpy(newPlace,p->memLocation,p->memSize); // Move memory contents
    free(p->memLocation,p->memSize); // Free the old memory space
    p->memLocation=newPlace; // Move relocation register
}
```

5. zadatak, drugi kolokvijum, maj 2009.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu primenjuje se kontinualna alokacija operativne memorije, uz primenu algoritma *first fit* i alokaciju u jedinicama veličine 1KB. Ukupan prostor za alokaciju memorije za korisničke procese je veličine 128KB. Posmatra se sledeća sekvenca zahteva za alokacijom i dealokacijom memorije:

$A + 45, B + 23, C + 38, B-, D + 16, E + 16, F + 5, C-, G + 25$

U oznaci svakog zahteva prvo slovo označava proces koji izdaje zahtev, simbol „+“ označava operaciju alokacije memorije, simbol „-“, označava operaciju dealokacije memorije, dok broj označava veličinu memorije koja se alocira izraženu u KB.

1. Popuniti sledeću tabelu vrednostima koje opisuju stanje zauzetosti memorije nakon ove sekvence:

Proces	A	D	E	F	G
Adresa početka ($\cdot 2^{10}$)					

2. Broj slobodnih fragmenata je _____ .

Ukupna veličina slobodne memorije je _____ .

Veličina najvećeg slobodnog fragmenta je _____ .

Veličina najmanjeg slobodnog fragmenta je _____ .

Rešenje

1.

Proces	A	D	E	F	G
Adresa početka ($\cdot 2^{10}$)	0	45	106	61	66

2. Broj slobodnih fragmenata je 2.

Ukupna veličina slobodne memorije je 21 KB.

Veličina najvećeg slobodnog fragmenta je 15 KB.

Veličina najmanjeg slobodnog fragmenta je 6 KB.

5. zadatak, drugi kolokvijum, maj 2007.

- [Postavka](#)
- [Rešenje](#)

U nekom operativnom sistemu primenjuje se kontinualna alokacija operativne memorije. Alokator održava spisak slobodnih delova memorije kao ulančanu listu uređenih parova (adresa početka slobodnog dela, veličina slobodnog dela). U nekom trenutku lista ima sledeće elemente redom (sve vrednosti su heksadecimalne):

(7F1A, 24), (1FFC, 42), (A000, 20), (0770, 7A), (4010, 68), (3A0A, A0), (01B0, 30)

Koji deo će biti alociran ako se traži alokacija prostora veličine 47h (navesti samo adresu slobodnog dela koji je odabran), ako sistem primenjuje sledeći algoritam alokacije:

1. *first-fit*
2. *best-fit*
3. *worst-fit*.

Rešenje

1. 0770h
2. 4010h
3. 3A0Ah

Segmentna organizacija

2. zadatak, prvi kolokvijum, mart 2023.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu sa segmentnom organizacijom memorije adresibilna jedinica je bajt, virtuelni adresni prostor je velične 1 MB, a maksimalna veličina fizičkog segmenta je 4 KB. Dat je spisak početnih (virtuelnih) adresa, veličina (obe vrednosti su zapisane heksadecimalno) i vrsta logičkih segmenata (regiona) koje je alocirao neki proces.

Adresa segmenta (hex)	Veličina (hex)	Vrsta segmenta
0	2890	instrukcije
3000	FF0	konstantni podaci inicijalizovani statički
4000	3E68	instrukcije
28000	189A	promenljivi podaci
FE000	2000	stek

1. U datu tabelu upisati parametre svih fizičkih segmenata koje je operativni sistem organizovao u SMT (sve ulaze u SMT koji nisu *null*) za ovaj proces, po rastućem redosledu broja segmenta. Broj segmenta i granicu segmenta (*limit*, najvišu dozvoljenu vrednost pomeraja u segmentu koji se sme adresirati) zapisati heksadecimalno, a bite prava pristupa *RWX* binarno tim redom. (Broj redova u datoj tabeli ne mora da odgovara broju segmenata koje treba upisati; eventualni višak redova date tabele ostaviti prazne.)
2. Ako stek raste ka nižim adresama, koja virtuelna adresa je prva koja je van dozvoljenog opsega i nije dozvoljena za adresiranje ako stek prekorači svoj dozvoljeni kapacitet?

Odgovor (hex): _____

Kom fizičkom segmentu pripada ta adresa?

Odgovor (hex): _____

Segment # (hex)	Limit (hex)	RWX (bin)

Slika 1: Tabela data uz stavku 1 drugog zadatka.

Rešenje

Segment # (hex)	Limit (hex)	RWX (bin)
0	FFF	001
1	FFF	001
2	88F	001
3	FEF	100
4	FFF	001
5	FFF	001
6	FFF	001
7	E67	001
28	FFF	110
29	899	110
FE	FFF	110
FF	FFF	110

2. Adresa FDFFFh pripada segmentu broj FDh.

1. zadatak, prvi kolokvium, mart 2022.

- [Postavka](#)
- [Rešenje](#)

Neki sistem vodi evidenciju alociranih (deklarisanih) logičkih segmenata (regiona) za svaki proces u jednostruko ulančanoj listi elemenata tipa `RegionDesc` uređenih po početnoj virtuelnoj adresi segmenta (polje `addr`). Svaki logički segment opisuje jedan deskriptor tipa `RegionDesc` u kom polje `size` sadrži veličinu segmenta.

```
struct RegionDesc {
    byte* addr;
    size_t size;
    RegionDesc* next;
};
```

```
int createRegion(RegionDesc** phead, byte* addr, size_t sz);
```

Implementirati funkciju `createRegion` koja se koristi u implementaciji sistemskog poziva `mmap`. Ova funkcija treba da napravi nov deskriptor logičkog segmenta koji se deklarise na zadatoj adresi (`addr`) tražene veličine (`sz`) i uključi ga u evidenciju segmenata procesa, ali samo ako na traženom mestu u virtuelnom adresnom prostoru procesa ima dovoljno prostora između već alociranih segmenata, odnosno ako se traženi segment ne preklapa ni sa jednim već deklarisanim. Ukoliko to nije slučaj, ova funkcija treba da vrati grešku (negativnu vrednost). Najveća adresa u virtuelnom adresnom prostoru je `MAX_VADDR`. Argument `phead` je pokazivač na glavu liste deskriptora segmenata datog procesa. Tip `byte` predstavlja celobrojni tip veličine jedne adresibilne jedinice. Pretpostaviti da su argumenti `addr` i `size` po potrebi već poravnati na stranice. Dinamičku alokaciju prostora za potrebe struktura jezgra radi funkcija `kmalloc` koja ima isti potpis i dejstvo kao standardna C funkcija `malloc`.

Rešenje

```
int createRegion(RegionDesc** phead, byte* addr, size_t sz) {
    if (!phead || !sz || addr + sz - 1 > MAX_VADDR) {
        return -1;
    }
    RegionDesc *prev = nullptr, *next = *phead;
    while (next && addr >= next->addr) {
        prev = next;
        next = next->next;
    }
    if (prev && prev->addr + prev->size > addr) {
        return -1;
    }
}
```

```

    if (next && addr + sz > next->addr) {
        return -1;
    }
    RegionDesc* dsc = (RegionDesc*) kmalloc(sizeof(RegionDesc));
    if (!dsc) {
        return -2;
    }
    dsc->addr = addr;
    dsc->size=sz;
    dsc->next = next;
    if (prev) {
        prev->next = dsc;
    } else {
        *phead = dsc;
    }
    return 0;
}

```

3. zadatak, prvi kolokvijum, mart 2022.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi segmentnu organizaciju memorije sa fizičkim segmentom maksimalne veličine 128 KB. Adresibilna jedinica je 16-bitna reč, virtuelni adresni prostor je veličine 8 GB, a fizički adresni prostor je veličine 1 GB. U deskriptoru segmenta u SMT najviša tri bita predstavljaju prava pristupa (RWX), u bitima do njih je maksimalna dozvoljena adresa unutar segmenta (*limit*), dok je u najnižim bitima bazna fizička adresa segmenta; ako segment nije alociran, ceo deskriptor ima vrednost 0.

1. Prikazati logičku strukturu virtuelne adrese i označiti širinu svih polja.
2. Implementirati funkciju `shareSeg` koja se koristi u implementaciji sistemskog poziva kojim se zahteva logičko deljenje segmenta broj `seg1` procesa čija je SMT data parametrom `smt1` sa segmentom `seg2` procesa čija je SMT data parametrom `smt2`, tako da ovaj drugi već postoji, a ovaj prvi postaje deljen sa drugim. Ukoliko drugi proces nije alocirao segment `seg2` ili je prvi proces već alocirao segment `seg1`, ova funkcija treba da vrati grešku (negativnu vrednost). Tip `long` je širine 64 bita, tip `int` 32 bita, a tip `short` 16 bita.

```

typedef unsigned long SMT[SMT_SIZE];
int shareSeg(SMT smt1, unsigned seg1, SMT smt2, unsigned seg2);

```

Rešenje

1. VA: Segment(16):Offset(16)
2.

```

int shareSeg(SMT smt1, unsigned seg1, SMT smt2, unsigned seg2) {
    if (smt1[seg1] || !smt2[seg2]) {
        return -1;
    }
    smt1[seg1] = smt2[seg2];
    return 0;
}

```

1. zadatak, prvi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

Neki sistem vodi evidenciju alociranih (deklarisanih) logičkih segmenata (regiona) za svaki proces u jednostruko ulančanoj listi elemenata tipa `RegionDesc` uređenih po početnoj virtuelnoj adresi segmenta (polje `addr`). Svaki logički segment opisuje jedan deskriptor tipa `RegionDesc` u kom polje `size` sadrži veličinu segmenta. Ova lista je

uvek neprazna, jer kernel za svaki proces, prilikom njegovog pokretanja, alokira jedan logički segment na najnižim adresama virtuelnog adresnog prostora (počev od adrese 0) koji preslikava u svoj memorijski prostor.

```
struct RegionDesc {
    byte* addr;
    size_t size;
    RegionDesc* next;
};

void* createRegion(RegionDesc* phead, size_t sz);
```

Implementirati funkciju `createRegion` koja se koristi u implementaciji sistemskog poziva `mmap` sa parametrom `start_addr` jednakim `NULL`. Ova funkcija treba da alokira (deklariše) nov logički segment tražene veličine (`sz`) na bilo kom mestu u virtuelnom adresnom prostoru procesa u kom ima dovoljno prostora tražene veličine tako da se ne preklapa sa već alociranim segmentima i uključi ga u evidenciju segmenata procesa. U slučaju uspeha funkcija vraća početnu adresu alocirano segmenta, a u slučaju neuspeha vraća `null`. Najveća adresa u virtuelnom adresnom prostoru je `MAX_VADDR`. Argument `phead` je glava liste deskriptora segmenata datog procesa. Tip `byte` predstavlja celobrojni tip veličine jedne adresibilne jedinice. Ne treba poravnavati adrese i veličine segmenata. Dinamičku alokaciju prostora za potrebe struktura jezgra radi funkcija `kmalloc` koja ima isti potpis i dejstvo kao standardna C funkcija `malloc`.

Rešenje

```
void* createRegion(RegionDesc* phead, size_t sz) {
    if (!phead || !sz) {
        return nullptr;
    }
    RegionDesc* prev = phead;
    while (prev->next && prev->addr + prev->size + sz > prev->next->addr) {
        prev = prev->next;
    }
    byte* addr = prev->addr + prev->size;
    if (!prev->next && addr + sz - 1 > MAX_VADDR) {
        return nullptr;
    }
    RegionDesc* dsc = (RegionDesc*)kmalloc(sizeof(RegionDesc));
    if (!dsc) {
        return nullptr;
    }
    dsc->addr = addr;
    dsc->size = sz;
    dsc->next = prev->next;
    prev->next = dsc;
    return addr;
}
```

1. zadatak, kolokvijum, jun 2021.

- [Postavka](#)
- [Rešenje](#)

Neki sistem ima segmentnu organizaciju memorije. Virtuelna adresa je 32-bitna, adresibilna jedinica je bajt, a maksimalna veličina fizičkog segmenta je 1 MB. Fizički adresni prostor je veličine 1 TB. Jedan ulaz u SMT-u sadrži prava pristupa *rwX* u najniža tri bita, granicu fizičkog segmenta (*limit* u opsegu od 0 do maksimalne veličine segmenta minus 1) u bitima do njih, a zatim fizičku adresu u bitima do njih; ovaj deskriptor (ulaz u SMT-u) zauzima najmanji potreban ceo broj bajtova.

Operativni sistem alokira segmente na zahtev, tako da pri kreiranju procesa ne alokira i ne učitava nijedan fizički segment. Vrednost 0 u polju za fizičku adresu u deskriptoru segmenta u SMT-u označava da preslikavanje nije moguće (segment nije alocirano ili nije učitano).

Jedan alocirani logički segment procesa opisan je deskriptorom tipa `SegDesc` u kom su, između ostalog, sledeća polja:

- `unsigned startAddr`: početna virtuelna adresa logičkog segmenta, svakako poravnata na početak fizičkog segmenta;
- `unsigned size`: veličina logičkog segmenta u bajtovima (može biti i veća od maksimalne veličine fizičkog segmenta);
- `unsigned short rwx`: prava pristupa za ceo logički segment u tri najniža bita.

Implementirati sledeću funkciju:

```
void initSegment (SegDesc* sd, unsigned long* smt);
```

Ovu funkciju poziva kod kernela kada inicijalizuje SMT novokreiranog procesa za svaki logički segment sa datim deskriptorom `sd`. Na već alocirani SMT ukazuje `smt`. Veličine tipova su sledeće: `int` – 32 bita, `long` – 64 bita, `short` – 16 bita.

Rešenje

- VA(32): Segment(12). Offset(20)
- PA(40)

```
const unsigned SEG_WIDTH = 12, OFFS_WIDTH = 20,
              MAX_SEG_SIZE = 1U<<OFFS_WIDTH;
inline void setSMTEntry(unsigned long* smt,
                       unsigned seg,
                       unsigned limit,
                       unsigned long paddr,
                       unsigned short rwx) {
    smt[seg] = (paddr<<(OFFS_WIDTH+3)) | (limit<<3) | rwx;
}

void initSegment(SegDesc* sd, unsigned long* smt) {
    unsigned size = sd->size;
    unsigned saddr = sd->startAddr;
    while (size > 0) {
        setSMTEntry(smt, saddr>>OFFS_WIDTH,
                   (size>MAX_SEG_SIZE?MAX_SEG_SIZE-1:size-1), 0, sd->rwx);
        if (size < MAX_SEG_SIZE) break;
        size -= MAX_SEG_SIZE;
        saddr += MAX_SEG_SIZE;
    }
}
```

2. zadatak, kolokvijum, oktobar 2020.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi segmentnu organizaciju memorije. Slobodne fragmente operativne memorije sistem opisuje strukturama tipa `FreeSegDesc` za svaki proces. Kako bi pretraga pri alokaciji memorije bila efikasnija, ovi deskriptori organizovani su u uređeno binarno stablo, tako da su u levom podstablu svakog čvora deskriptori slobodnih fragmenata koji su manji ili jednaki, a desno onih koji su veći od fragmenta datog čvora. U strukturi `FreeSegDesc` polja `left` i `right` ukazuju na koren levog odnosno desnog podstabla, polje `addr` sadrži početnu adresu fragmenta, a polje `sz` sadrži veličinu fragmenta. Tip `size_t` je neoznačen celobrojni tip dovoljno velik da predstavi veličinu adresnog prostora.

```
struct FreeSegDesc {
    FreeSegDesc *left, *right;
    size_t sz;
    void* addr;
```

```
...
};
```

Implementirati operaciju koja pretragom u stablu na čiji koren ukazuje prvi parametar pronalazi i vraća deskriptor slobodnog fragmenta u koji se može alocirati segment zadate veličine, a *null* ako takvog nema, i to radi primenom *best fit* algoritma alokacije. Ova operacija ne treba da ažurira ni stablo ni pronađeni deskriptor, već samo da pronađe i vrati pokazivač na odgovarajući deskriptor.

```
SegDesc* findSegDesc (SegDesc* root, size_t size);
```

Rešenje

```
SegDesc* findSegDesc (SegDesc* root, size_t size) {
    SegDesc *sd = root, *bestFit = nullptr;
    while (sd) {
        if (sd->sz==size) return sd;
        else
            if (sd->sz<size) sd = sd->right;
        else {
            bestFit = sd;
            sd = sd->left;
        }
    }
    return bestFit;
}
```

3. zadatak, kolokvijum, septembar 2020.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi segmentnu organizaciju memorije. Slobodne fragmente operativne memorije sistem opisuje strukturama tipa `FreeSegDesc` za svaki proces. Kako bi pretraga pri alokaciji memorije bila efikasnija, ovi deskriptori organizovani su u uređeno binarno stablo, tako da su u levom podstablu svakog čvora deskriptori slobodnih fragmenata koji su manji ili jednaki, a desno onih koji su veći od fragmenta datog čvora. U strukturi `FreeSegDesc` polja `left` i `right` ukazuju na koren levog odnosno desnog podstabla, polje `addr` sadrži početnu adresu fragmenta, a polje `sz` sadrži veličinu fragmenta. Tip `size_t` je neoznačen celobrojni tip dovoljno velik da predstavi veličinu adresnog prostora.

```
struct FreeSegDesc {
    FreeSegDesc *left, *right;
    size_t addr, sz;
    ...
};
```

Implementirati operaciju koja pretragom u stablu na čiji koren ukazuje prvi parametar pronalazi i vraća deskriptor slobodnog fragmenta u koji se može alocirati segment zadate veličine, a *null* ako takvog nema, i to radi primenom *first fit* algoritma alokacije. Ova operacija ne treba da ažurira ni stablo ni pronađeni deskriptor, već samo da pronađe i vrati pokazivač na odgovarajući deskriptor.

```
SegDesc* findSegDesc (SegDesc* root, size_t size);
```

Rešenje

```
SegDesc* findSegDesc(SegDesc* root, size_t size) {
    SegDesc* sd = root;
    while (sd != nullptr) {
        if (sd->sz >= size) return sd;
        else sd = sd->right;
    }
}
```



```

    return nullptr;
}

```

2. zadatak, prvi kolokvijum, mart 2014.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentnu organizaciju virtuelne memorije, pri čemu je virtuelna adresa 16-bitna, fizički adresni prostor je veličine 4GB, a adresibilna jedinica je bajt. Maksimalna veličina segmenta je 4KB. U sledećoj tabeli dat je sadržaj nekoliko ulaza u tabeli preslikavanja nekog procesa (sve vrednosti su heksadecimalne):

Segment#	Size	In memory?	Starting physical address
1	500	Yes	250000
2	600	Yes	AFF00
A	700	No	-
B	800	No	-
C	900	Yes	D6700

Popuniti sledeću tabelu na sledeći način: ako data virtuelna adresa uzrokuje grešku prekoračenja - napisati X, ukoliko izaziva straničnu grešku (*page fault*) - napisati P, a ukoliko je preslikavanje uspešno, upisati fizičku adresu u koju se preslikava (heksadecimalno).

Virtual address (hex)	Mapping result (hex)
12FA	
C0FF	
1675	
B014	
CDAB	

Rešenje

Virtual address (hex)	Mapping result (hex)
12FA	2502FA
C0FF	D67FF
1675	X
B014	P
CDAB	X

2. zadatak, prvi kolokvijum, april 2014.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentnu organizaciju virtuelne memorije, pri čemu je virtuelna adresa 20-bitna, fizički adresni prostor je veličine 16MB, a adresibilna jedinica je bajt. Maksimalna veličina segmenta je 4KB. U sledećoj tabeli dat je sadržaj nekoliko ulaza u tabeli preslikavanja nekog procesa (sve vrednosti su heksadecimalne):

Segment#	Size	In memory?	Starting physical address
70	500	Yes	250000
12	600	Yes	AFF000
A0	700	No	-
B0	800	No	-
C0	900	Yes	D67000

Popuniti sledeću tabelu na sledeći način: ako data virtuelna adresa uzrokuje grešku prekoračenja - napisati X, ukoliko izaziva straničnu grešku (*page fault*) - napisati P, a ukoliko je preslikavanje uspešno, upisati fizičku adresu u koju se preslikava (heksadecimalno).

Virtual address (hex)	Mapping result (hex)
12FA0	
C00F0	
70750	
B0140	
C02AB	

Rešenje

Virtual address (hex)	Mapping result (hex)
12FA0	X
C00F0	D670F0
70750	X
B0140	P
C02AB	D672AB

2. zadatak, prvi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava segmentnu organizaciju virtuelne memorije. Virtuelni adresni prostor je veličine 16MB, maksimalna veličina segmenta je 256B, a adresibilna jedinica je bajt. Fizički adresni prostor je veličine 16MB. U tabeli preslikavanja segmenata (SMT) svaki ulaz zauzima samo onoliko prostora koliko je potrebno da se smesti početna adresa segmenta i stvarna veličina segmenta umanjena za 1 (najveći pomeraj koji se unutar segmenta sme generisati), pošto se informacije o smeštaju segmenata na disku čuvaju u drugoj strukturi. Pri tome, vrednost 0 u polju za stvarnu veličinu segmenta u ulazu u SMT označava da dati segment nije dozvoljen za pristup, jer proces nije deklarirao da koristi taj deo adresnog prostora, a za segmente koje proces sme da koristi, vrednost 0 u polju za fizičku adresu segmenta označava da dati segment trenutno nije u fizičkoj memoriji. Naravno, nijedan segment ne sme početi pre fizičke adrese 1. Trenutno stanje SMT je sledeće (brojevi ulaza i vrednosti su dati heksadecimalno):

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0245	01000140	0127	5400094C	0	CDD01227	34	402001A9	0

U donjoj tabeli date su virtuelne adrese koje se posmatraju nezavisno jedna od druge (ne u sekvenci, već se posmatra svaka za dato stanje SMT nezavisno od drugih, kao da druge nisu generisane). Popuniti donju tabelu za svaku od datih virtuelnih adresa (zapis je heksadecimalan) na sledeći način: ukoliko je pristup toj virtuelnoj adresi nedozvoljen, odnosno ako se generiše izuzetak zbog nelegalnog pristupa memoriji (engl. *memory access violation*), u polje upisati MAV; ako data adresa generiše izuzetak tipa stranične greške (engl. *page fault*), upisati PF; ako data adresa upada u zamku (engl. *trap*) usled prekoračenja veličine segmenta, upisati T; inače, upisati vrednost (pune) fizičke adrese u koju se ova preslikava, u heksadecimalnom zapisu.

Virtuelna adresa	222	FF32	4002D8	FE3A	FE14
Rezultat adresiranja					

Rešenje

Virtuelna adresa	222	FF32	4002D8	FE3A	FE14
Rezultat adresiranja	000023	PF	MAV	T	CDD026

3. zadatak, prvi kolokvijum, april 2009.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava segmentnu organizaciju virtuelne memorije. Virtuelni adresni prostor je veličine 4GB, maksimalna veličina segmenta je 4MB, a adresibilna jedinica je bajt. Fizički adresni prostor je veličine 64MB. U tabeli preslikavanja segmenata (SMT) svaki ulaz zauzima samo onoliko prostora koliko je potrebno da se smesti početna adresa segmenta i stvarna veličina segmenta umanjena za 1 (najveći pomeraj koji se unutar segmenta sme generisati), pošto se informacije o smeštaju stranica na disku čuvaju u drugoj strukturi. Pri tome, vrednost 0 u

polju za stvarnu veličinu segmenta u ulazu u SMT označava da dati segment nije dozvoljen za pristup, jer proces nije deklarirao da koristi taj deo adresnog prostora, a za segmente koje proces sme da koristi, vrednost 0 u polju za fizičku adresu segmenta označava da dati segment trenutno nije u fizičkoj memoriji. Naravno, nijedan segment ne sme početi pre fizičke adrese 1. Trenutno stanje SMT je sledeće (brojevi ulaza i vrednosti su dati heksadecimalno):

Ulaz	0	1	2	3	...	FE	FF	100	...
Vrednost	0245	1000140	0127	5400094C	0	2CDD00027	034	43C002001A9	0

U donjoj tabeli date su virtuelne adrese koje se posmatraju nezavisno jedna od druge (ne u sekvenci, već se posmatra svaka za dato stanje SMT nezavisno od drugih, kao da druge nisu generisane). Popuniti donju tabelu za svaku od datih virtuelnih adresa (zapis je heksadecimalan) na sledeći način: ukoliko je pristup toj virtuelnoj adresi nedozvoljen, odnosno ako se generiše izuzetak zbog nelegalnog pristupa memoriji (engl. *memory access violation*), u polje upisati MAV; ako data adresa generiše izuzetak tipa stranične greške (engl. *page fault*), upisati PF; ako data adresa upada u zamku (engl. *trap*) usled prekoračenja veličine segmenta, upisati T; inače, upisati vrednost (pune) fizičke adrese u koju se ova preslikava, u heksadecimalnom zapisu.

Virtuelna adresa	80013A	EB28C32	3F80028D	403001E2	F0A001E4
Rezultat adresiranja					

Rešenje

Virtuelna adresa	80013A	EB28C32	3F80028D	403001E2	F0A001E4
Rezultat adresiranja	PF	MAV	DC4 (28D+B37)	T	MAV

4. zadatak, drugi kolokvijum, maj 2006.

- [Postavka](#)
- [Rešenje](#)

Virtuelni adresni prostor sistema je 1GB, adresibilna jedinica je bajt, a virtuelni adresni prostor je organizovan segmentno. Svaki proces može imati najviše 64 segmenta. Fizički adresni prostor je veličine 512MB. Prostor za zamenu (*swap space*) na disku je duplo veći od fizičke memorije i koristi se za čuvanje zamenjenih segmenata kojima se direktno pristupa na disku (na „presnoj“ particiji, ne kroz fajl-sistem). Pristup segmentima se kontroliše pomoću tri bita zaštite koji se označavaju sa R (*read*), W (*write*) i E (*execute*). Kada se stranica nalazi u memoriji, prostor koji je zauzimala na disku se oslobađa za zamenu drugih stranica. Odgovoriti na sledeće pitanja i kratko, ali precizno obrazložiti odgovor: ako se deskriptori segmenata u tabeli preslikavanja segmenata (SMT) maksimalno kompaktno smeštaju, kolika je veličina SMT za jedan proces?

Rešenje

Virtuelna adresa (VA), 30 bita: Segment (6) : Offset(24)

Fizička adresa (PA), 29 bita

Swap 1GB $\Rightarrow$ Disk adr. (30)

Deskriptor stranice: Da li je u memoriji (1) : RWE(3) : Size(24) : Segment adr. (29) ili Disk Adr. (30)

Odatle sledi da je deskriptor segmenta veličine 58 bita, SMT ima po jedan ulaz za svaki segment, što znači 64 ulaza. Tako da je veličina SMTa $58 \cdot 64 / 8 \text{ B} = 464 \text{ B}$

Napomena: Nije logično da se deskriptori pakuju tako „gusto“, tako da se jedan deskriptor rasprostire u više bajtova/reči, jer se njima pristupa prilikom preslikavanja adresa, što bi bilo neefikasno. Nije nemoguće, ali nije baš ni logično. Tako da je rešenje u kome se jedan deskriptor zaokruže na ceo broj bajtova/reči takođe prihvatljivo. Odnosno veličina deskriptora $64 \text{ b} = 8 \text{ B} \Rightarrow$ veličina SMT $8 \cdot 64 \text{ B} = 512 \text{ B}$. Na ovaj način je SMT veća za 10%, ali su zato performanse drastično bolje.

Segmentno-stranična organizacija

2. zadatak, prvi kolokvijum, maj 2022.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu adresibilna jedinica je bajt, virtuelni adresni prostor je veličine 4 GB, a fizički adresni prostor je veličine 1 GB. Sistem koristi segmentno-straničnu organizaciju memorije sa stranicom veličine 1 KB i segmentom maksimalne veličine 4 MB. Jedan ulaz u SMT zauzima dve 32-bitne reči; u nižoj je pokazivač (cela adresa) na PMT (ili 0 ako segment nije alocirano), a u višoj reči su prava pristupa (RWX) u najnižih tri bita i maksimalan dozvoljen broj stranice u segmentu (*limit*) u bitima do njih; preostali biti se ne koriste. Jedan ulaz u PMT zauzima jednu 32-bitnu reč (koriste se najniži biti).

Da bi obradio systemske pozive u kojima se neki parametar zadaje kao pokazivač (virtuelna adresa u adresnom prostoru pozivajućeg procesa), operativni sistem mora da konvertuje datu virtuelnu u fizičku adresu, jer se kod kernela izvršava u režimu bez preslikavanja adresa. Implementirati funkciju koja obavlja ovu konverziju (vraća *null* ako virtuelna adresa ne pripada alociranoj delu virtuelnog adresnog prostora). Prvi parametar ove funkcije je pokazivač na SMT pozivajućeg procesa. Pretpostaviti da je `uint32` deklarisan kao neoznačeni 32-bitni celobrojni tip. Napisati sve potrebne deklaracije za tipove SMT i PMT.

```
void* v2pAddr(SMT pmt, void* vaddr);
```

Rešenje

```
const uint32 offsetw = 10;
const uint32 pagew = 12;
const uint32 PMT_size = 4096;
const uint32 SMT_size = 1024;

typedef uint32 PMT[PMT_SIZE];
typedef uint32 SMT[SMT_SIZE][2];

void* v2pAddr(SMT smt, void* vaddr) {
    uint32 seg = (uint32)vaddr >> (pagew + offsetw);
    uint32 page = ((uint32)vaddr >> offsetw) & ~((uint32)-1 << pagew);
    uint32 offset = (uint32)vaddr & ~((uint32)-1 << offsetw);
    uint32* pmt = (uint32*)smt[seg][0];
    if (!pmt) {
        return nullptr;
    }
    uint32 limit = smt[seg][1] >> 3;
    if (page > limit) {
        return nullptr;
    }
    uint32 frame = pmt[page];
    if (!frame) {
        return nullptr;
    }
    uint32 paddr = (frame << offsetw) + offset;
    return (void*)paddr;
}
```

3. zadatak, prvi kolokvijum, jun 2022.

- [Postavka](#)
- [Rešenje](#)

Neki sistem ima segmentno-straničnu organizaciju memorije. Virtuelna adresa je 32-bitna, adresibilna jedinica je

bajt, a maksimalna veličina fizičkog segmenta je 16 MB. Stranica je veličine 64 KB. Fizički adresni prostor je veličine 4 GB. Jedan ulaz u SMT-u zauzima dve 32-bitne reči; u prvoj (nižoj) su prava pristupa rwx u najviša tri bita, a granica fizičkog segmenta (*limit* u opsegu od 0 do maksimalno dozvoljenog broja stranice koji se sme adresirati) u najnižim bitima; druga (viša) reč ulaza u SMT-u sadrži 32-bitni pokazivač na PMT tog segmenta.

Operativni sistem alocira stranice na zahtev, tako da pri kreiranju procesa ne alocira nijednu stranicu. Vrednost 0 u polju za pokazivač na PMT u deskriptoru segmenta u SMT-u, kao i u deskriptoru stranice u PMT-u označava da preslikavanje nije moguće.

Jedan alociran logički segment procesa opisan je deskriptorom tipa `SegDesc` u kom su, između ostalog, sledeća polja:

- `unsigned startAddr`: početna virtuelna adresa logičkog segmenta, svakako poravnata na početak fizičkog segmenta;
- `unsigned size`: veličina logičkog segmenta u bajtovima (može biti i veća od maksimalne veličine fizičkog segmenta);
- `unsigned short rwx`: prava pristupa za ceo logički segment u tri najniža bita.

Implementirati sledeću funkciju:

```
int initSegment(SegDesc* sd, unsigned long smt[][2]);
```

Ovu funkciju poziva kod kernela kada inicijalizuje SMT novokreiranog procesa za svaki logički segment sa datim deskriptorom `sd`. Na već alociran SMT ukazuje `smt`. Potrebno je inicijalizovati ulaze u SMT i napraviti pridružene PMT za sve korišćene fizičke segmente. Veličine tipova su sledeće: `int` – 32 bita, `long` – 64 bita, `short` – 16 bita.

Statička funkcija `PMT::alloc()` alocira prostor u memoriji kernela za smeštanje PMT jednog segmenta, inicijalizuje ceo taj PMT na 0 i vraća 32-pokazivač taj PMT. Kako bi se obezbedilo da bude dovoljno prostora za sve PMT-ove potrebne za sve fizičke segmente koje zauzima jedan logički segment, funkcija `initSegment` treba najpre da proveriti da li ima dovoljno prostora za njih. Ovu proveru obavlja statička funkcija `PMT::reserve(int segs)` koja proverava da li ima dovoljno prostora za PMT-ove `segs` segmenata, rezerviše taj prostor (da bi narednih `segs` poziva `PMT::alloc` sigurno uspeo) i vraća 0 u slučaju uspeha, a negativnu vrednost u slučaju nedostatka prostora. Funkcija `initSegment` treba da vrati 0 u slučaju uspeha, a negativnu vrednost u suprotnom.

Rešenje

- `VA(32): Segment(8).Page(8).Offset(16);`
- `PA: Frame(16).Offset(16)`

```
const unsigned SEG_WIDTH = 8, PAGE_WIDTH = 8, OFFS_WIDTH = 16,
               MAX_SEG_SIZE = 1U << PAGE_WIDTH << OFFS_WIDTH,
               PAGE_SIZE = 1U << OFFS_WIDTH;

inline void setSMTEntry(unsigned smt[][2], unsigned seg, unsigned limit,
                       unsigned short* pmt, unsigned short rwx) {
    smt[seg][0] = ((unsigned)rwx << 29) | limit;
    smt[seg][1] = (unsigned)pmt;
}

int initSegment(SegDesc* sd, unsigned smt[][2]) {
    unsigned size = sd->size;
    unsigned saddr = sd->startAddr;
    unsigned segs = (size + MAX_SEG_SIZE - 1) / MAX_SEG_SIZE;
    if (PMT::reserve(segs) < 0) {
        return -1;
    }
    while (size > 0) {
        unsigned short* pmt = PMT::alloc();
        unsigned limit = size > MAX_SEG_SIZE ? MAX_SEG_SIZE-1 : size-1;
        limit >>= OFFS_WIDTH;
        unsigned seg = saddr >> (PAGE_WIDTH + OFFS_WIDTH);
        setSMTEntry(smt, seg, limit, pmt, sd->rwx);
        if (size < MAX_SEG_SIZE) {
```

```

        break;
    }
    size -= MAX_SEG_SIZE;
    saddr += MAX_SEG_SIZE;
}
return 0;
}

```

2. zadatak, kolokvijum, avgust 2020.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi segmentno-straničnu organizaciju memorije. Logičke segmente koje je proces alocirao sistem opisuje strukturama tipa `SegDesc` za svaki proces. Kako bi se pretraga za logičkim segmentom kom odgovara adresirana stranica koja je generisala straničnu grešku što efikasnije izvršila, ovi deskriptori organizovani su u uređeno binarno stablo za svaki proces, tako da su u levom podstablu svakog čvora segmenti koji zauzimaju niže adrese, a desno oni koji zauzimaju više adrese od segmenta datog čvora. Logički segment je uvek poravnat i zaokružen na stranice. U strukturi `SegDesc` polja `left` i `right` ukazuju na koren levog odnosno desnog podstabla, polje `pg` sadrži broj prve stranice logičkog segmenta, a polje `sz` sadrži veličinu segmenta izraženu u broju stranica. Tip `size_t` je neoznačen celobrojni tip dovoljno velik da predstavi veličinu virtuelnog adresnog prostora.

```

struct SegDesc {
    SegDesc *left, *right;
    size_t pg, sz;
    ...
};

```

1. Implementirati operaciju koja pretragom u stablu na čiji koren ukazuje prvi parametar pronalazi i vraća deskriptor segmenta kom pripada data stranica, a *null* ako takvog nema:

```
SegDesc* findSegDesc (SegDesc* root, size_t page);
```

U strukturi PCB svakog procesa postoji polje `segdesc` tipa `SegDesc*` koje ukazuje na koren strukture stabla deskriptora logičkih segmenata tog procesa. U strukturi `SegDesc` postoji polje `vtp` (*virtual table pointer*) kao pokazivač na strukturu (tabelu) pokazivača na funkcije koje za dati tip logičkog segmenta implementiraju operacije koje kernel poziva u različitim situacijama. Ova struktura ima polje `loadPage` koje ukazuje na funkciju koja datu stranicu učitava u dati okvir i ima sledeći potpis (vraća negativnu vrednost u slučaju greške):

```
int loadPage (size_t page, size_t frame);
```

Postoji i globalna funkcija bez parametara `allocateFrame` koja alocira slobodan okvir u operativnoj memoriji i vraća njegov broj tipa `size_t`, a 0 u slučaju neuspeha. Konačno, postoji funkcija kernela koja postavlja vrednost ulaza u PMT procesa na čiji PCB ukazuje prvi parametar, za stranicu datu drugim parametrom, i u taj ulaz upisuje broj okvira dat trećim parametrom, a prava pristupa podešava prema zapisu u deskriptoru segmenta datom poslednjim parametrom:

```
void setPMTEntry (PCB* pcb, size_t page, size_t frame, SegDesc* sd);
```

2. Implementirati internu funkciju kernela `handlePageFault` koju kernel poziva kada treba da obradi straničnu grešku procesa na čiji PCB ukazuje prvi parametar, generisanu za stranicu datu drugim parametrom. Ova funkcija treba da vrati 0 u slučaju uspeha, a različite negativne kodove u slučaju različitih grešaka. Ova funkcija ne treba da rukuje stanjem procesa (to rade drugi delovi).

```
int handlePageFault (PCB* pcb, size_t page);
```

Rešenje

1.

```
SegDesc* findSegDesc (SegDesc* root, size_t page) {
    SegDesc* sd = root;
    while (sd) {
```

```

    if (sd->pg<=page && page<sd->pg+sd->sz) return sd;
    if (page<sd->pg) sd = sd->left;
    else sd = sd->right;
}
return nullptr;
}

2. int handlePageFault (PCB* pcb, size_t page) {
    SegDesc* sd = findSegDesc(pcb->segdesc,page);
    if (!sd) return -1; // Memory access violation
    size_t frame = allocateFrame();
    if (!frame) return -2; // No free memory
    int s = sd->vtp->loadPage(page,frame);
    if (s<0) return s; // Error loading page
    setPMTEntry(pcb,page,frame,sd);
    return 0;
}

```

2. zadatak, prvi kolokvijum, septembar 2014.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentno-straničnu organizaciju virtuelne memorije, pri čemu je virtuelni adresni prostor veličine 16GB, adresibilna jedinica je 32-bitna reč, a fizički adresni prostor je veličine 1GB. Maksimalna veličina segmenta je 64MB, a stranica je veličine 1KB.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svakog polja.
2. Napisati heksadecimalni kod fizičke adrese u koju se preslikava virtuelna adresa u segmentu 56h, stranici broj 34h u tom segmentu, reč broj DDh u toj stranici, ako se ta stranica preslikava u okvir broj FF00h.

Rešenje

1. VA: Segment(8):Page(16):Offset(8)
PA: Frame(20):Offset(8).
2. FF00DDh

2. zadatak, prvi kolokvijum, mart 2013.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentno-straničnu organizaciju virtuelne memorije, pri čemu je virtuelna adresa 16-bitna, fizički adresni prostor je veličine 8GB, a adresibilna jedinica je 16-bitna reč. Stranica je veličine 512B. Maksimalan broj segmenata u virtuelnom adresnom prostoru je 4. Prikazati logičku strukturu virtuelne i fizičke adrese i navesti širinu svakog polja.

Rešenje

VA(16): Segment(2):Page(6):Offset(8)
PA(32): Frame(24):Offset(8)

2. zadatak, prvi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentno-straničnu organizaciju virtuelne memorije, pri čemu je virtuelni adresni prostor veličine 8GB, adresibilna jedinica je 16-bitna reč, a fizički adresni prostor je veličine 512MB. Maksimalna veličina segmenta je 32MB, a stranica je veličine 512B.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svakog polja.
2. Napisati heksadecimalni kod fizičke adrese u koju se preslikava virtuelna adresa u segmentu 45h, stranici broj 23h u tom segmentu, reč broj FFh u toj stranici, ako se ta stranica preslikava u okvir broj FF00h.

Rešenje

1. VA: Segment(8):Page(16):Offset(8)
PA: Frame(20):Offset(8).
2. FF00FFh

2. zadatak, prvi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentno-straničnu organizaciju virtuelne memorije, pri čemu je virtuelni adresni prostor veličine 16GB, adresibilna jedinica je 32-bitna reč, a fizički adresni prostor je veličine 1GB. Maksimalna veličina segmenta je 64MB, a stranica je veličine 1KB.

1. Prikazati logičku strukturu virtuelne i fizičke adrese i označiti širinu svakog polja.
2. Napisati heksadecimalni kod fizičke adrese u koju se preslikava virtuelna adresa u segmentu 24h, stranici broj 45h u tom segmentu, reč broj 15h u toj stranici, ako se ta stranica preslikava u okvir broj FF00h.

Rešenje

1. VA: Segment(8):Page(16):Offset(8)
PA: Frame(20):Offset(8).
2. FF0015h

3. zadatak, prvi kolokvijum, april 2007.

- [Postavka](#)
- [Rešenje](#)

Neki sistem podržava segmentno-straničnu organizaciju virtuelne memorije. Za procese kreirane nad istim programom moguće je uštedeti na potrošnji fizičke memorije tako što se programski kod tog programa u fizičku memoriju smešta samo na jedno mesto, pri čemu se okviri u kojima se nalaze stranice koje pripadaju segmentima koji sadrže samo programski kod fizički dele između ovih procesa (zajednički su).

1. Precizno objasniti kako se ovo deljenje stranica može implementirati.
2. Čija je odgovornost da obezbedi ovo deljenje stranica, hardvera ili operativnog sistema?
3. Precizno objasniti zašto ovakvo deljenje nije moguće za stranice koje pripadaju segmentima koji sadrže stek.

Rešenje

1. Potrebno je da postoji tabela svih pokrenutih programa. Za svaki program u tabeli se čuva lista (pokazivač na prvi u listi) PCB-ova procesa koji su pokrenuti nad tim programom. Pri pokretanju novog, potrebno ga je uvezati u odgovarajuću listu. Ukoliko već postoji neki proces pokrenutim nad istim programom, potrebno je u SMT i PMT tabele novog procesa upisati trenutno stanje prisutnosti stranica u memoriji i odgovarajuće brojeve okvira. Jedan način da to bude moguće uraditi je da se uz svaki proces pamti u koji dio virtuelnog adresnog prostora tog procesa je učitani program.

Kada se neka stranica učitava u OM, potrebno je ažurirati deskriptore u svim PMT svih procesa koji dijele tu stranicu. Isto je i kada neka stranica bude izbačena iz OM. Moguće je i efikasnije rješenje (razmisliti o još po

jednoj tabeli po programu u kojoj se čuvaju deskriptori njegovih stranica, a da se u SMT/PMT procesa vrši redirekcija na ovu tabelu).

2. Opisani način se u potpunosti realizuje u OS.
3. Stek je dio konteksta svakog procesa i kao takav pravi razliku izmedju različitih izvršavanja jednog koda. Stek je struktura podataka u koju se podaci i upisuju, a ne samo čitaju. To znači da svaki proces ima svoj zaseban stek i iz tog razloga nije moguće djeljenje stranica sa stekom. (Stranice su male da bi u njih moglo stati više stekova koji se neće preklapati).

3. zadatak, prvi kolokvijum, april 2006.

- [Postavka](#)
- [Rešenje](#)

Neki računar podržava segmentno-stranični mehanizam virtuelne memorije, pri čemu je virtuelna adresa 16-bitna, fizički adresni prostor je veličine 8GB, a adresibilna jedinica je 16-bitna reč. Stranica je veličine 512B. Maksimalan broj segmenata u virtuelnom adresnom prostoru je 4. Prikazati logičku strukturu virtuelne i fizičke adrese i navesti širinu svakog polja.

Rešenje

- VA(16): seg(2), page(6), offset(8)
- PA(32): block(24), offset(8)

Dinamičko učitavanje

3. zadatak, prvi kolokvijum, april 2024.

- [Postavka](#)
- [Rešenje](#)

U nekom fajlu zapisana su dva veoma velika celobrojna niza iste zadate veličine `arr_size`. Najpre je, počev od bajta 0, zapisano `arr_size` elemenata niza `a` tipa `int`, a odmah iza toga isto toliko elemenata niza `b`. Iza toga je u fajlu obezbeđen prostor za smeštanje istog tolikog niza `c`. Celobrojni elementi su veličine 4 bajta, a u fajlu su zapisani u istom formatu u kom se i smeštaju u operativnu memoriju (niži bajt na nižoj adresi, *little endian*).

Potrebno je realizovati funkciju `arr_add` koja treba da sabere ova dva niza `a` i `b` (element po element) i njihov rezultat zapiše na mesto obezbeđeno za niz `c` u datom fajlu. Ona se realizuje za neki skroman mikroracunar za vrlo malo RAM-a, tako da za smeštanje (delova) nizova u memoriju ukupno ne treba utrošiti više od 32 KB RAM-a. Na raspolaganju su funkcije `read_block` i `write_block` koje iz fajla učitavaju, odnosno u fajl upisuju (respektivno) `size` celih brojeva smeštenih na adresu datu pokazivačem `buffer`, počev od bajta sa rednim brojem `offset` u navedenom fajlu.

```
void read_block(size_t offset, size_t size, int* buffer);
void write_block(size_t offset, size_t size, const int* buffer);
void arr_add(size_t arr_size);
```

Rešenje

```
void arr_add(size_t arr_size) {
    static const size_t ELEM_SIZE = 4;
    // 32KB / 2 / sizeof(int)
    static const size_t BLOCK_SIZE = (1 << 14) / ELEM_SIZE;
    static int a[BLOCK_SIZE], b[BLOCK_SIZE];
    size_t block = 0, remain = arr_size;
    while (remain > 0) {
        size_t size = (remain > BLOCK_SIZE) ? BLOCK_SIZE : remain;
        read_block(block * BLOCK_SIZE * ELEM_SIZE, size, &a);
        read_block(arr_size * ELEM_SIZE + block * BLOCK_SIZE * ELEM_SIZE, size, &b);
        for (size_t i = 0; i < size; i++) a[i] += b[i];
        write_block(2 * arr_size * ELEM_SIZE + block * BLOCK_SIZE * ELEM_SIZE, size, &a);
        remain = (remain > size) ? remain - size : 0;
        block++;
    }
}
```

3. zadatak, prvi kolokvijum, maj 2023.

- [Postavka](#)
- [Rešenje](#)

U nekom sistemu postoje sledeći sistemski pozivi vezani za deljene biblioteke sa dinamičkim vezivanjem (DLL):

- `int mapDLL(const char* dllName)`: po potrebi učitava DLL sa zadatim imenom i mapira ga u virtuelni adresni prostor pozivajućeg procesa; u slučaju uspeha, vraća pozitivnu celobrojnu vrednost koja predstavlja identifikator datog DLL-a u kontekstu procesa; u slučaju greške, vraća negativnu vrednost;
- `int mapDLLSymbol(int dll, const char* symbolName)`: u DLL-u koji je prethodno mapiran pronalazi simbol sa zadatim imenom i vraća njegovu (virtuelnu) adresu; u slučaju neuspeha, vraća `null`.

U DLL-u “mydll.dll” definisane su dve funkcije, `f1` i `f2`, čiji su potpisi dati dole. Simboli za njih kodovani su za povezivanje kao `f1@int@int*@int` i `f2@double@X*`, respektivno (ime fje – povrtani tip – tipovi parametara). Korišćenjem datih sistemskih poziva realizovati “patrljke” (*stub*) za ove dve funkcije koje program treba da poziva da bi pristupio njihovim implementacijama u DLL-u. U slučaju greške pozvati funkciju `handleError` koja ima

isti potpis i ponašanje kao biblioteka funkcija `printf`, samo što ispis šalje na standardni izlaz za greške i potom završava pozivajući proces.

```
int f1(int*, int);
double f2(X*);
```

Rešenje

```
int getDLL() {
    static const char* dllName = "mydll.dll";
    static int dll = 0;
    if (dll == 0) dll = mapDLL(dllName);
    if (dll <= 0) handleError("DLL cannot be mapped: %s.\n", dllName);
    return dll;
}

int f1(int* arg0, int arg1) {
    static int (*pf)(int*, int) = 0;
    if (pf == 0) {
        int dll = getDLL();
        pf = mapDLLSymbol(dll, "f1@int@int*@int");
        if (pf == 0) handleError("DLL symbol cannot be mapped: %s.\n", "int f1(int*,int)");
    }
    return pf(arg0, arg1);
}

double f2(X* arg0) {
    static double (*pf)(X*) = 0;
    if (pf == 0) {
        int dll = getDLL();
        pf = mapDLLSymbol(dll, "f2@double@X*");
        if (pf == 0) handleError("DLL symbol cannot be mapped: %s.\n", "double f2(X*)");
    }
    return pf(arg0);
}
```

2. zadatak, drugi kolokvijum, maj 2019.

- [Postavka](#)
- [Rešenje](#)

Neki program koristi dinamičko učitavanje. Proces uvek zauzima kontinualan deo svog virtuelnog adresnog prostora određene potrebne veličine, počev od virtuelne adrese 0. Za svaki modul predviđen za dinamičko učitavanje prevodilac u glavnom modulu programa organizuje sledeću strukturu – deskriptor tog modula:

```
struct ModDesc {
    char* base; // Base address of the module
    size_t size; // Size of the module in sizeof(char)
    const char* name; // Name of the module's file
};
```

U polju `base` je početna virtuelna adresa modula, ukoliko je taj modul učitani; ako modul još nije učitani, ovo polje ima vrednost `null`. Polje `size` definiše veličinu modula, a polje `name` naziv fajla sa sadržajem modula.

1. Na jeziku C napisati pomoćnu funkciju `load_module` koja treba da proširi alocirani deo virtuelnog adresnog prostora procesa za veličinu datog modula i učita taj modul u to proširenje, pod sledećim pretpostavkama:

```
void load_module (ModDesc* mod);
```

- globalna promenljiva programa `mem_size` tipa `size_t` sadrži trenutnu veličinu zauzetog (alociranog) dela virtuelnog adresnog prostora (u odnosu na početnu virtuelnu adresu 0); ovaj prostor treba proširiti i modul učitati u to proširenje;

- `mem_extend(size_t)`: sistemski poziv koji proširuje alocirani deo virtuelnog adresnog prostora za zadanu veličinu;
 - `load(const char* filename, char* addr)`: sistemski poziv koji na zadanu adresu `addr` u virtuelnom adresnom prostoru procesa učitava sadržaj fajla sa zadatim imenom;
 - ignorisati greške u sistemskim pozivima (ukoliko ne može da izvrši uslugu traženu sistemskim pozivom, sistem gasi pozivajući proces).
2. Na assembleru nekog zamišljenog jednostavnog RISC procesora napisati kod koji koristi opisanu funkciju `load_module`, a koji prevodilac generiše za svaki poziv nekog potprograma `fun` koji se nalazi u nekom modulu koji se dinamički učitava, pod sledećim pretpostavkama:
- svi registri i pokazivači su 32-bitni, kao i sva polja u strukturi `ModDesc`; adresibilna jedinica je bajt;
 - pre izvršavanja traženog koda, stvarni argumenti potprograma `fun` već su stavljeni na stek, a u registru `R0` je adresa deskriptora modula (`ModDesc*`) u kome se nalazi potprogram `fun` (ovo prevodilac zna u toku prevođenja);
 - pomeraj (relativna adresa u odnosu na početak modula) potprograma `fun` je poznat prevodiocu u toku prevođenja; ovaj pomeraj označiti simboličkom konstantom `fun`.

Rešenje

1.

```
void load_module (ModDesc* mod) {
    mem_extend(mod->size); // Extend memory
    mod->base = (char*)0 + mem_size;
    mem_size += mod->size;
    load(mod->name, mod->base);
}
```
2.

```
load r1, [r0] ; Load module's base address
and r1, r1, r1 ; If loaded,
jnz call_fun ; call the subroutine
push r0 ; Else, load the module
call load_module
pop r0
load r1, [r0] ; Load module's base address
call_fun: call #fun[r1] ; Add the subroutine's offset
; to the module's base addr and call it
```

3. zadatak, drugi kolokvijum, septembar 2011.

- [Postavka](#)
- [Rešenje](#)

Klasa `GeoRegion`, čiji je interfejs dat u nastavku, apstrahuje geografski region i implementirana je u potpunosti. Objekti ove klase zauzimaju mnogo prostora u memoriji, pa se mogu učitavati po potrebi, dinamički. Ovo učitavanje obavlja statička operacija `GeoRegion::load`, pri čemu se objekat identifikuje datim nazivom geografskog regiona (niz znakova). Ostale operacije ove klase vraćaju vrednosti nekih svojstava geografskog regiona. Potrebno je u potpunosti implementirati klasu `GeoRegionProxy` čiji je interfejs dat u nastavku. Objekti ove klase služe kao posrednici (*proxy*) do objekata klase `GeoRegion`, pri čemu pružaju isti interfejs kao i „originali“, s tim da skrivaju detalje implementacije i tehniku dinamičkog učitavanja od svojih korisnika. Korisnici klase `GeoRegionProxy` vide njene objekte na sasvim uobičajen način, mogu ih kreirati datim konstruktorom i pozivati date operacije interfejsa, ne znajući da odgovarajuća struktura podataka možda nije učitana u memoriju.

```
class GeoRegion {
public:
    static GeoRegion* load (char* regionName);
    double getSurface ();
    double getHighestPeak ();
};

class GeoRegionProxy {
```

```
public:
    GeoRegionProxy (char* regionName);
    double getSurface ();
    double getHighestPeak ();
};
```

Rešenje

```
class GeoRegion {
public:
    static GeoRegion* load (char* regionName);
    double getSurface ();
    double getHighestPeak ();
};

class GeoRegionProxy {
public:
    GeoRegionProxy (char* regionName);
    double getSurface ();
    double getHighestPeak ();
protected:
    GeoRegion* getServer ();
private:
    GeoRegion* myServer;
    char* myName;
};

GeoRegionProxy::GeoRegionProxy (char* regionName)
    : myServer(0), myName(regionName) {}

GeoRegion* GeoRegionProxy::getServer() {
    if (myServer==0) {
        myServer=GeoRegion::load(myName);
        if (myServer==0) ... // Raise exception
    }
    return myServer;
}

double GeoRegionProxy::getSurface () {
    return getServer()->getSurface();
}

double GeoRegionProxy::getHighestPeak () {
    return getServer()->getHighestPeak();
}
```

3. zadatak, drugi kolokvijum, maj 2009.

- [Postavka](#)
- [Rešenje](#)

Sledeća dva potprograma čine jedan programski modul koji treba učitavati dinamički (engl. *dynamic loading*):

```
int f (int,int);
double g (double);
```

Ovi potprogrami implementirani su u jednom C fajlu p.c koji izgleda ovako:

```
int _f (int,int);
```

```
double _g (double);
void* map[2] = {&_f, &_g};

int _f (int x, int y){
    ...
}

double _g (double x) {
    ...
}
```

Ovaj modul biće preveden u fajl `p.obj`. Ovaj modul ne uvozi spoljašnje simbole koji su definisani u drugim modulima, već su u njemu sva adresiranja internih simbola. Napisati C deo koda glavnog modula programa sa *stub* procedurama za ove potprograme, koje obezbeđuju dinamičko učitavanje ovog modula. Na raspolaganju je sistemska usluga:

```
void* load_module(char* filename);
```

koja učitava izvršni kod (bez zaglavlja sa simbolima) iz `.obj` fajla sa datim imenom u adresni prostor pozivajućeg procesa i vraća adresu na koju je učitao taj modul, a prilikom učitavanja razrešava sva adresiranja internih simbola (u ovom slučaju elemente niza `map` postavlja na apsolutne adrese funkcija `_f` i `_g` određene prilikom učitavanja). Zanimariti moguće greške prilikom učitavanja modula.

Rešenje

```
static void* module_p = NULL;

int f (int x, int y) {
    if (module_p == NULL) module_p = load_module("p.obj");
    int (*_f)(int,int) = (int(*) (int,int))(module_p[0]);
    if (module_p!=NULL && _f != NULL) return _f(x,y);
}

double g (double x) {
    if (module_p == NULL) module_p = load_module("p.obj");
    double (*_g)(double) = (double(*) (double))(module_p[1]);
    if (module_p!=NULL && _g != NULL) return _g(x);
}
```

3. zadatak, drugi kolokvijum, maj 2007.

- [Postavka](#)
- [Rešenje](#)

Neki prevodilac koji podržava dinamičko učitavanje (*dynamic loading*) na početku svakog modula koji je predviđen za dinamičko učitavanje generiše tabelu adresa potprograma koji su definisani u tom modulu. Adrese su relativne u odnosu na početak modula. Na primer, za modul M u kome su definisani sledeći potprogrami:

```
void f(int);
int g(int,int);
double h(double,int);
```

prevodilac će na samom početku binarnog fajla sa prevodom koda tog modula generisati sledeću strukturu („tabelu“):

```
void* _funtbl[] = { &f, &g, &h };
```

Osim toga, u glavnom modulu datog programa koji se uvek inicijalno učitava pri kreiranju procesa nad tim programom, prevodilac generiše *stub* („patrljak“) za svaki potprogram koji je definisan u nekom modulu koji je predviđen za dinamičko učitavanje. Računar i operativni sistem na kome se izvršavaju ovakvi programi ne podržavaju virtuelnu memoriju. Operativni sistem obezbeđuje uslugu alokacije dela operativne memorije i učitavanja sadržaja iz fajla u taj deo memorije sledećim sistemskim pozivom:

```
void* alloc_and_load(char* filename);
```

Ovaj poziv vraća adresu u operativnoj memoriji gde je alociran prostor i učitani sadržaj fajla sa datim imenom, odnosno NULL ako je došlo do greške (npr. nema slobodnog prostora ili ne postoji fajl sa datim imenom).

Napisati kod na jeziku C za *stub* („patrljak“) funkcije *h* iz gore pomenutog modula *M* čiji je binarni oblik u fajlu „m.dlm“.

Rešenje

```
double h (double _1, int _2) {
    typedef double (*PFUN)(double,int);
    static PFUN _my_impl = NULL;
    if (_my_impl == NULL) {
        void* _m = alloc_and_load("m.dlm");
        if (_m == NULL) exit(1); // Error
        _my_impl = (PFUN)((int)((void**)_m)[2])+(int)_m
    }
    return _my_impl(_1,_2);
}
```

Preklopi

1. zadatak, kolokvijum, jul 2021.

- [Postavka](#)
- [Rešenje](#)

U nekom velikom binarnom fajlu zapisane su dve velike kvadratne matrice a i b elemenata tipa `int`, dimenzije $N \times N$ i obezbeđen prostor za smeštanje još jedne takve matrice c u koju treba upisati rezultat množenja $c = a \times b$. Korišćenjem principa preklopa (*overlay*), implementirati funkciju `mat_mul()` koja realizuje ovo množenje. N je velik broj tipa `size_t`, pa u memoriju ne mogu da stanu cele navedene matrice, već samo po jedna vrsta ili kolona svake od njih. Na raspolaganju su sledeće implementirane funkcije:

- `void load_arow (int (*)[], size_t i):` u niz na koji ukazuje prvi parametar iz fajla učitava vrstu matrice a datu drugim parametrom;
- `void load_bcol (int (*)[], size_t i):` u niz na koji ukazuje prvi parametar iz fajla učitava kolonu matrice b datu drugim parametrom;
- `void store_crow (int (*)[], size_t i):` iz niza na koji ukazuje prvi parametar u fajl upisuje vrstu matrice c datu drugim parametrom.

Ne treba otvarati dati fajl (on je već otvoren) niti ga zatvarati (zatvoriće ga pozivalac). Sve eventualne greške obrađuju date implementirane funkcije.

Rešenje

```
const size_t N = ...;

int ai[N], bj[N], ci[N];

void mat_mul() {
    for (size_t i = 0; i < N; i++) {
        load_arow(&ai, i);
        for (size_t j = 0; j < N; j++) {
            load_bcol(&bj, j);
            ci[j] = 0;
            for (size_t k = 0; k < N; k++) {
                ci[j] += ai[k] * bj[k];
            }
        }
        store_crow(&ci, i);
    }
}
```

2. zadatak, drugi kolokvijum, jun 2017.

- [Postavka](#)
- [Rešenje](#)

U nastavku je data implementacija jednog programa koji ciklično vrši neku obradu podeljenu u dve faze. Prva faza obrade koristi (čita i menja) podatke koji se samo koriste u toj fazi; svi ovakvi podaci grupisani su u jednu veliku strukturu podataka tipa `Phase1Data`; ali koristi i podatke koji su zajednički za obe faze, odnosno preko kojih obrade u ove dve faze razmenjuju informacije; ovi podaci grupisani su u strukturu tipa `CommonData`. Analogno radi i obrada u drugoj fazi.

Sa ciljem značajnog smanjenja potrebe za operativnom memorijom, potrebno je restrukturirati ovaj program tako da koristi preklap (*overlay*) u koji se smeštaju podaci prve, odnosno druge faze koji se ne koriste istovremeno. Za čuvanje izbačenog sadržaja treba kreirati dva privremena fajla u tekućem direktorijumu procesa. Za rad sa fajlovima na raspolaganju su sledeći sistemski pozivi; sve ove funkcije vraćaju 0 u slučaju uspeha, a negativnu vrednost u slučaju greške:

- `FILE fopen(char* filename)`: otvara fajl sa zadatim imenom za čitanje i upis; ako fajl ne postoji, kreira ga;
- `int fread(FILE file, int offset, int size, void* buffer)`: iz datog fajla, sa pozicije rednog broja bajta `offset` (numeracija počev od 0), čita niz bajtova dužine `size` u memoriju na lokaciju na koju ukazuje `buffer`; ukoliko se čitanjem prekorači granica sadržaja fajla, vraća grešku;
- `int fwrite(FILE file, int offset, int size, void* buffer)`: u dati fajl, na poziciju rednog broja bajta `offset` (numeracija počev od 0), upisuje niz bajtova dužine `size` sa lokacije na koju ukazuje `buffer`; ukoliko se upisom prekorači granica sadržaja fajla, sadržaj fajla se proširuje tako da primi sav upisani sadržaj;
- `int fclose(FILE file)`: zatvara dati fajl.

```

struct Phase1Data p1data;
struct Phase2Data p2data;
struct CommonData cdata;

void main () {
    init_cdata(&cdata);    // Initialize common data (cdata)
    init_p1data(&p1data);  // Initialize data for phase 1 (p1data)
    init_p2data(&p2data);  // Initialize data for phase 2 (p2data)

    do {
        process_phase_1(&p1data,&cdata);    // Perform phase 1 processing
        process_phase_2(&p2data,&cdata);    // Perform phase 2 processing
    } while (!cdata.completed);
}

```

Rešenje

```

union OverlaidData {
    struct Phase1Data p1data;
    struct Phase2Data p2data;
} data;
struct CommonData cdata;

int main () {
    FILE fd1 = fopen("overlay1.tmp");
    if (fd1<0) return fd1;
    FILE fd2 = fopen("overlay2.tmp");
    if (fd2<0) {
        fclose(fd1);
        return fd2;
    }

    int ret = 0;

    init_cdata(&cdata);
    init_p1data(&data.p1data);
    process_phase_1(&data.p1data,&cdata);
    if ((ret=fwrite(fd1,0,sizeof(data.p1data),&data.p1data))<0) {
        fclose(fd1);
        fclose(fd2);
        return ret;
    }

    init_p2data(&data.p2data);
    process_phase_2(&data.p2data,&cdata);

    while (!cdata.completed) {
        if ((ret=fwrite(fd2,0,sizeof(data.p2data),&data.p2data))<0) break;
    }
}

```

```

    if ((ret=fread(fd1,0,sizeof(data.p1data),&data.p1data))<0) break;
    process_phase_1(&data.p1data,&cdata);
    if ((ret=fwrite(fd1,0,sizeof(data.p1data),&data.p1data))<0) break;

    if ((ret=fread(fd2,0,sizeof(data.p2data),&data.p2data))<0) break;
    process_phase_2(&data.p2data,&cdata);
}

fclose(fd1);
fclose(fd2);
return ret;
}

```

2. zadatak, drugi kolokvijum, maj 2016.

- [Postavka](#)
- [Rešenje](#)

Neki sistem koristi preklope (*overlays*). Prevodilac i linker u generisanom kodu prevedenog programa koji koristi preklope statički alociraju i adekvatno inicijalizuju sledeće strukture podataka:

- za svaki modul (preklop, *overlay*) postoji sledeći deskriptor; moduli koji se preklapaju imaju istu početnu adresu:

```

struct OverlayDescr {
    const char* filename; // Naziv fajla u kome je binarni sadržaj preklopa
    void* addr; // Adresa u adresnom prostoru procesa na kojoj se nalazi
    bool isLoading; // Da li je preklop trenutno učitano?
}

```

- tabela svih preklopa-modula:

```

const int numOfOverlays = ...; // Ukupan broj preklopa
OverlayDescr overlays[numOfOverlays]; // Tabela svih preklopa

```

- svakom potprogramu koji se nalazi u nekom preklopu prevodilac pridružuje jedan jedinstveni ceo broj (identifikator), koji predstavlja ulaz u tabelu tih potprograma; svaki ulaz u ovoj tabeli sadrži pokazivač na deskriptor preklopa u kome se nalazi taj potprogram:

```

const int numOfProcs = ...; // Ukupan broj potprograma
OverlayDescr* procedureMap[numOfProcs]; // Tabela potprograma

```

Za svaki poziv potprograma koji se nalazi u nekom preklopu, na uvek istoj i prevodiocu poznatoj adresi u adresnom prostoru procesa, npr. `proc(a,b,c)`, prevodilac generiše kod koji je ekvivalent sledećeg koda:

```

ensureOverlay (proc_id); // proc_id je identifikator za proc
proc(a,b,c); // standardan poziv na poznatoj adresi

```

Funkcija `ensureOverlay(int procID)` treba da obezbedi da je potprogram sa datim identifikatorom prisutan u memoriji, odnosno po potrebi u čita njegov preklop. Potrebno je implementirati ovu funkciju, pri čemu je na raspolaganju sistemski poziv koji učitava binarni sadržaj iz fajla sa zadatim imenom na zadatu adresu u adresnom prostoru procesa:

```

void sys_loadBinary(const char* filename, void* address);

```

Rešenje

```

void ensureOverlay (int procedureID) {
    OverlayDescr* ovrl = procedureMap[procedureID];
    if (!ovrl->isLoading) {
        for (int i=0; i<numOfOverlays; i++)

```

```

        if (overlays[i].addr==ovrl->addr) overlays[i].isLoading = false;
        sys_loadBinary(ovrl->filename,ovrl->addr);
        ovrl->isLoading = true;
    }
}

```

3. zadatak, drugi kolokvijum, septembar 2013.

- [Postavka](#)
- [Rešenje](#)

Data je jedna realizacija klase `DArray` koja apstrahuje dinamički niz elemenata tipa `double` velike dimenzije zadate prilikom inicijalizacije parametrom `size`. U toj implementaciji, u svakom trenutku se u memoriji drži samo jedan keširani blok ovog niza veličine zadate parametrom `blockSize`. Implementacija koristi dinamičko učitavanje bloka u kome se nalazi element kome se pristupa uz zamenu (*swapping*), zapravo neku vrstu preklopa (*overlay*), tako da se u memoriji uvek nalazi samo jedan keširani blok kome se trenutno pristupa, dok se ceo niz nalazi u fajlu. Interfejs te klase je u njenom javnom delu.

U funkcijama `fread` i `fwrite` za pristup fajlu i učitavanje, odnosno snimanje datog niza elemenata tipa `double` na zadatu poziciju u fajlu, pozicija se izražava u jedinicama veličine binarnog zapisa tipa `double`, počev od 0, što znači da se fajl kroz ove funkcije može posmatrati kao veliki niz elemenata tipa `double`:

```

void fread (FHANDLE, int position, double* buffer, int bufferSize);
void fwrite(FHANDLE, int position, double* buffer, int bufferSize);

```

Modifikovati datu implementaciju ove klase (bez modifikacije njenog interfejsa) tako da se u memoriji nalaze uvek dva bloka iz niza u dva slotu (umesto u jednom, kako je sada), pri čemu se u jedan slot preslikavaju parni, a u drugi neparni blokovi niza.

```

class DArray {
public:
    inline DArray (int size, int blockSize, FHANDLE fromFile);

    inline double get (int i); // Get element [i]
    inline void set (int i, double x); // Set element [i]

protected:
    inline void save();
    inline void load(int blockNo);
    inline void fetch(int blockNo);

private:
    FHANDLE file;
    int size, blockSize;
    int curBlock;
    int dirty;
    double* block;
};

DArray::DArray (int s, int bs, FHANDLE f) :
    file(f), size(s), blockSize(bs), curBlock(0), dirty(0) {
    block = new double[bs];
    if (block) load(curBlock);
}

void DArray::save () {
    fwrite(file,curBlock*blockSize,block,blockSize);
    dirty=0;
}

```

```

}

void DArray::load (int b) {
    curBlock = b;
    fread(file, curBlock*blockSize, block, blockSize);
    dirty = 0;
}

void DArray::fetch(int b) {
    if (curBlock!=b) {
        if (dirty) save();
        load(b);
    }
}

double DArray::get (int i) {
    if (block==0 || i<0 || i>=size) return 0; // Exception
    fetch(i/blockSize);
    return block[i%blockSize];
}

void DArray::set (int i, double x) {
    if (block==0 || i<0 || i>=size) return; // Exception
    fetch(i/blockSize);
    if (block[i%blockSize]!=x) {
        block[i%blockSize]=x;
        dirty=1;
    }
}

```

Rešenje

```

class DArray {
public:
    DArray (int size, int blockSize, FHANDLE fromFile);

    inline double get (int i); // Get element [i]
    inline void set (int i, double x); // Set element [i]

protected:
    inline void save(int slot);
    inline void load(int blockNo, int slot);
    inline int fetch(int blockNo);

private:
    FHANDLE file;
    int size, blockSize;
    int curBlock[2];
    int dirty[2];
    double* block[2];
};

DArray::DArray (int s, int bs, FHANDLE f) :
    file(f), size(s), blockSize(bs) {
    for (int slot=0; slot<2; slot++) {
        curBlock[slot]=slot;
    }
}

```

```

    dirty[slot]=0;
    block[slot] = new double[bs];
    load(curBlock[slot],slot);
}
}

void DArray::save (int slot) {
    fwrite(file,curBlock[slot]*blockSize,block[slot],blockSize);
    dirty[slot]=0;
}

void DArray::load (int b, int slot) {
    curBlock[slot] = b;
    fread(file,curBlock[slot]*blockSize,block[slot],blockSize);
    dirty[slot] = 0;
}

int DArray::fetch(int b) {
    int slot = b%2;
    if (curBlock[slot]!=b) {
        if (dirty[slot]) save(slot);
        load(b,slot);
    }
    return slot;
}

double DArray::get (int i) {
    if (block==0 || i<0 || i>=size) return 0; // Exception
    int slot = fetch(i/blockSize);
    return block[slot][i%blockSize];
}

void DArray::set (int i, double x) {
    if (block==0 || i<0 || i>=size) return; // Exception
    int slot = fetch(i/blockSize);
    if (block[slot][i%blockSize]!=x) {
        block[slot][i%blockSize]=x;
        dirty[slot]=1;
    }
}
}

```

3. zadatak, drugi kolokvijum, maj 2012.

- [Postavka](#)
- [Rešenje](#)

Neki program treba da izračuna proizvod dve ogromne matrice $A[m \times k] \times B[k \times n] = C[m \times n]$, gde su dimenzije matrica veoma velike (i po nekoliko miliona). Predložiti i precizno opisati kako biste primenili tehniku preklopa (*overlays*) u ovom programu. Posebno objasniti način smeštanja matrica u preklope i redosled učitavanja u preklope.

Rešenje

Dovoljno je, na primer, da se samo jedna vrsta (i) matrice A smesti u jedan preklop. U drugi preklop smeštena je jedna kolona (j) matrice B. Tako se iz njih može izračunati $C[i, j]$. Ako se u unutrašnjoj petlji varira j ($j:=1..n$), u jednoj iteraciji spoljašnje petlje dobija se jedna vrsta i matrice C. Ova vrsta je u svom, trećem preklopu. U narednoj iteraciji spoljašnje petlje po i , u prvi preklop biće učitana nova vrsta matrice A, u drugom preklopu biće učitavana

jedna po jedna kolona ($j:=1..n$) matrice B u unutrašnjoj petlji, a u trećem preklopu biće izračunavana nova vrsta matrice C.

3. zadatak, drugi kolokvijum, maj 2011.

- [Postavka](#)
- [Rešenje](#)

U potpunosti realizovati klasu koja apstrahuje dinamički niz elemenata tipa `double` velike dimenzije zadate prilikom inicijalizacije parametrom `size`. U svakom trenutku se u memoriji drži samo jedan keširani blok ovog niza veličine zadate parametrom `blockSize`. Implementacija treba da koristi dinamičko učitavanje bloka u kome se nalazi element kome se pristupa uz zamenu (*swapping*), zapravo neku vrstu preklopa (*overlay*), tako da se u memoriji uvek nalazi samo jedan keširani blok kome se trenutno pristupa, dok se ceo niz nalazi u fajlu. Interfejs ove klase treba da bude:

```
class DLArray {
public:
    DLArray (int size, int blockSize, FHANDLE fromFile);
    double get (int i); // Get element [i]
    void set (int i, double x); // Set element [i]
};
```

Inicijalno se vrednosti celog niza nalaze u binarnom fajlu zadatom trećim argumentom (otvaranje i zatvaranje tog fajla je u odgovornosti korisnika ove klase). Na raspolaganju su sledeće funkcije za pristup fajlu i učitavanje, odnosno snimanje datog niza elemenata tipa `double` na zadatu poziciju u fajlu. Pozicija se izražava u jedinicama veličine binarnog zapisa tipa `double`, počev od 0, što znači da se fajl kroz ove funkcije može posmatrati kao veliki niz elemenata tipa `double`:

```
void fread (FHANDLE, int position, double[], int bufferSize);
void fwrite (FHANDLE, int position, double[], int bufferSize);
```

Rešenje

```
class DLArray {
public:
    inline DLArray (int size, int blockSize, FHANDLE fromFile);

    inline double get (int i); // Get element [i]
    inline void set (int i, double x); // Set element [i]

protected:
    inline void save();
    inline void load(int blockNo);
    inline void fetch(int blockNo);

private:
    FHANDLE file;
    int size, blockSize;
    int curBlock;
    int dirty;
    double* block;
};

DLArray::DLArray (int s, int bs, FHANDLE f) :
    file(f), size(s), blockSize(bs), curBlock(0), dirty(0) {
    block = new double[bs];
    if (block) load(curBlock);
}
```

```

void DArray::save () {
    fwrite(file,curBlock*blockSize,block,blockSize);

    dirty=0;
}

void DArray::load (int b) {
    curBlock = b;
    fread(file,curBlock*blockSize,block,blockSize);
    dirty = 0;
}

void DArray::fetch(int b) {
    if (curBlock!=b) {
        if (dirty) save();
        load(b);
    }
}

double DArray::get (int i) {
    if (block==0 || i<0 || i>=size) return 0; // Exception
    fetch(i/blockSize);
    return block[i%blockSize];
}

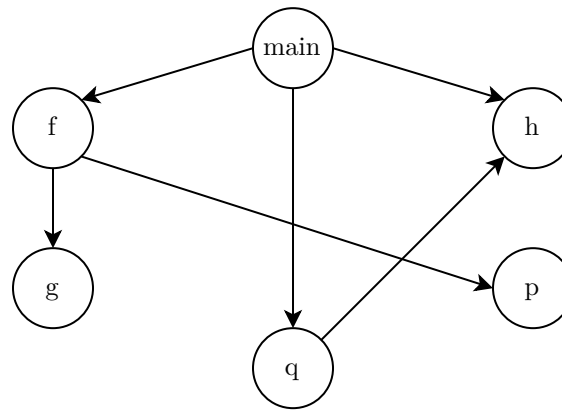
void DArray::set (int i, double x) {
    if (block==0 || i<0 || i>=size) return; // Exception
    fetch(i/blockSize);
    if (block[i%blockSize]!=x) {
        block[i%blockSize]=x;
        dirty=1;
    }
}

```

3. zadatak, drugi kolokvijum, maj 2010.

- [Postavka](#)
- [Rešenje](#)

1. Ukoliko prevodilac generiše samo uobičajeni kod za povratak iz potprograma (skidanje samo sačuvane povratne adrese sa steka i skok na tu adresu), da li se dve procedure, od kojih jedna poziva onu drugu, mogu nalaziti u dva različita modula-preklopa (*overlays*) koji se učitavaju na isto mesto (jedan preko drugog)? Obrazložiti odgovor.
2. Na slici je dat graf poziva potprograma nekog programa. Čvorovi grafa predstavljaju potprograme, a grana je usmerena od pozivaoca prema pozvanom potprogramu. Za pretpostavke u tački a) i na osnovu zaključka iz te tačke, odgovoriti da li je korektna svaka od sledeće dve konfiguracije preklopa.



Slika 2: Graf

1. Konfiguracija 1:
 - Modul *A*: *main*, *h*
 - Modul *B*: *f*, *g*, *p*
 - Modul *C*: *q*
 - Preklapaju se modul *B* i modul *C*, modul *A* je uvek učitán. Korektna?
2. Konfiguracija 2:
 - Modul *A*: *main*, *p*
 - Modul *B*: *f*, *g*
 - Modul *C*: *q*, *h*
 - Preklapaju se modul *B* i modul *C*, modul *A* je uvek učitán. Korektna?

Rešenje

1. Ne. Kada se iz pozivajućeg potprograma pozove onaj drugi, na isto mesto modula kome pripada pozivajući potprogram se učitava modul u kome je pozvani potprogram. Kada se vrši povratak iz tog pozvanog potprograma, ukoliko prevodilac generiše samo kod za jednostavni indirektni skok preko adrese skinute sa steka, skok će biti na adresu unutar istog modula, a ne na kod unutar pozivajućeg potprograma, jer je on u modulu koji je izbačen, što nije korekno. Prema tome, potprogrami koji su u relaciji pozivalac-pozvani se mogu nalaziti ili u istom modulu, ili u dva modula koji se ne preklapaju (ne učitavaju na isto mesto jedan preko drugog).
2. Obe konfiguracije su korektne, pošto je za sve grane zadovoljen uslov iz zaključka prethodne tačke.